Capítulo 5.

Machine Learning 1- Aprendizaje Supervisado

Curso de IA

Descripción del capítulo

Objetivos del capítulo

- ✔ Ser capaz de introducir análisis de datos basados en Machine Learning de acuerdo con el objetivo, la estrategia y la política del negocio y gestionar el proceso general.
- ✔ Ser capaz de seleccionar y aplicar un algoritmo Machine Learning que sea el más adecuado para el problema dado y realizar un ajuste de hiperparámetros.
- ✔ Ser capaz de diseñar, mantener y optimizar un flujo de trabajo Machine Learning para el modelado de IA utilizando datos estructurados y no estructurados.

Contenido del capítulo

- ✔ Unidad 1. Análisis de datos basado en Machine Learning
- ✔ Unidad 2. Aplicación del Modelo de Aprendizaje Supervisado para la Predicción Numérica
- ✔ Unidad 3. Aplicación del Modelo de Aprendizaje Supervisado para la Clasificación
- ✔ Unidad 4. Árbol de Decisión
- ✔ Unidad 5. Algoritmo ingenuo de Bayes
- ✔ Unidad 6. Algoritmo KNN
- ✔ Unidad 7. Algoritmo SVM
- ✔ Unidad 8. Algoritmo de conjunto

Unidad 1.

Análisis de datos basado en Machine Learning

- 1.1. ¿Qué es Machine Learning?
- 1.2. Python scikit-aprende library para Machine Learning
- 1.3. Preparación y división del conjunto de datos.
- 1.4. Preprocesamiento de datos para crear un buen conjunto de datos de entrenamiento
- 1.5. Practicando para encontrar un método óptimo para resolver problemas con scikit-learn

¿Qué es Machine Learning?

- ¿Qué es Machine Learning?



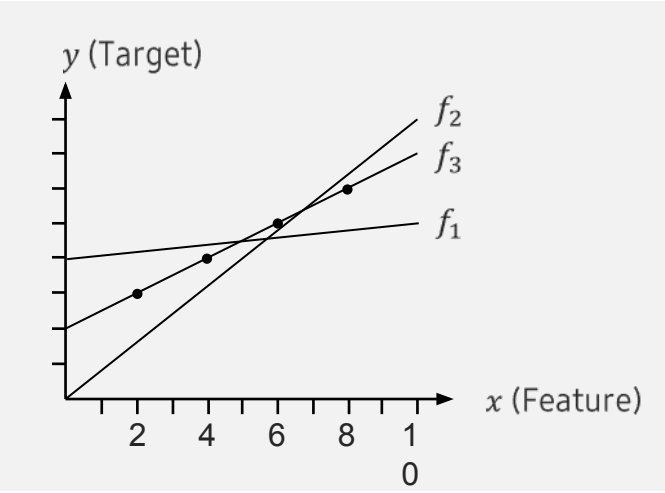
- Un modelo estadístico que aprende de los datos..
- Un modelo bastante simple puede hacer predicciones complejas.

1.1. ¿Qué es Machine Learning?

- La definición de Samuel en la fase inicial de la inteligencia artificial.
  - "Programar computadoras para aprender de la experiencia debería eventualmente eliminar la necesidad de gran parte de este esfuerzo de programación detallada". –Samuel, 1959

- definición moderna
  - "Se dice que un programa de computadora aprende de la experiencia E con respecto a alguna clase de tareas T y medida de desempeño P, si su desempeño en las tareas en T, medido por P, mejora con la experiencia E". – Mitchell, 1997 (p.2)
  - "Programar computadoras para optimizar un criterio de desempeño utilizando datos de ejemplo o experiencias pasadas". – Alpaydin, 2010
    – , 2010
  - "Métodos computacionales que utilizan la experiencia para mejorar el rendimiento o hacer predicciones precisas". –Mohri, 2012, 2012

- Definición matemática
  - Supongamos que el eje x son los gastos de publicidad invertidos mientras que el eje y son las ventas.

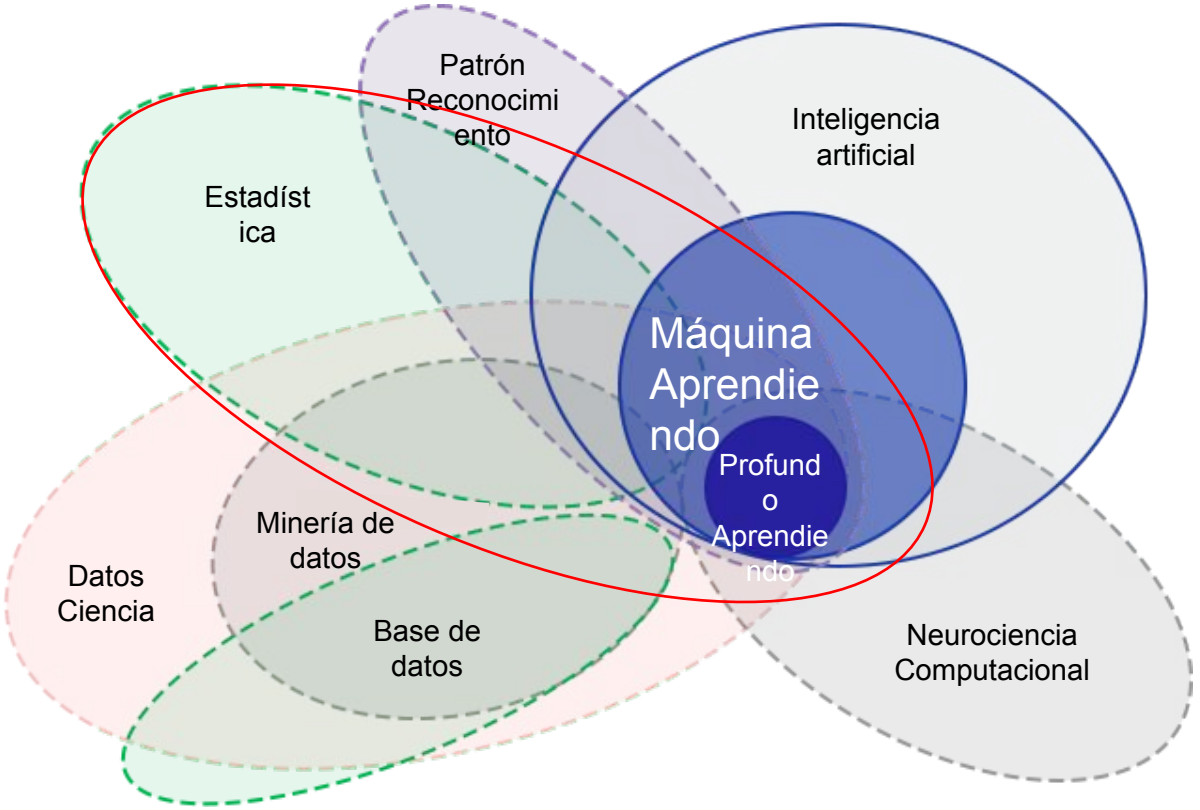


  - Pregunta sobre la predicción: ¿Cuáles son las ventas cuando se dan los gastos de publicidad aleatorios?
  - Regresión lineal
    - w y b como parámetros

$$y = \underline{w}x + \underline{b}$$

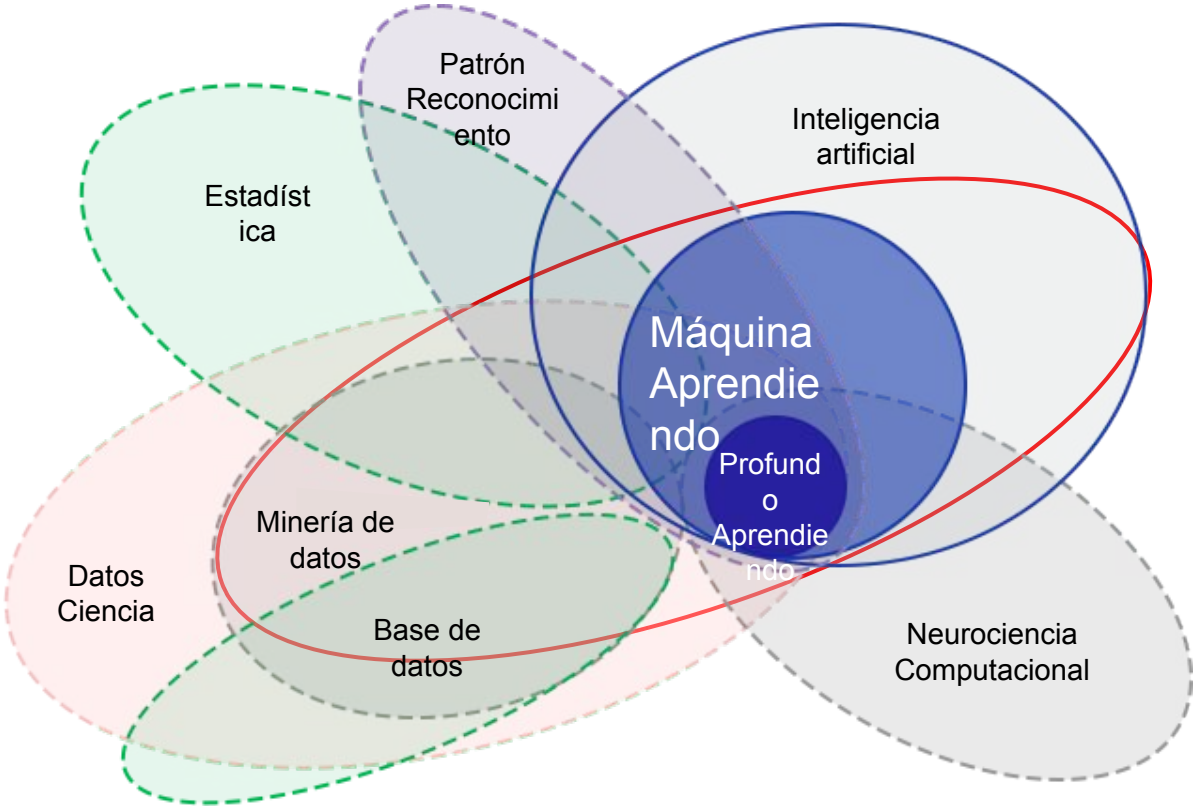
    - 'w'" se usa comúnmente como abreviatura de "peso".
  - Dado que al principio se desconoce el valor óptimo, comience con un valor arbitrario y luego alcance el valor óptimo mejorando gradualmente el rendimiento.
    - Del gráfico, comienza desde f1 para continuar como f1 → f2 → f3.
    - El valor óptimo es f3 donde w=0.5 y b=2.0.

•Estadísticas y Machine Learning desde una perspectiva de análisis de datos



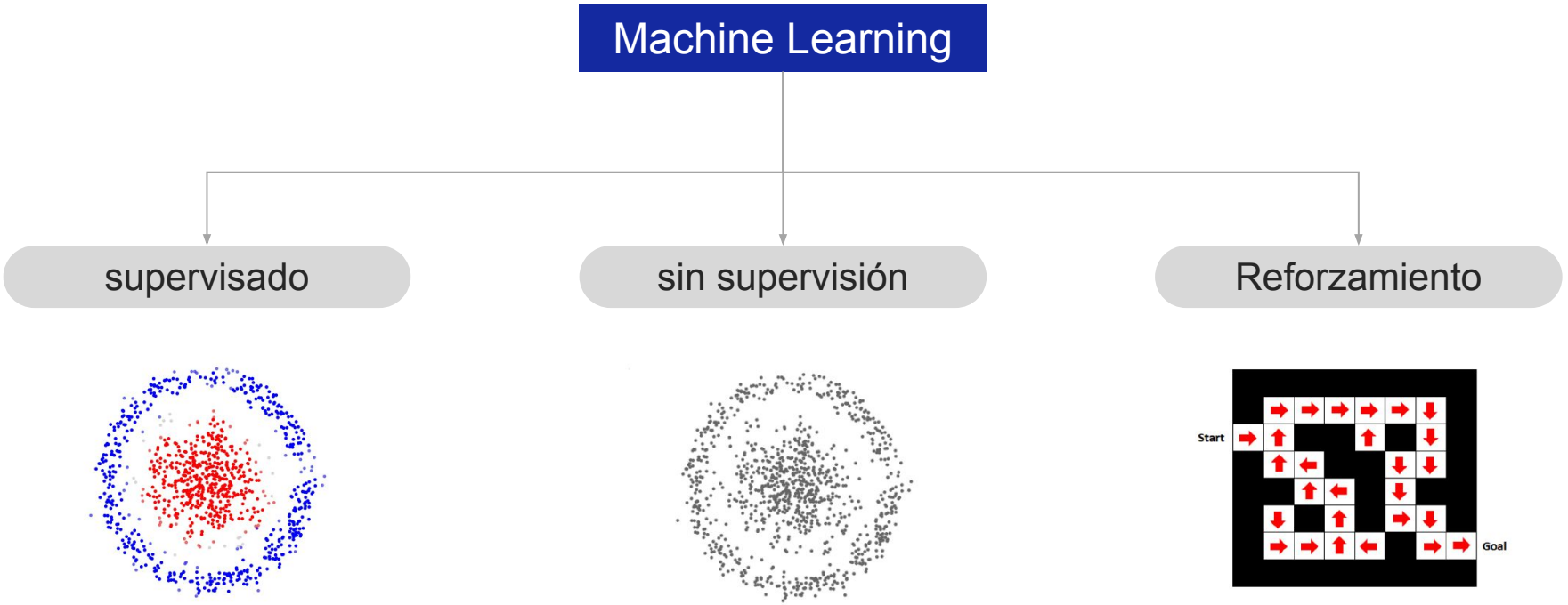
Conexión entre Machine Learning y diferentes tipos de estudio.

- Minería de datos y Machine Learning desde una perspectiva de análisis de datos



Conexión entre Machine Learning y diferentes tipos de estudio.

- Tipos de Machine Learning según métodos de supervisión

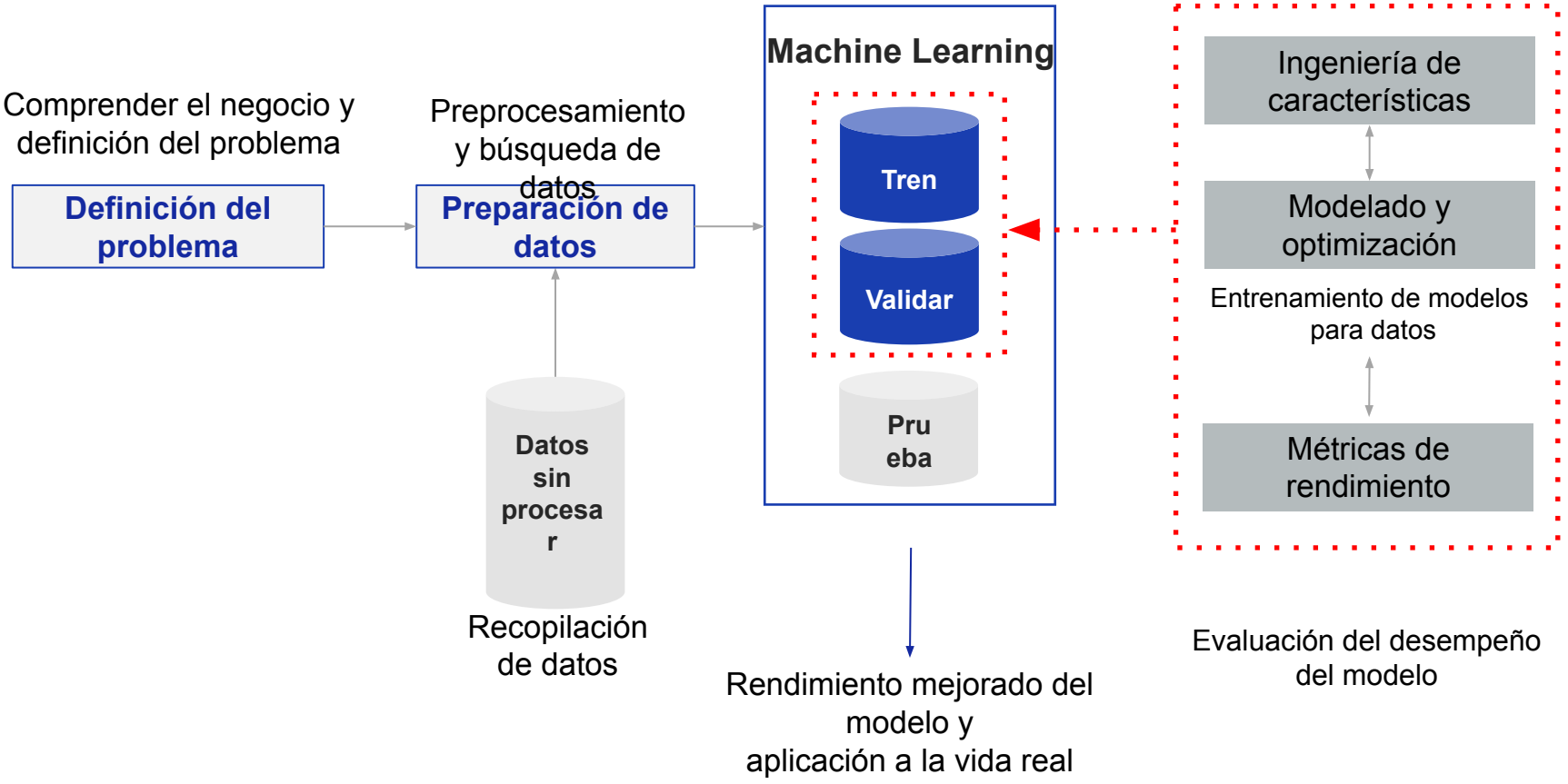


Se proporciona el patrón objetivo.

Se debe descubrir el patrón objetivo.

Optimización de políticas

- Flujo de trabajo Machine Learning



•Machine Learning tipos:

Tipo	Algoritmo/Método
Aprendizaje no supervisado	Agrupación
	MDS, t-SNE
	PCA, NMF
	Análisis de asociación
Aprendizaje supervisado	Regresión lineal
	Regresión logística
	Árbol, Bosque aleatorio, Ada Boost, XGBoost
	Bayes ingenuo
	knn
	Máquina de vectores de soporte (SVM)
	Red neuronal

Parámetros frente a hiperparámetros

- Parámetros
  - Aprendido de los datos mediante entrenamiento y no configurado manualmente por el practicante.
  - Contiene el patrón de datos.

    Ex Coeficientes de regresión lineal.

    Ex Pesos de la red neuronal

- Hiperparámetros
  - **Puede ser configurado manualmente por el profesional.**
  - Se puede ajustar para optimizar el rendimiento de Machine Learning.

    Ex k en KNN algoritmo

    Ex Tasa de aprendizaje en red neuronal.

    Ex Profundidad máxima en el algoritmo de árbol

Unidad 1.

Análisis de datos basado en Machine Learning

- 1.1. ¿Qué es Machine Learning?
- 1.2. Python scikit-aprende library para Machine Learning
- 1.3. Preparación y división del conjunto de datos.
- 1.4. Preprocesamiento de datos para crear un buen conjunto de datos de entrenamiento
- 1.5. Practicando para encontrar un método óptimo para resolver problemas con scikit-learn

Características de scikit-learn library

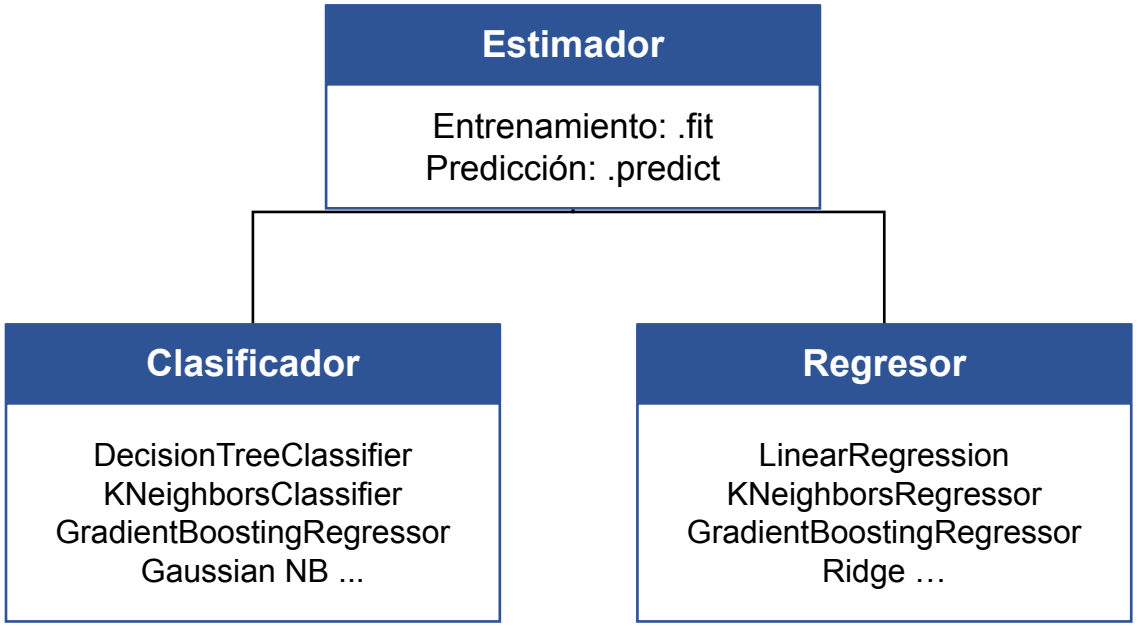
- Características
  - Interfaz library integrada mediante la aplicación del patrón de diseño de fachada
  - Instalado con varios tipos de algoritmos Machine Learning, selección de modelos y funciones de preprocesamiento de datos.
  - Herramienta simple y eficiente para analizar datos pronosticados.
  - Basado en NumPy, SciPy y matplotlib
  - Fácilmente accesible y reutilizable en muchas situaciones diferentes.
  - Altamente compatible con diferentes libraries
  - No es compatible con GPU
  - Puede utilizarse como código abierto y con fines comerciales.

Mecanismo de scikit-learn

- scikit-learn se caracteriza por su interfaz intuitiva y sencilla, completa con API de alto nivel.



- Estimador, Clasificador, Regresor
  - Estimador se refiere a un objeto que puede ajustarse al modelo y deducir ciertas características de nuevos datos basándose en los datos de entrenamiento. of on
  - Clasificador se refiere a una clase que implementa un algoritmo de clasificación, mientras que regresión se refiere a una clase que implementa un algoritmo de regresión.



Scikit-learn library

- Acerca de scikit-learn library
  - Es un Python representativo Machine Learning library.
  - Para importar un algoritmo Machine Learning como clase:

    de sklearn.<familia> importar <algoritmo Machine Learning>.<>

    Ex de sklearn.linear_model importar LinearRegression
  - Los hiperparámetros se especifican cuando se crea una instancia del objeto Machine Learning:

    Ex myModel = KNeighborsClassifier(n_neighbors=10) # KNN with k = 10

- Acerca de scikit-learn library
  - Para entrenar un modelo de aprendizaje supervisado: myModel.fit(X_train, Y_train)
  - Para entrenar un modelo de aprendizaje no supervisado: myModel.fit(X_train)
  - Para predecir usando un modelo ya entrenado: myModel.predict(X_test)
  - Para importar un preprocesador como clase: from sklearn.preprocessing import <a preprocessor>
  - Para dividir el conjunto de datos en un conjunto de entrenamiento y un conjunto de prueba:

    X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.3, random_state=123)
  - Para calcular una métrica de rendimiento (precisión): metrics.accuracy_score(Y_test, Y_pred)
  - Para realizar una validación cruzada y realizar un ajuste de hiperparámetros al mismo tiempo:

    myGridCV = GridSearchCV(estimator, parameter_grid, cv=k)

    myGridCV.fit(X_train, Y_train)

- Practicando scikit-learn
  - El módulo sklearn.datasets incluye utilidades para cargar conjuntos de datos, incluidos métodos para cargar y recuperar conjuntos de datos de referencia populares. También cuenta con algunos generadores de datos artificiales.

```
[1]: 1 from sklearn.datasets import load_breast_cancer
```

  - Importe datos con load_breast_cancer().().

```
[2]: 1 type(load_breast_cancer())

[2]: sklearn.utils._bunch.Bunch

[3]: 1 data=load_breast_cancer()
```

  - Objeto contenedor que expone las claves como atributos.
  - Los objetos Bunch se utilizan a veces como salida para funciones y métodos. Extienden los diccionarios al permitir que se acceda a los valores mediante una clave, bunch["valor_clave"], o mediante un atributo, bunch.valor_clave.

- Practicando scikit-learn

[4]:
```
1 import pandas as pd
2 pd.DataFrame(data['data'])
```

[4]:

	0	1	2	3	4	5	6	7	8	9	...	20	21	22	23	24	25	26	27	28	29
0	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.30010	0.14710	0.2419	0.07871	...	25.380	17.33	184.60	2019.0	0.16220	0.66560	0.7119	0.2654	0.4601	0.11890
1	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.08690	0.07017	0.1812	0.05667	...	24.990	23.41	158.80	1956.0	0.12380	0.18660	0.2416	0.1860	0.2750	0.08902
2	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.19740	0.12790	0.2069	0.05999	...	23.570	25.53	152.50	1709.0	0.14440	0.42450	0.4504	0.2430	0.3613	0.08758
3	11.42	20.38	77.58	386.1	0.14250	0.28390	0.24140	0.10520	0.2597	0.09744	...	14.910	26.50	98.87	567.7	0.20980	0.86630	0.6869	0.2575	0.6638	0.17300
4	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.19800	0.10430	0.1809	0.05883	...	22.540	16.67	152.20	1575.0	0.13740	0.20500	0.4000	0.1625	0.2364	0.07678
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
564	21.56	22.39	142.00	1479.0	0.11100	0.11590	0.24390	0.13890	0.1726	0.05623	...	25.450	26.40	166.10	2027.0	0.14100	0.21130	0.4107	0.2216	0.2060	0.07115
565	20.13	28.25	131.20	1261.0	0.09780	0.10340	0.14400	0.09791	0.1752	0.05533	...	23.690	38.25	155.00	1731.0	0.11660	0.19220	0.3215	0.1628	0.2572	0.06637
566	16.60	28.08	108.30	858.1	0.08455	0.10230	0.09251	0.05302	0.1590	0.05648	...	18.980	34.12	126.70	1124.0	0.11390	0.30940	0.3403	0.1418	0.2218	0.07820
567	20.60	29.33	140.10	1265.0	0.11780	0.27700	0.35140	0.15200	0.2397	0.07016	...	25.740	39.42	184.60	1821.0	0.16500	0.86810	0.9387	0.2650	0.4087	0.12400
568	7.76	24.54	47.92	181.0	0.05263	0.04362	0.00000	0.00000	0.1587	0.05884	...	9.456	30.37	59.16	268.6	0.08996	0.06444	0.0000	0.0000	0.2871	0.07039

569 rows × 30 columns

Línea 4

- Estos datos se convierten en x (variable independiente, datos).

- Practicando scikit-learn

```
[5]: 1 pd.DataFrame(data['target'])
```

[5]:

	0
0	0
1	0
2	0
3	0
4	0
...	...
564	0
565	0
566	0
567	0
568	1

569 rows × 1 columns

Línea 5

- Estos datos se convierten en y (variable dependiente, valor real).

- Practicando scikit-learn

```
[6]: 1 print(data.DESCR)
```

```
[7]: 1 from sklearn.model_selection import train_test_split
     2
     3 help(train_test_split)
```

Línea 6

- Proporciona detalles sobre los datos.

Línea 7

- La ayuda muestra que el valor predeterminado de test_size es 0,25. is 0.25.

- Practicando scikit-learn

```
[8]:  1 len(data.data)

[8]: 569
```

Línea 8

- Del total de 569 valores observados, divida los datos para entrenamiento y evaluación en 7:3 u 8:2. 7,5:2,5 es el valor predeterminado.

- Practicando scikit-learn
  - Utilice train_test_split() para dividir los datos para crear y evaluar el modelo.

```
[9]:  1 # train_test_split
```

```
[10]: 1 569*0.75
```

```
[10]: 426.75
```

```
[11]: 1 from sklearn.model_selection import train_test_split
```

```
[12]: 1 X_train, X_test, y_train, y_test = train_test_split(data.data, data.target)
```

- Practicando scikit-learn

```
[13]: 1 len(X_train)

[13]: 426

[14]: 1 569*0.25

[14]: 142.25

[15]: 1 len(X_test)

[15]: 143
```

Línea 13

- Se encuentran 426 valores observados (75%) de un total de 569 observaciones.

Línea 15

- Se encuentran 143 valores observados (25%) de un total de 569 observaciones.

- Practicando scikit-learn
  - Para crear instancias, utilice el hiperparámetro del modelo como argumento. El hiperparámetro es una opción que requiere una configuración humana y afecta en gran medida el rendimiento del modelo.

[16]:
```
1 from sklearn.datasets import load_breast_cancer
2 from sklearn.model_selection import train_test_split
3 from sklearn.tree import DecisionTreeClassifier
4
5 data=load_breast_cancer()
6 X_train, X_test, y_train, y_test = train_test_split(data.data, data.target, random_state=42)
7
8 model = DecisionTreeClassifier(criterion='entropy')
9 model
```

[16]:
```
▾ DecisionTreeClassifier
DecisionTreeClassifier(criterion='entropy')
```

Línea 16-5
- Cargando el conjunto de datos de prueba

Línea 16-8
- Instancia del estimador y configuración de hiperparámetros
- Inicialización del modelo utilizando la entropía para la ramificación.

- adaptar
  - Utilice el método de ajuste con un estimador de instancia para el entrenamiento. Envíe los datos de entrenamiento y los datos de etiqueta juntos como argumento para el algoritmo de aprendizaje supervisado.

- predecir
  - El estimador de instancia que ha completado el entrenamiento con ajuste se puede aplicar con el método de predicción. 'Predecir' convierte los resultados estimados del modelo con respecto a los datos ingresados.

```
[17]: 1 model.fit(X_train, y_train)
      2 y_pred=model.predict(X_test)
      3 y_pred
```

```
[17]: array([1, 0, 0, 1, 1, 0, 0, 0, 0, 1, 1, 0, 1, 1, 1, 0, 1, 1, 1, 0, 1, 1,
             0, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1,
             1, 1, 1, 1, 1, 1, 0, 0, 1, 1, 1, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 1,
             0, 0, 1, 1, 0, 0, 1, 0, 1, 1, 1, 1, 1, 1, 0, 1, 1, 0, 0, 0, 0, 0,
             1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 1, 0, 0, 1, 0, 0, 1, 1, 1, 0, 1, 1,
             0, 1, 1, 0, 1, 0, 1, 1, 1, 0, 1, 1, 1, 0, 1, 0, 0, 1, 1, 0, 0, 0,
             1, 1, 1, 0, 0, 1, 1, 0, 1, 0, 1])
```

Línea 17

- Es un valor estimado, por lo que el valor real de X_test puede variar. Mida la precisión comparando los dos valores.

- Practicando scikit-learn

```
[18]: 1 pred=model.predict(X_test)

[19]: 1 pred_2d=pred.reshape(len(pred), 1)

[20]: 1 pred_2d
```

```
[20]: array([[1],
             [0],
             [0],
             [1],
             [1],
             [0],
             [0],
             [0],
             [0],
             [1],
             [1],
             [0],
             [1],
             [1],
             [1],
             [0],
             [1],
             [1],
```

- Practicando scikit-learn

```
[21]: 1 import numpy as np

[22]: 1 y_test_2d=y_test.reshape(len(pred), 1)

[23]: 1 df1=pd.DataFrame(pred_2d, columns=['pred'])

[24]: 1 df2=pd.DataFrame(y_test_2d, columns=['real'])
```

- Practicando scikit-learn

```
[25]: 1 pd.concat([df1, df2], axis=1)
```

[25]:

	pred	real			
0	1	1	138	1	1
1	0	0	139	0	0
2	0	0	140	1	1
3	1	1	141	0	0
4	1	1	142	1	1
...	...	...			

143 rows × 2 columns

Línea 25

- El marco de datos muestra un resultado en el que el valor previsto y el valor real difieren.

- Practicando scikit-learn

```
[26]: 1 df_concat = pd.concat([df1, df2], axis=1)

[27]: 1 df_concat.columns

[27]: Index(['pred', 'real'], dtype='object')
```

- Practicando scikit-learn

[28]:
```
1 df_concat[df_concat['pred'] == df_concat['real']]
```

[28]:

	pred	real			
0	1	1	**138**	1	1
1	0	0	**139**	0	0
2	0	0	**140**	1	1
3	1	1	**141**	0	0
4	1	1	**142**	1	1
...	...	...			

137 rows × 2 columns

- Practicando scikit-learn

```
[29]: 1 len(df_concat[df_concat['pred'] == df_concat['real']])

[29]: 137

[30]: 1 137/143

[30]: 0.958041958041958
```

Línea 30

- 137/143

- Practicando scikit-learn

$$x = \frac{x-m}{\sigma} \ (m: \text{average}, \sigma: \text{standard deviation})$$

- Mostró una precisión del 96%, lo cual es un resultado bastante raro. De hecho, se requiere un proceso para aumentar la precisión de los datos durante el preprocesamiento de los mismos, y la estandarización es una de las opciones. A continuación se presenta un breve resumen de la estandarización.
  - La estandarización se puede realizar calculando la distribución normal estándar. Otro término para la estandarización es transformación z, y el valor estandarizado también se conoce como puntuación z. Se obtendría una precisión del 97% a partir de la clasificación de vinos KNN mediante la estandarización.
  - La estandarización se usa ampliamente en el preprocesamiento de datos en general, además de KNN, y la siguiente es la ecuación.

$$x = \frac{x-m}{\sigma} \ (m: \text{average}, \sigma: \text{standard deviation})$$

  - La estandarización está disponible como clase StandardScaler en scikit-learn.

- Practicando scikit-learn

```
[31]: 1 from sklearn.preprocessing import StandardScaler

[32]: 1 scaler=StandardScaler()

[33]: 1 X_train
```

```
[33]: array([[1.289e+01, 1.312e+01, 8.189e+01, ..., 5.366e-02, 2.309e-01,
        6.915e-02],
       [1.340e+01, 2.052e+01, 8.864e+01, ..., 2.051e-01, 3.585e-01,
        1.109e-01],
       [1.296e+01, 1.829e+01, 8.418e+01, ..., 6.608e-02, 3.207e-01,
        7.247e-02],
       ...,
       [1.429e+01, 1.682e+01, 9.030e+01, ..., 3.333e-02, 2.458e-01,
        6.120e-02],
       [1.398e+01, 1.962e+01, 9.112e+01, ..., 1.827e-01, 3.179e-01,
        1.055e-01],
       [1.218e+01, 2.052e+01, 7.722e+01, ..., 7.431e-02, 2.694e-01,
        6.878e-02]])
```

- Practicando scikit-learn

```
[34]: 1 df_scale_previous=pd.DataFrame(data['data'])

[35]: 1 scaler.fit(X_train)

[35]: ▾ StandardScaler
      StandardScaler()

[36]: 1 X_train=scaler.transform(X_train)

[37]: 1 df_scale_after=pd.DataFrame(X_train)
```

Línea 34

- Marco de datos antes de la estandarización

- Practicando scikit-learn

```
[38]: 1 df_scale_previous
```

[38]:

	0	1	2	3	4	5	6	7	8	9	...	20	21	22	23	24	25	26	27	28	29
0	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.30010	0.14710	0.2419	0.07871	...	25.380	17.33	184.60	2019.0	0.16220	0.66560	0.7119	0.2654	0.4601	0.11890
1	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.08690	0.07017	0.1812	0.05667	...	24.990	23.41	158.80	1956.0	0.12380	0.18660	0.2416	0.1860	0.2750	0.08902
2	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.19740	0.12790	0.2069	0.05999	...	23.570	25.53	152.50	1709.0	0.14440	0.42450	0.4504	0.2430	0.3613	0.08758
3	11.42	20.38	77.58	386.1	0.14250	0.28390	0.24140	0.10520	0.2597	0.09744	...	14.910	26.50	98.87	567.7	0.20980	0.86630	0.6869	0.2575	0.6638	0.17300
4	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.19800	0.10430	0.1809	0.05883	...	22.540	16.67	152.20	1575.0	0.13740	0.20500	0.4000	0.1625	0.2364	0.07678
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
564	21.56	22.39	142.00	1479.0	0.11100	0.11590	0.24390	0.13890	0.1726	0.05623	...	25.450	26.40	166.10	2027.0	0.14100	0.21130	0.4107	0.2216	0.2060	0.07115
565	20.13	28.25	131.20	1261.0	0.09780	0.10340	0.14400	0.09791	0.1752	0.05533	...	23.690	38.25	155.00	1731.0	0.11660	0.19220	0.3215	0.1628	0.2572	0.06637
566	16.60	28.08	108.30	858.1	0.08455	0.10230	0.09251	0.05302	0.1590	0.05648	...	18.980	34.12	126.70	1124.0	0.11390	0.30940	0.3403	0.1418	0.2218	0.07820
567	20.60	29.33	140.10	1265.0	0.11780	0.27700	0.35140	0.15200	0.2397	0.07016	...	25.740	39.42	184.60	1821.0	0.16500	0.86810	0.9387	0.2650	0.4087	0.12400
568	7.76	24.54	47.92	181.0	0.05263	0.04362	0.00000	0.00000	0.1587	0.05884	...	9.456	30.37	59.16	268.6	0.08996	0.06444	0.0000	0.0000	0.2871	0.07039

569 rows × 30 columns

Línea 38

- Las diferencias entre los valores de las columnas eran enormes antes de la estandarización.

- Practicando scikit-learn

```
[39]: 1 df_scale_after
```

[39]:

	0	1	2	3	4	5	6	7	8	9	...	20	21	22	23	24	25	26	27	28	29
0	-0.349138	-1.438513	-0.411726	-0.390479	-1.863662	-1.268607	-0.826171	-0.952866	-1.729368	-0.941541	...	-0.542685	-1.655655	-0.589864	-0.525560	-1.510669	-0.891500	-0.750217	-0.916711	-0.925086	-0.808411
1	-0.204687	0.312640	-0.133673	-0.275880	1.078073	0.863546	0.726314	0.898441	1.178770	1.474377	...	0.041487	0.689899	0.194128	-0.051934	1.129415	0.923942	1.222217	1.436560	1.149559	1.569111
2	-0.329312	-0.215072	-0.317394	-0.364357	-1.579880	-0.457451	-0.597310	-0.764588	0.275343	-0.501024	...	-0.435901	-0.148985	-0.320159	-0.446032	-1.634396	-0.106752	-0.539891	-0.723713	0.534970	-0.619348
3	1.027403	2.089824	1.046922	0.917584	0.316303	0.562037	1.048527	0.930437	-0.325697	-0.477474	...	1.113515	2.165006	1.165793	0.997696	0.383604	0.860948	1.872819	1.310691	0.152884	0.421636
4	1.828969	0.696001	1.763681	1.783821	-0.333674	0.628175	0.974660	1.265740	-0.131572	-1.713139	...	1.471556	0.387568	1.556276	1.385595	-0.577759	0.296680	0.595768	1.232995	0.050452	-1.406351
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
421	-1.482661	-1.078817	-1.363286	-1.154195	0.162946	0.984150	-0.017962	-0.503063	0.379872	3.767006	...	-1.356547	-1.634059	-1.341770	-1.043821	-0.452739	-0.027674	-0.625430	-1.006838	-1.014510	1.426745
422	-0.703187	-0.205607	-0.688544	-0.680910	1.336057	-0.015692	-0.213582	-0.340964	0.002822	0.359231	...	-0.643188	0.623452	-0.647074	-0.623509	1.620874	0.122440	0.098634	0.142137	-0.117015	0.434165
423	0.047396	-0.562937	-0.065292	-0.062691	-2.240606	-1.473633	-1.021664	-1.097247	-1.090996	-1.225522	...	-0.272584	-0.806803	-0.376764	-0.334976	-1.962898	-1.322676	-1.152453	-1.232624	-0.682827	-1.261137
424	-0.040408	0.099662	-0.031514	-0.155663	0.748426	0.209953	0.320044	0.446522	-0.489956	0.392478	...	0.173397	0.879270	0.212290	-0.007830	1.297546	0.730938	0.700428	1.088480	0.489445	1.261600
425	-0.550238	0.312640	-0.604098	-0.551143	-1.105476	-1.208500	-0.810506	-0.794470	-0.228635	-0.808555	...	-0.601311	1.218146	-0.675225	-0.577280	-0.814867	-1.066276	-0.770847	-0.595824	-0.299115	-0.829481

426 rows × 30 columns

Línea 39

- Después de la estandarización, los valores de las columnas no se desvían significativamente de 0.
- Sería posible un mejor rendimiento en comparación con el rendimiento anterior a la estandarización.

- transformar
  - El procesamiento de funciones se realiza con "transform" para devolver el resultado.

[40]:
```
1  from sklearn.preprocessing import StandardScaler
2  X_train, X_test, y_train, y_test = train_test_split(data.data, data.target)
3
4  print(X_train)
5
6  scaler = StandardScaler()
7  scaler.fit(X_train)
8  X_train = scaler.transform(X_train)
9
10 X_train
```

```
[[1.152e+01 1.875e+01 7.334e+01 ... 6.316e-02 3.306e-01 7.036e-02]
 [1.624e+01 1.877e+01 1.088e+02 ... 1.732e-01 2.770e-01 1.063e-01]
 [1.147e+01 1.603e+01 7.302e+01 ... 6.548e-02 2.851e-01 8.763e-02]
 ...
 [1.025e+01 1.618e+01 6.652e+01 ... 9.744e-02 2.608e-01 9.702e-02]
 [1.390e+01 1.924e+01 8.873e+01 ... 8.150e-02 2.356e-01 7.603e-02]
 [1.249e+01 1.685e+01 7.919e+01 ... 2.784e-02 1.917e-01 6.174e-02]]
```

Línea 40-4

- Salida antes del preprocesamiento

- transformar
  - El procesamiento de funciones se realiza con "transform" para devolver el resultado.

[40]:
```
1  from sklearn.preprocessing import StandardScaler
2  X_train, X_test, y_train, y_test = train_test_split(data.data, data.target)
3
4  print(X_train)
5
6  scaler = StandardScaler()
7  scaler.fit(X_train)
8  X_train = scaler.transform(X_train)
9
10 X_train
```

```
[[1.152e+01 1.875e+01 7.334e+01 ... 6.316e-02 3.306e-01 7.036e-02]
 [1.624e+01 1.877e+01 1.088e+02 ... 1.732e-01 2.770e-01 1.063e-01]
 [1.147e+01 1.603e+01 7.302e+01 ... 6.548e-02 2.851e-01 8.763e-02]
 ...
 [1.025e+01 1.618e+01 6.652e+01 ... 9.744e-02 2.608e-01 9.702e-02]
 [1.390e+01 1.924e+01 8.873e+01 ... 8.150e-02 2.356e-01 7.603e-02]
 [1.249e+01 1.685e+01 7.919e+01 ... 2.784e-02 1.917e-01 6.174e-02]]
```

Línea 40-8

- Preprocesamiento: aplicar escala

- transformar
  - El procesamiento de funciones se realiza con "transform" para devolver el resultado.

[40]:
```
1  from sklearn.preprocessing import StandardScaler
2  X_train, X_test, y_train, y_test = train_test_split(data.data, data.target)
3
4  print(X_train)
5
6  scaler = StandardScaler()
7  scaler.fit(X_train)
8  X_train = scaler.transform(X_train)
9
10 X_train
```

```
[[1.152e+01 1.875e+01 7.334e+01 ... 6.316e-02 3.306e-01 7.036e-02]
 [1.624e+01 1.877e+01 1.088e+02 ... 1.732e-01 2.770e-01 1.063e-01]
 [1.147e+01 1.603e+01 7.302e+01 ... 6.548e-02 2.851e-01 8.763e-02]
 ...
 [1.025e+01 1.618e+01 6.652e+01 ... 9.744e-02 2.608e-01 9.702e-02]
 [1.390e+01 1.924e+01 8.873e+01 ... 8.150e-02 2.356e-01 7.603e-02]
 [1.249e+01 1.685e+01 7.919e+01 ... 2.784e-02 1.917e-01 6.174e-02]]
```

Línea 40-10

- Comprobación de resultados después del preprocesamiento

- transformar
  - El procesamiento de funciones se realiza con "transform" para devolver el resultado.

```
[40]: array([[-7.51008336e-01, -1.18885714e-01, -7.79499896e-01, ...,
              -8.05167768e-01,  6.51784189e-01, -7.75029283e-01],
             [ 6.09465521e-01, -1.14029513e-01,  7.03395530e-01, ...,
               8.87122610e-01, -2.33936016e-01,  1.26187368e+00],
             [-7.65420135e-01, -7.79328969e-01, -7.92881919e-01, ...,
              -7.69488800e-01, -1.00086507e-01,  2.03749633e-01],
             ...,
             [-1.11706804e+00, -7.42907466e-01, -1.06470426e+00, ...,
              -2.77980253e-01, -5.01635033e-01,  7.35928788e-01],
             [-6.50066875e-02,  9.11962517e-05, -1.35908226e-01, ...,
              -5.23119372e-01, -9.18055727e-01, -4.53681486e-01],
             [-4.71419429e-01, -5.80224752e-01, -5.34859788e-01, ...,
              -1.34834930e+00, -1.64348701e+00, -1.26356861e+00]])
```

- fit_transform
  - Ajustar y transformar se combina como fit_transform..

[41]:
```
1 print(X_test)
2
3 X_test = scaler.fit_transform(X_test)
4
5 X_test
```

```
[[8.950e+00 1.576e+01 5.874e+01 ... 3.846e-02 1.652e-01 7.722e-02]
 [9.683e+00 1.934e+01 6.105e+01 ... 3.846e-02 2.552e-01 7.920e-02]
 [1.240e+01 1.768e+01 8.147e+01 ... 7.370e-02 2.556e-01 9.359e-02]
 ...
 [9.000e+00 1.440e+01 5.636e+01 ... 1.389e-02 2.991e-01 7.804e-02]
 [8.196e+00 1.684e+01 5.171e+01 ... 2.564e-02 3.105e-01 7.409e-02]
 [9.755e+00 2.820e+01 6.168e+01 ... 4.866e-02 2.321e-01 7.211e-02]]
```

Línea 41-1 y 41-5
- Antes y Después

Línea 41-3
- Combinación de ajuste y transformación.

•Principales módulos de scikit

Clasificación	Módulo	Funciones integradas
Ejemplo de datos	sklearn.datasets	Conjunto de datos para practicar
Procesamiento de características	sklearn.preprocessing	Técnicas de preprocesamiento (codificación one-hot, normalización, escalado, etc.)
	sklearn.feature_selection	Técnica para buscar y seleccionar una característica que proporcione un impacto significativo al modelo.
	sklearn.feature_extraction	Extracción de características de los datos de origen La API de soporte para la extracción de características relacionadas con la imagen está presente en la imagen del submódulo, mientras que la API de soporte para la extracción de características de datos de texto está presente en la prueba del submódulo.
Reducción de dimensiones	sklearn.decomposition	Algoritmos relacionados con la reducción de dimensiones (PCA, NMF, SVD truncado, etc.)
Validación, ajuste de hiperparámetros, separación de datos.	sklearn.model_selection	Validación, ajuste de hiperparámetros, separación de datos, etc. (corss_validate, GridSearchCV, train_test_split, learning_curve, etc.) (, , ,
Evaluación del modelo	sklearn.metrics	Técnicas para medir y evaluar el rendimiento del modelo (exactitud, precisión, recuperación, curva ROC, etc.)

•Principales módulos de scikit

Clasificación	Módulo	Funciones integradas
Algoritmo Machine Learning	sklearn.ensemble	Algoritmos de conjunto (bosque aleatorio, AdaBoost, embolsado, etc.) (
	sklearn.linear_model	Algoritmos lineales (regresión lineal, regresión logística, SGD, etc.) (
	sklearn.naïve_bayes	Algoritmos ingenuos de Bayes (Bernoulli NB, Gaussian NB, distribución multinomial NB, etc.)
	sklearn.neighbors	Algoritmos de vecino más cercano (K-NN, etc.))
	sklearn.svm	Soporta algoritmos de máquina vectorial
	sklearn.tree	Algoritmos de árbol de decisión
	sklearn.cluster	Algoritmos de aprendizaje no supervisado (agrupación) (Kmeans, DBSCAN, etc.) ()
Utilidad	sklearn.pipeline	Conversión en serie de procesamiento de funciones y algoritmos Machine Learning, etc.

Unidad 1.

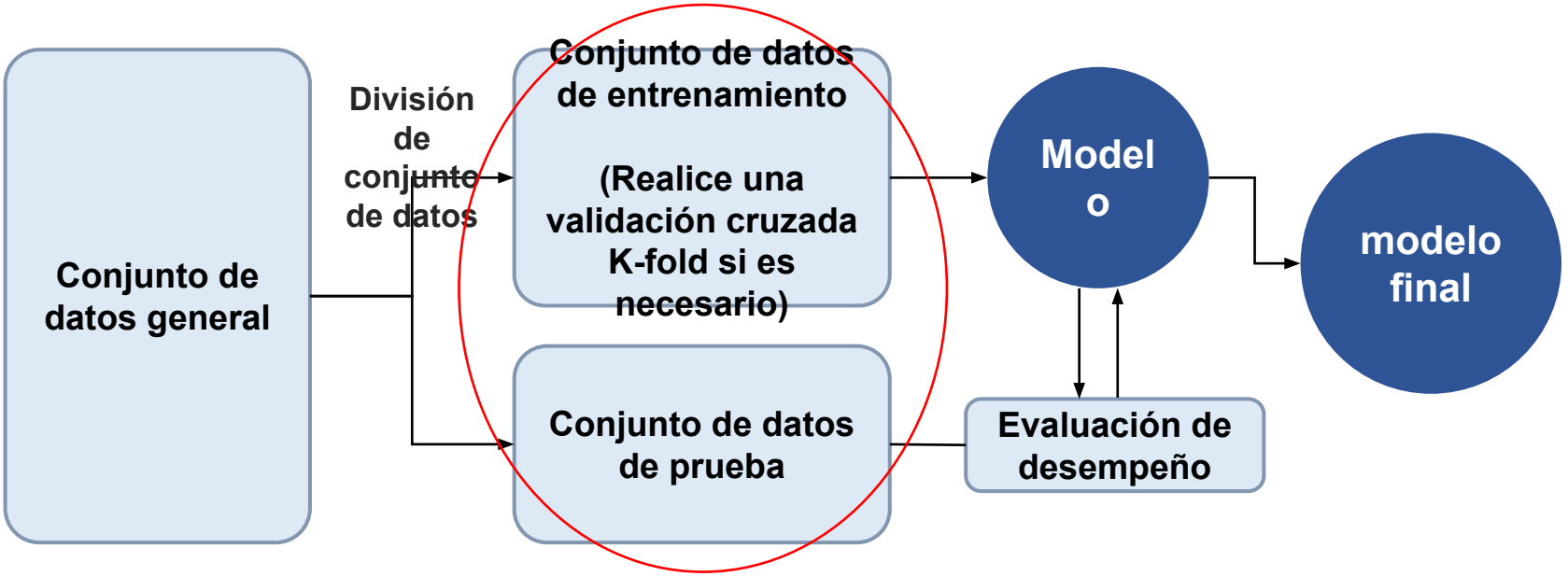
Análisis de datos basado en Machine Learning

- 1.1. ¿Qué es Machine Learning?
- 1.2. Python scikit-aprende library para Machine Learning
- 1.3. Preparación y división del conjunto de datos.
- 1.4. Preprocesamiento de datos para crear un buen conjunto de datos de entrenamiento
- 1.5. Practicando para encontrar un método óptimo para resolver problemas con scikit-learn

Preparación y división del conjunto de datos.

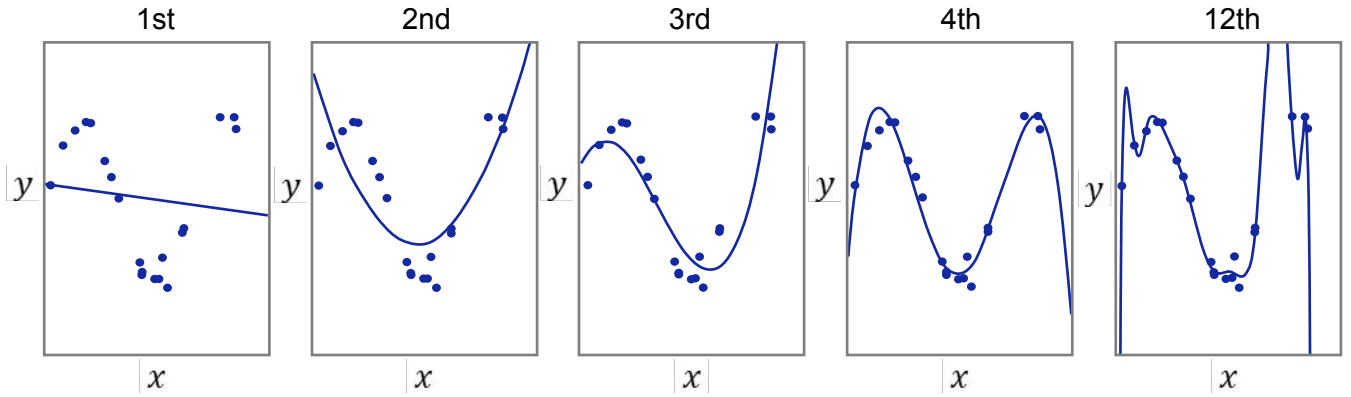
- Objetivos del capítulo
  - Ser capaz de comprender el significado y el efecto dominó del sobreajuste y la generalización y diseñar la división del conjunto de datos para resolver problemas.
  - Ser capaz de dividir adecuadamente los conjuntos de datos de entrenamiento y prueba para la aplicación de la técnica Machine Learning según el propósito del análisis y las características del conjunto de datos.
  - Ser capaz de dividir los conjuntos de datos de entrenamiento y validación y decidir el valor k apropiado para la validación cruzada decidiendo la necesidad de la validación cruzada según el problema y la técnica aplicada. Ser capaz de dividir los conjuntos de datos y realizar muestreos considerando los resultados de la predicción en función de las características de los datos y la distribución de variables clasificadas.
  - Ser capaz de analizar diferencias entre varios métodos de muestreo para la división de conjuntos de datos y aplicar métodos de muestreo apropiados.

- Necesidad de dividir el conjunto de datos
  - Al analizar datos basados en Machine Learning, especialmente cuando se aplica un modelo basado en aprendizaje supervisado, no analice el conjunto de datos general, sino analice dividiendo los conjuntos de datos de entrenamiento y evaluación (prueba).



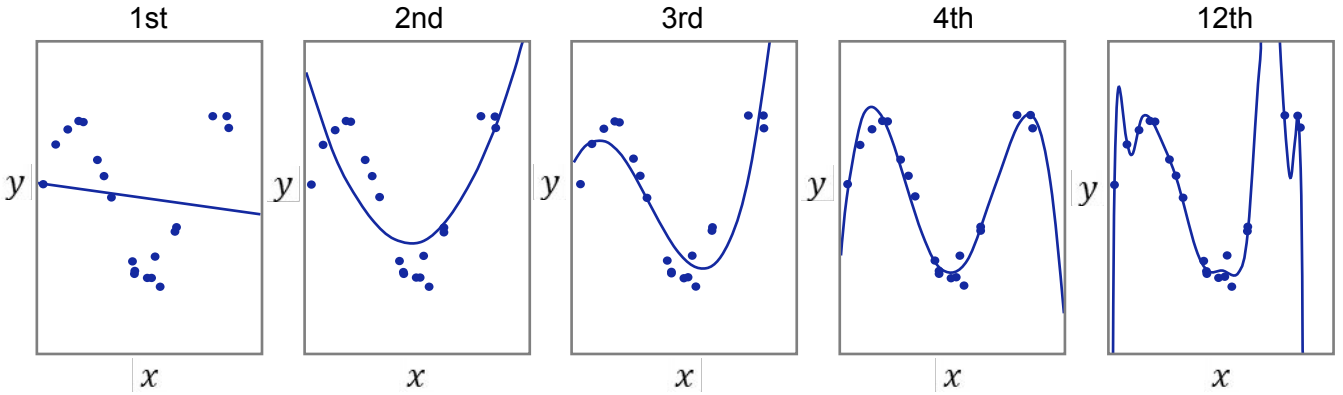
Proceso de modelado Machine Learning mediante la división de conjuntos de datos de entrenamiento y prueba.

- Sobreajuste y generalización del modelado.
  - Estrictamente hablando, los datos incluidos en el conjunto de datos de entrenamiento proporcionado pueden considerarse valores obtenidos por casualidad. Por lo tanto, un nuevo conjunto de datos obtenido para predecir los valores de nuevas variables objetivas (o variables de respuesta) no es el mismo que el conjunto de datos de entrenamiento existente.
  - La posibilidad de que los patrones de los datos de entrenamiento y los datos nuevos concuerden perfectamente es extremadamente baja. Por lo tanto, cuando se aprende el modelo basado en Machine Learning, se produce un sobreajuste al resaltar el patrón del conjunto de datos de entrenamiento cuando se refleja demasiado del conjunto de datos de entrenamiento. Al mismo tiempo, la generalización para una predicción precisa de nuevos datos no se realiza correctamente.
  - Para evitar estos problemas, los conjuntos de datos generalmente se dividen en conjuntos de datos de entrenamiento y de prueba. Mida cómo el modelo Machine Learning aprendido con el conjunto de datos de entrenamiento predice con precisión las variables objetivas (o variables de respuesta) del conjunto de datos de prueba. El estándar resultante se convertirá en el estándar modelo de evaluación del desempeño.



Sobreajuste y desajuste

- Sobreajuste y generalización del modelado.

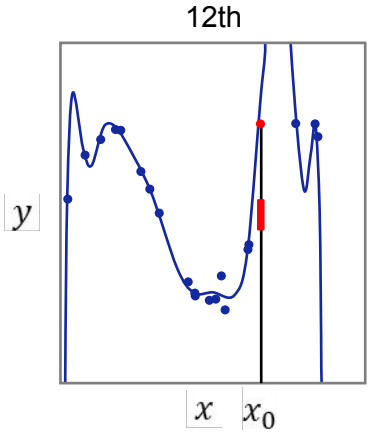


Sobreajuste y desajuste

- Incluso si Machine Learning encuentra la solución óptima en la distribución de datos, se produce un amplio margen de error ya que el modelo tiene una capacidad pequeña. Este fenómeno se conoce como desadaptación; el modelo de ecuación lineal en la figura más a la izquierda arriba es un ejemplo.
- Una alternativa sencilla es utilizar polinomios de mayor grado, que son ecuaciones no lineales.
- La figura más a la derecha proporcionada arriba se aplica con el polinomio de orden 12.th
- La capacidad del modelo aumentó y hay 13 parámetros para la estimación.

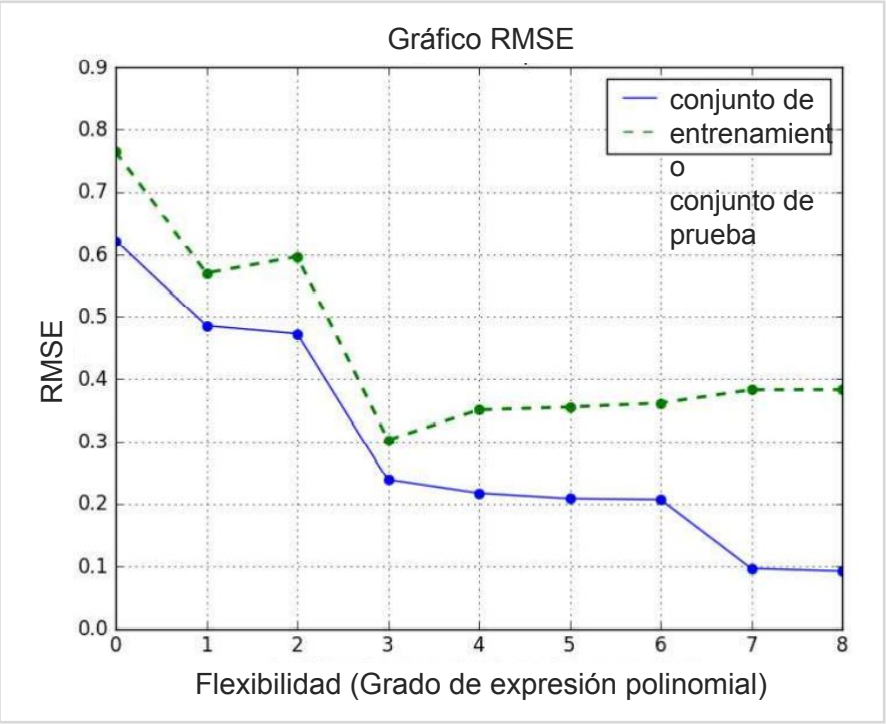
$$y = w_{12}x^{12} + w_{11}x^{11} + w_{10}x^{10} \cdots w_1x^1 + w_0$$

- Sobreajuste
  - When choosing a 12th-order polynomial curve, it approximates almost perfectly to the training set.
  - However, an issue occurs when predicting new data.
    - The region around the red bar at x_0 should be predicted, but the red dot is predicted instead.
  - The reason is because of the large capacity of the model.
    - Accepting the noise during the learning process → Overfitting
  - Model selection is required to select an adequate size model.



Predicción inexacta en sobreajuste

- Sobreajuste y generalización del modelado.

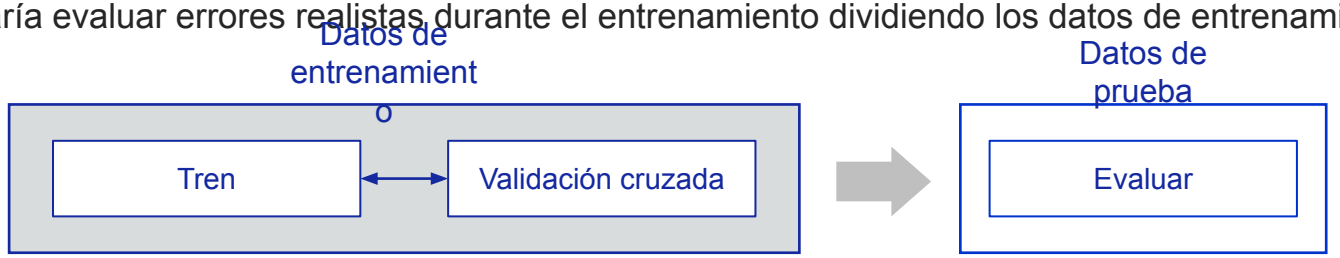


Comparación de la diferencia entre la raíz del error cuadrático medio de los datos de entrenamiento y de prueba

- A medida que aumenta la flexibilidad de la técnica Machine Learning (en otras palabras, la flexibilidad aumenta a medida que aumenta la posibilidad de que los patrones de datos dados coincidan con precisión entre sí, seguido de un mayor orden del polinomio):
  - la raíz del error cuadrático medio del conjunto de datos de entrenamiento muestra una tendencia decreciente monótona.
  - La raíz del error cuadrático medio del conjunto de datos de prueba disminuye al principio a medida que aumenta el orden del polinomio, pero aumenta después de cierto punto.
- En resumen, la figura de la izquierda muestra una tendencia de sobreajuste que refleja demasiado el patrón del conjunto de datos de entrenamiento después del polinomio de cuarto orden. Si no hubiera un índice de error cuadrático medio calculado con datos de prueba, el error cuadrático medio de los datos de entrenamiento disminuiría a medida que aumenta el orden del polinomio, lo que llevaría a seleccionar un modelo de sobreajuste.th
- Por lo tanto, se requieren datos de prueba.

Método y proceso de división del conjunto de datos

- Validación cruzada:
  - Los datos deben dividirse en un conjunto de entrenamiento y un conjunto de prueba.
  - ¡En principio, el equipo de prueba sólo debe usarse una vez! ¡No se debe reutilizar!
  - Si el conjunto de entrenamiento también se utiliza para la evaluación, los errores pueden ser irrealmente pequeños.
  - Nos gustaría evaluar errores realistas durante el entrenamiento dividiendo los datos de entrenamiento en dos.



- Validación cruzada y optimización de hiperparámetros:
  - Como podemos evaluar errores repetidamente durante el entrenamiento, también es posible ajustar los hiperparámetros.

- Validación cruzada:

1) Divida los datos en un conjunto de entrenamiento y un conjunto de prueba.

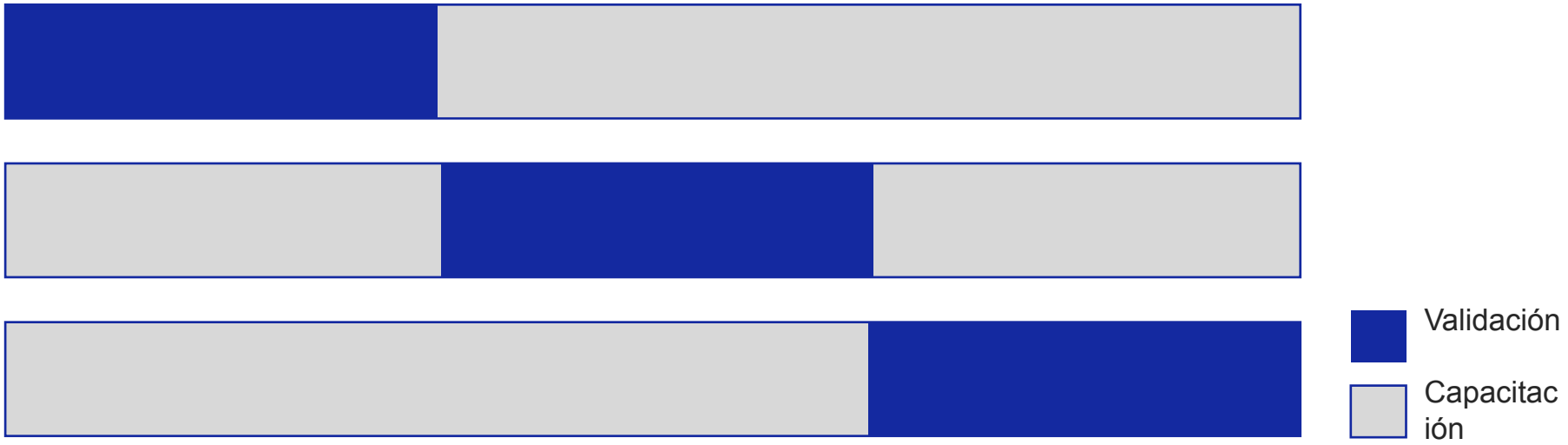
2) Subdividir aún más el conjunto de entrenamiento en un conjunto de entrenamiento más pequeño y un conjunto de validación.

3) Entrene el modelo con el conjunto de entrenamiento más pequeño.

4) Evaluar los errores con el conjunto de validación.

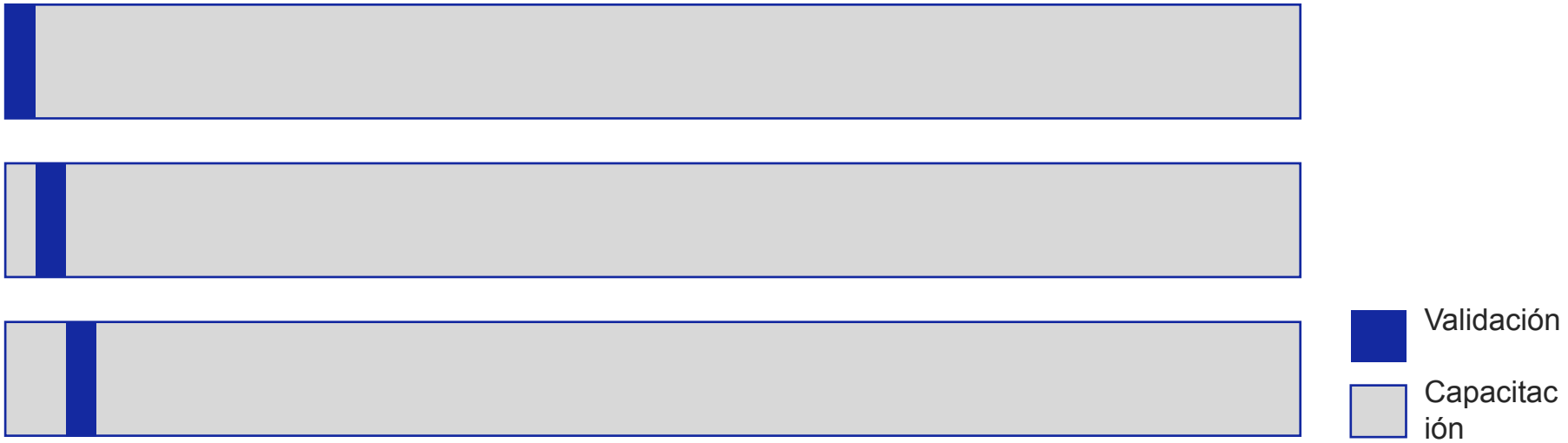
5) Repita desde el paso 2) varias veces.

- Método de validación cruzada: k-Fold



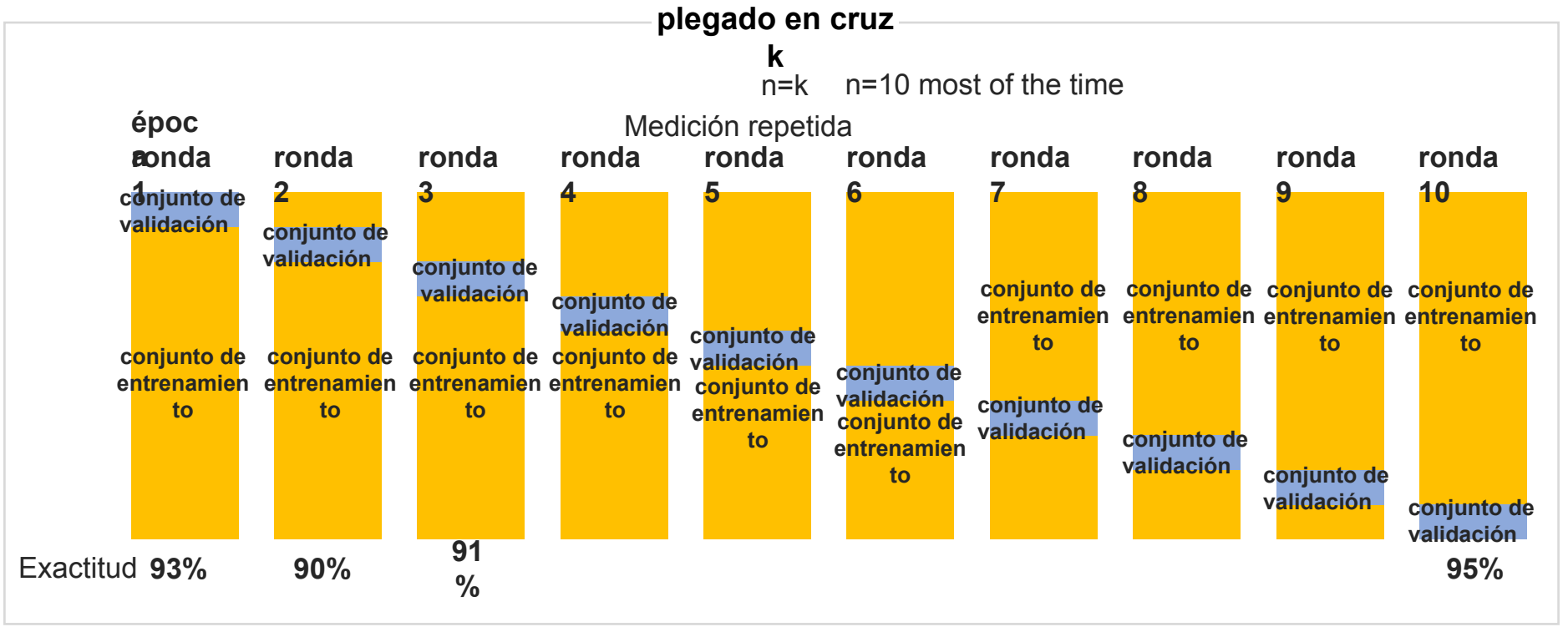
- Subdivide el conjunto de datos de entrenamiento en k partes iguales. Luego, aplicar secuencialmente.k

- Método de validación cruzada: dejar uno fuera (LOO)



- Deje solo una observación para validación. Aplicar secuencialmente. Más tiempo.

- Método de validación cruzada: plegado k-cruzado



Precisión promedio final (Ronda 1, Ronda 2,…, Ronda 10)

Unidad 1.

Análisis de datos basado en Machine Learning

- 1.1. ¿Qué es Machine Learning?
- 1.2. Python scikit-aprende library para Machine Learning
- 1.3. Preparación y división del conjunto de datos.
- 1.4. Preprocesamiento de datos para crear un buen conjunto de datos de entrenamiento
- 1.5. Practicando para encontrar un método óptimo para resolver problemas con scikit-learn

Procesamiento de valores faltantes

- La limpieza de datos para el análisis de datos basado en Machine Learning utiliza valores faltantes y procesamiento de ruido para eliminar discrepancias en los datos recopilados.
  - El procesamiento de valores faltantes se realiza de la siguiente manera.
  - Primero, importe los datos del iris para un examen rápido.

```
[1]: 1 from sklearn.datasets import load_iris

[2]: 1 iris=load_iris()

[3]: 1 import pandas as pd

[4]: 1 iris['data']
```

```
[4]: array([[5.1, 3.5, 1.4, 0.2],
           [4.9, 3. , 1.4, 0.2],
           [4.7, 3.2, 1.3, 0.2],
           [4.6, 3.1, 1.5, 0.2],
           [5. , 3.6, 1.4, 0.2],
           [5.4, 3.9, 1.7, 0.4],
           [4.6, 3.4, 1.4, 0.3],
```

- Procesamiento de valores faltantes

[5]:
```
iris_df = pd.DataFrame(iris['data'],
                       columns=['Sepal_Length', 'Sepal_Width', 'Petal_Length', 'Petal_Width'])
```

[6]:
```
iris_df.columns
```

[6]:
```
Index(['Sepal_Length', 'Sepal_Width', 'Petal_Length', 'Petal_Width'], dtype='object')
```

[7]:
```
iris_df['Sepal_Length']
```

[7]:
```
0      5.1
1      4.9
2      4.7
3      4.6
4      5.0
      ...
145    6.7
146    6.3
147    6.5
148    6.2
149    5.9
Name: Sepal_Length, Length: 150, dtype: float64
```

- Procesamiento de valores faltantes

```
[8]: 1 import numpy as np

[9]: 1 np.mean(iris_df['Sepal_Length'])

[9]: np.float64(5.843333333333334)
```

- Procesamiento de valores faltantes

1) Ignorar el registro (fila)

- En la clasificación de datos, ignore el registro si no se distingue la etiqueta de clase.

Ex En el caso de los datos del iris, ignore la cuarta fila como se muestra en la siguiente tabla.

	x1	x2	x3	x4	y
	Sépalo.Longitud	Sépalo.Ancho	Pétalo.Longitud	Pétalo.Ancho	Especies
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	
5	5	3.6	1.4	0.2	setosa
6	5.4	3.9	1.7	0.4	setosa

- "Ignorar el registro" es extremadamente ineficaz si se producen valores faltantes con frecuencia.

- Procesamiento de valores faltantes

2) Inserte el valor que falta

- Ingrese un valor determinado como "desconocido" para el valor faltante. O ingrese el valor promedio de los datos, como el valor promedio general, el valor mediano o la clase que pertenece al mismo registro.

	x1	x2	x3	x4	y
	Sépalo.Longitud	Sépalo.Ancho	Pétalo.Longitud	Pétalo.Ancho	Especies
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	desconocido
5	5	3.6	1.4	0.2	setosa
6	5.4	3.9	1.7	0.4	setosa

- Procesamiento de valores faltantes

2) Inserte el valor que falta

- El valor promedio de Sepal.Length para el iris es 5,843 como se proporcionó anteriormente, por lo tanto, inserte 5,8

	x1	x2	x3	x4	y
	Sépalo.Longitud	Sépalo.Ancho	Pétalo.Longitud	Pétalo.Ancho	Especies
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4		3.1	1.5	0.2	setosa
5	5	3.6	1.4	0.2	setosa
6	5.4	3.9	1.7	0.4	setosa

	x1	x2	x3	x4	y
	Sépalo.Longitud	Sépalo.Ancho	Pétalo.Longitud	Pétalo.Ancho	Especies
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	5.843	3.1	1.5	0.2	setosa
5	5	3.6	1.4	0.2	setosa
6	5.4	3.9	1.7	0.4	setosa

- Procesamiento de valores faltantes

3) Entrada manual

- Una persona responsable (o un experto) debe verificar los datos y modificarlos a un valor apropiado.
- Requiere mucho tiempo pero proporciona una alta confiabilidad.

- Procesamiento de valores faltantes
  - Identifique el valor que falta en el tipo de datos de la tabla siguiente y realice el procesamiento adecuado.
    - In En Python, el valor faltante se especifica como np.nan o valor nulo.
    - 'nan' es una abreviatura de Not a Number.

```
[10]: 1 data=[[1.0,2.0,3.0,4.0],[5.0,6.0,np.nan,8.0],[10.0,11.0,12.0,np.nan]]

[11]: 1 np.array(data)

[11]: array([[ 1.,  2.,  3.,  4.],
             [ 5.,  6., nan,  8.],
             [10., 11., 12., nan]])
```

- Procesamiento de valores faltantes

```
[12]: 1 pd.DataFrame(np.array(data), columns=['A','B','C','D'])
```

[12]:

	A	B	C	D
0	1.0	2.0	3.0	4.0
1	5.0	6.0	NaN	8.0
2	10.0	11.0	12.0	NaN

```
[13]: 1 df=pd.DataFrame(np.array(data), columns=['A','B','C','D'])
```

- El valor omitido se cambia a NaN. En este caso, no es problemático debido a la pequeña cantidad de datos, pero es extremadamente inconveniente encontrar manualmente los valores faltantes en un marco de datos enorme.

- Procesamiento de valores faltantes
  - isnull() devuelve el marco de datos con un valor booleano, que muestra si la celda tiene un valor numérico (Falso) o se omite con un valor numérico (Verdadero). Luego, sum() se utiliza para obtener el número de omisiones.
  - Es obligatorio verificar la cantidad de valores faltantes al importar datos.

```
[14]: 1 df.isnull()
```

[14]:

	A	B	C	D
0	False	False	False	False
1	False	False	True	False
2	False	False	False	True

```
[15]: 1 df.isnull().sum()
```

```
[15]: A    0
      B    0
      C    1
      D    1
      dtype: int64
```

Línea 15

- Se puede contar el número de valores faltantes.

- Eliminar la muestra de entrenamiento o característica con valor faltante
  - Utilice dropna() para eliminar por completo una determinada muestra de entrenamiento (fila) o característica (columna).
  - help(df.dropna) shows axis=0 is default.

```
[16]: 1 df.dropna()
```

[16]:

	A	B	C	D
0	1.0	2.0	3.0	4.0

Línea 16

- axis=0 is default, so the row with the NaN value is deleted.

- Eliminar la muestra de entrenamiento o característica con valor faltante
  - Al intentar reflejar el resultado eliminado inmediatamente en el objeto, no omitas la opción inplace=True.

```
[17]: 1 df.dropna(inplace=True)

[18]: 1 df
```

[18]:

	A	B	C	D
0	1.0	2.0	3.0	4.0

- Eliminar la muestra de entrenamiento o característica con valor faltante
  - Elimine la fila con el valor faltante.

```
[19]: 1 df=pd.DataFrame(np.array(data), columns=['A','B','C','D'])
```

```
[20]: 1 df
```

[20]:

	A	B	C	D
0	1.0	2.0	3.0	4.0
1	5.0	6.0	NaN	8.0
2	10.0	11.0	12.0	NaN

- Eliminar la muestra de entrenamiento o característica con valor faltante

```
[21]: 1 df.dropna(axis=1, inplace=True)
```

```
[22]: 1 df
```

[22]:

	A	B
0	1.0	2.0
1	5.0	6.0
2	10.0	11.0

- Eliminar la muestra de entrenamiento o característica con valor faltante
  - Si todas las filas tienen NaN, utiliza how='all' para eliminar.

```
[23]: 1 df=pd.DataFrame(np.array(data), columns=['A','B','C','D'])

[24]: 1 df
```

[24]:

	A	B	C	D
0	1.0	2.0	3.0	4.0
1	5.0	6.0	NaN	8.0
2	10.0	11.0	12.0	NaN

- Eliminar la muestra de entrenamiento o característica con valor faltante

```
[25]: 1 df.loc[3]=[np.nan,np.nan,np.nan,np.nan]
```

```
[26]: 1 df
```

[26]:

	A	B	C	D
0	1.0	2.0	3.0	4.0
1	5.0	6.0	NaN	8.0
2	10.0	11.0	12.0	NaN
3	NaN	NaN	NaN	NaN

- Eliminar la muestra de entrenamiento o característica con valor faltante

```
[27]: 1 df.dropna(how='all', inplace=True)
```

```
[28]: 1 df
```

[28]:

	A	B	C	D
0	1.0	2.0	3.0	4.0
1	5.0	6.0	NaN	8.0
2	10.0	11.0	12.0	NaN

- Eliminar la muestra de entrenamiento o característica con valor faltante
  - Para eliminar filas con menos de 3 valores que no sean NaN, utilice Thresh.

```
[29]: 1 df=pd.DataFrame(np.array(data), columns=['A','B','C','D'])
      2 df.loc[3]=[np.nan,np.nan,np.nan,3]

[30]: 1 df
```

[30]:

	A	B	C	D
0	1.0	2.0	3.0	4.0
1	5.0	6.0	NaN	8.0
2	10.0	11.0	12.0	NaN
3	NaN	NaN	NaN	3.0

- Eliminar la muestra de entrenamiento o característica con valor faltante

```
[31]: 1 df.dropna(thresh=3, inplace=True)
```

```
[32]: 1 df
```

[32]:

	A	B	C	D
0	1.0	2.0	3.0	4.0
1	5.0	6.0	NaN	8.0
2	10.0	11.0	12.0	NaN

- Eliminar la muestra de entrenamiento o característica con valor faltante
  - Al eliminar una fila con NaN en una determinada columna, utilice el asunto.

```
[33]: 1 df
```

[33]:

	A	B	C	D
0	1.0	2.0	3.0	4.0
1	5.0	6.0	NaN	8.0
2	10.0	11.0	12.0	NaN

```
[34]: 1 df.dropna(subset=['C'], inplace=True)
```

```
[35]: 1 df
```

[35]:

	A	B	C	D
0	1.0	2.0	3.0	4.0
2	10.0	11.0	12.0	NaN

- Imputación
  - A veces es difícil eliminar una muestra de entrenamiento o una determinada columna porque se pierden demasiados datos útiles. Si es así, estime los valores faltantes de otras muestras de entrenamiento en el conjunto de datos mediante kriging. El método más utilizado es imputar el valor promedio, que consiste en cambiar el valor faltante al promedio general de una determinada columna. En scikit-learn, use la clase SimpleImputer

```
[36]: 1 from sklearn.impute import SimpleImputer
```

```
[37]: 1 df=pd.DataFrame(np.array(data), columns=['A','B','C','D'])
```

```
[38]: 1 impt=SimpleImputer(missing_values=np.nan, strategy='mean')
```

  - Impute con el uso de df.values..

```
[39]: 1 df.values
```

```
[39]: array([[ 1.,  2.,  3.,  4.],
              [ 5.,  6., nan,  8.],
              [10., 11., 12., nan]])
```

- Imputación

```
[40]: 1 help(impt.fit)
```

```
Help on method fit in module sklearn.impute._base:

fit(X, y=None) method of sklearn.impute._base.SimpleImputer instance
    Fit the imputer on `X`.

    Parameters
    ----------
    X : {array-like, sparse matrix}, shape (n_samples, n_features)
        Input data, where `n_samples` is the number of samples and
        `n_features` is the number of features.

    y : Ignored
        Not used, present here for API consistency by convention.

    Returns
    -------
    self : object
        Fitted estimator.
```

- Imputación

```
[41]: 1 imputed=impt.fit(df.values)

[42]: 1 imputed.transform(df.values)

[42]: array([[ 1. ,  2. ,  3. ,  4. ],
             [ 5. ,  6. ,  7.5,  8. ],
             [10. , 11. , 12. ,  6. ]])

[43]: 1 (3+12)/2

[43]: 7.5
```

Línea 43

- Verifique que sea el promedio de la columna.

- Para los parámetros de estrategia, también se pueden establecer la mediana y la mayoría de las veces.

- Revisión de la API del estimador scikit-learn
  - Imputa el valor faltante del conjunto de datos utilizando la clase SimpleImputer de scikit-learn de la cláusula anterior.
  - La clase SimpleImputer es una clase transformadora de scikit-learn que se utiliza para la conversión de datos.
  - Los dos métodos principales del estimador incluyen el ajuste y la transformación.
  - Utilice el método de ajuste para conocer el parámetro del modelo en los datos de entrenamiento.
  - Utilice el método de transformación para convertir los datos en el parámetro aprendido.
  - Los datos Array para la conversión deben ser los mismos que la cantidad de características de datos utilizadas en el aprendizaje del modelo.

- Procesamiento de datos categóricos
  - Los datos generalmente se clasifican en escalas categóricas y continuas según sus características.
  - En artes liberales y ciencias sociales, los cuestionarios se utilizan principalmente para recopilar datos.
  - **escala categórica**
    - Es una escala que puede distinguir datos en diferentes categorías y se clasifica en escalas nominal y ordinal.
  - **Escala continua**
    - Es una escala que divide los datos vinculados según el propósito de la encuesta. Se clasifica en escalas de intervalo y de razón.
  - Los conjuntos de datos reales incluirían más de una característica categórica. Como se explicó anteriormente, los datos categóricos clasificarían características secuenciales y no secuenciales. La escala ordinal puede denominarse escala categórica secuencial que puede Array funciones con secuencias.

- Procesamiento de datos categóricos

[44]:
```
df = pd.DataFrame([['green', 'M', 10.1, 'class2'],
                   ['red', 'L', 13.5, 'class1'],
                   ['blue', 'XL', 15.3, 'class2']])
```

[45]:
```
df
```

[45]:

	0	1	2	3
0	green	M	10.1	class2
1	red	L	13.5	class1
2	blue	XL	15.3	class2

- Los datos de la tabla tienen características que tienen pedidos y no tienen pedidos. Se pide la talla, pero no se pide el color. Así, el tamaño se clasifica en la escala ordinal, mientras que el color es la escala nominal.

- Procesamiento de datos categóricos
  - Convertir datos ordenados en valores numéricos. La razón por la que se cambian datos de texto a datos numéricos es para permitir que una computadora procese operaciones aritméticas.

[46]:
```
size_mapping = {'XL': 3,
                'L': 2,
                'M': 1}
```

[47]:
```
df.columns=['color', 'size', 'price', 'classlabel']
```

[48]:
```
df
```

[48]:

	color	size	price	classlabel
0	green	M	10.1	class2
1	red	L	13.5	class1
2	blue	XL	15.3	class2

- Procesamiento de datos categóricos

```
[49]: 1 df['size']
```

```
[49]: 0     M
      1     L
      2    XL
      Name: size, dtype: object
```

```
[50]: 1 df['size']=df['size'].map(size_mapping)
```

```
[51]: 1 df['size']
```

```
[51]: 0    1
      1    2
      2    3
      Name: size, dtype: int64
```

```
[52]: 1 df
```

[52]:

	color	size	price	classlabel
0	green	1	10.1	class2
1	red	2	13.5	class1
2	blue	3	15.3	class2

- Codificación de etiquetas de clase
  - La clase se refiere al valor y, una columna con un valor real.
  - Cree una asignación para convertir la etiqueta de clase de strings a números enteros.

```
[53]: 1 df['classlabel']

[53]: 0    class2
      1    class1
      2    class2
      Name: classlabel, dtype: object

[54]: 1 df['classlabel'].value_counts()

[54]: classlabel
      class2    2
      class1    1
      Name: count, dtype: int64

[55]: 1 np.unique(df['classlabel'])

[55]: array(['class1', 'class2'], dtype=object)
```

- Codificación de etiquetas de clase
  - 'enumerar' crea un objeto con un índice.

```
[56]: enumerate(np.unique(df['classlabel']))

[56]: <enumerate at 0x15fef9c2390>

[57]: for idx, label in enumerate(np.unique(df['classlabel'])):
          print(idx, label)

      0 class1
      1 class2
```

  - 'enumerar' crea un objeto con un índice.

```
[58]: {label: idx for idx, label in enumerate(np.unique(df['classlabel']))}

[58]: {'class1': 0, 'class2': 1}

[59]: class_mapping={label: idx for idx, label in enumerate(np.unique(df['classlabel']))}
```

- Codificación de etiquetas de clase
  - 'enumerar' crea un objeto con un índice.

```
[60]: 1 df['classlabel']=df['classlabel'].map(class_mapping)
      2 df
```

[60]:

	color	size	price	classlabel
0	green	1	10.1	1
1	red	2	13.5	0
2	blue	3	15.3	1

Línea 60

- Cambie la etiqueta de clase de strings a números enteros.

- Codificación de etiquetas de clase
  - Dado que el método de la diapositiva anterior es bastante inconveniente, scikit-learn admite LabelEncoder para una fácil conversión.

[61]:
```
df = pd.DataFrame([['green', 'M', 10.1, 'class2'],
                   ['red', 'L', 13.5, 'class1'],
                   ['blue', 'XL', 15.3, 'class2']])
df.columns=['color', 'size', 'price', 'classlabel']
```

[62]:
```
df
```

[62]:

	color	size	price	classlabel
0	green	M	10.1	class2
1	red	L	13.5	class1
2	blue	XL	15.3	class2

- Codificación de etiquetas de clase

```
[63]: 1 from sklearn.preprocessing import LabelEncoder

[64]: 1 enc=LabelEncoder()

[65]: 1 y=enc.fit_transform(df['classlabel'].values)

[66]: 1 y

[66]: array([1, 0, 1])
```

Aplicación de codificación One-Hot a funciones desordenadas

- Hay casos en los que es imposible utilizar directamente datos categóricos en algoritmos Machine Learning, como análisis de regresión, etc. Si es así, es necesario que una computadora reconozca la conversión.

  - En tales casos, utilice una variable ficticia expresada como 0 o 1. 0 o 1 no representa cómo el número es grande o pequeño, sino que muestra si una determinada característica está presente.
  - Si una determinada característica está presente, se expresa como 1; si no se encuentra, se clasifica como 0. Del mismo modo, la codificación one-hot convierte datos categóricos en un vector one-hot que consta de 0 o 1 que puede ser reconocido por una computadora.
  - Practica con el objeto iris.target.

```
[67]: 1 iris.target
```

```
[67]: array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
             0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
             0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
             1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
             1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2,
             2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
             2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2])
```

- La codificación se realiza con números enteros, así que inserte el valor de "especie" del iris.

```
[68]: 1 iris=pd.DataFrame(iris.target, columns=['species'])
```

```
[69]: 1 iris
```

[69]:

	species
0	0
1	0
2	0
3	0
4	0
...	...
145	2
146	2
147	2
148	2
149	2

150 rows × 1 columns

- La codificación se realiza con números enteros, así que inserte el valor de "especie" del iris.

```
[70]: 1 iris['species'] = iris['species'].replace({0:'setosa', 1:'versicolor', 2:'virginica'})

[71]: 1 iris
```

[71]:

	species
0	setosa
1	setosa
2	setosa
3	setosa
4	setosa
...	...
145	virginica
146	virginica
147	virginica
148	virginica
149	virginica

150 rows × 1 columns

- Utilice la función get_dummies() de pandas para convertir cada valor propio de variables categóricas en una nueva variable ficticia.

```
[72]: 1 pd.get_dummies(iris['species'])
```

[72]:

	setosa	versicolor	virginica
0	True	False	False
1	True	False	False
2	True	False	False
3	True	False	False
4	True	False	False
...	...	...	...

- Utilice sklearn library para procesar cómodamente la codificación one-hot. El resultado se da como una matriz dispersa en álgebra lineal. En la matriz dispersa, el valor de la mayoría de las matrices es 0. Un concepto opuesto a la matriz dispersa es el de matriz densa.

- OneHotEncoder

```
[73]: 1 from sklearn.preprocessing import OneHotEncoder

[74]: 1 onehot_encoder=OneHotEncoder()

[75]: 1 len_iris=len(iris['species'])
```

- OneHotEncoder

```
[76]: 1 np.array(iris['species'])
```

```
[76]: array(['setosa', 'setosa', 'setosa', 'setosa', 'setosa', 'setosa',
       'setosa', 'setosa', 'setosa', 'setosa', 'setosa', 'setosa',
       'setosa', 'setosa', 'setosa', 'setosa', 'setosa', 'setosa',
       'setosa', 'setosa', 'setosa', 'setosa', 'setosa', 'setosa',
       'setosa', 'setosa', 'setosa', 'setosa', 'setosa', 'setosa',
       'setosa', 'setosa', 'setosa', 'setosa', 'setosa', 'setosa',
       'setosa', 'setosa', 'setosa', 'setosa', 'setosa', 'setosa',
       'setosa', 'setosa', 'versicolor', 'versicolor', 'versicolor',
       'versicolor', 'versicolor', 'versicolor', 'versicolor',
       'versicolor', 'versicolor', 'versicolor', 'versicolor',
       'versicolor', 'versicolor', 'versicolor', 'versicolor',
       'versicolor', 'versicolor', 'versicolor', 'versicolor',
       'versicolor', 'versicolor', 'versicolor', 'versicolor',
       'versicolor', 'versicolor', 'versicolor', 'versicolor',
       'versicolor', 'versicolor', 'versicolor', 'versicolor',
       'versicolor', 'versicolor', 'versicolor', 'versicolor',
       'versicolor', 'versicolor', 'versicolor', 'versicolor',
       'versicolor', 'versicolor', 'versicolor', 'virginica', 'virginica',
       'virginica', 'virginica', 'virginica', 'virginica', 'virginica',
       'virginica', 'virginica', 'virginica', 'virginica', 'virginica',
       'virginica', 'virginica', 'virginica', 'virginica', 'virginica',
       'virginica', 'virginica', 'virginica', 'virginica', 'virginica',
       'virginica', 'virginica', 'virginica', 'virginica', 'virginica',
       'virginica', 'virginica', 'virginica', 'virginica', 'virginica',
       'virginica', 'virginica', 'virginica', 'virginica', 'virginica',
       'virginica', 'virginica', 'virginica'], dtype=object)
```

- OneHotEncoder

```
[77]: 1 iris_species_2d=np.array(iris['species']).reshape(len_iris, 1)

[78]: 1 onehot_encoder.fit_transform(iris_species_2d)

[78]: <Compressed Sparse Row sparse matrix of dtype 'float64'
        with 150 stored elements and shape (150, 3)>

[79]: 1 #Convert to a space matrix

[80]: 1 onehot_reshaped=onehot_encoder.fit_transform(iris_species_2d)
```

- Conversión a la matriz dispersa

```
[81]: 1 print(onehot_reshaped[0:50])
```

```
<Compressed Sparse Row sparse matrix of dtype 'float64'
        with 50 stored elements and shape (50, 3)>
  Coords        Values
  (0, 0)        1.0         (17, 0)       1.0         (34, 0)       1.0
  (1, 0)        1.0         (18, 0)       1.0         (35, 0)       1.0
  (2, 0)        1.0         (19, 0)       1.0         (36, 0)       1.0
  (3, 0)        1.0         (20, 0)       1.0         (37, 0)       1.0
  (4, 0)        1.0         (21, 0)       1.0         (38, 0)       1.0
  (5, 0)        1.0         (22, 0)       1.0         (39, 0)       1.0
  (6, 0)        1.0         (23, 0)       1.0         (40, 0)       1.0
  (7, 0)        1.0         (24, 0)       1.0         (41, 0)       1.0
  (8, 0)        1.0         (25, 0)       1.0         (42, 0)       1.0
  (9, 0)        1.0         (26, 0)       1.0         (43, 0)       1.0
  (10, 0)       1.0         (27, 0)       1.0         (44, 0)       1.0
  (11, 0)       1.0         (28, 0)       1.0         (45, 0)       1.0
  (12, 0)       1.0         (29, 0)       1.0         (46, 0)       1.0
  (13, 0)       1.0         (30, 0)       1.0         (47, 0)       1.0
  (14, 0)       1.0         (31, 0)       1.0         (48, 0)       1.0
  (15, 0)       1.0         (32, 0)       1.0         (49, 0)       1.0
  (16, 0)       1.0         (33, 0)       1.0
```

Línea 81

- (0, 0) is (0, 0) es 1, por lo tanto setosa (setosa hasta 50 matrices).

- Consulte la figura siguiente para una comprensión más sencilla.

```
[82]: 1 print(onehot_reshaped[50:100])
```

```
<Compressed Sparse Row sparse matrix of dtype 'float64'
        with 50 stored elements and shape (50, 3)>
  Coords        Values
  (0, 1)        1.0       (21, 1)       1.0       (42, 1)       1.0
  (1, 1)        1.0       (22, 1)       1.0       (43, 1)       1.0
  (2, 1)        1.0       (23, 1)       1.0       (44, 1)       1.0
  (3, 1)        1.0       (24, 1)       1.0       (45, 1)       1.0
  (4, 1)        1.0       (25, 1)       1.0       (46, 1)       1.0
  (5, 1)        1.0       (26, 1)       1.0       (47, 1)       1.0
  (6, 1)        1.0       (27, 1)       1.0       (48, 1)       1.0
  (7, 1)        1.0       (28, 1)       1.0       (49, 1)       1.0
  (8, 1)        1.0       (29, 1)       1.0
  (9, 1)        1.0       (30, 1)       1.0
  (10, 1)       1.0       (31, 1)       1.0
  (11, 1)       1.0       (32, 1)       1.0
  (12, 1)       1.0       (33, 1)       1.0
  (13, 1)       1.0       (34, 1)       1.0
  (14, 1)       1.0       (35, 1)       1.0
  (15, 1)       1.0       (36, 1)       1.0
  (16, 1)       1.0       (37, 1)       1.0
  (17, 1)       1.0       (38, 1)       1.0
  (18, 1)       1.0       (39, 1)       1.0
  (19, 1)       1.0       (40, 1)       1.0
  (20, 1)       1.0       (41, 1)       1.0
```

- Consulte la figura siguiente para una comprensión más sencilla.

```
[83]: 1 print(onehot_reshaped[100:150])
```

```
<Compressed Sparse Row sparse matrix of dtype 'float64'
        with 50 stored elements and shape (50, 3)>
  Coords        Values
  (0, 2)        1.0         (21, 2)       1.0         (42, 2)       1.0
  (1, 2)        1.0         (22, 2)       1.0         (43, 2)       1.0
  (2, 2)        1.0         (23, 2)       1.0         (44, 2)       1.0
  (3, 2)        1.0         (24, 2)       1.0         (45, 2)       1.0
  (4, 2)        1.0         (25, 2)       1.0         (46, 2)       1.0
  (5, 2)        1.0         (26, 2)       1.0         (47, 2)       1.0
  (6, 2)        1.0         (27, 2)       1.0         (48, 2)       1.0
  (7, 2)        1.0         (28, 2)       1.0         (49, 2)       1.0
  (8, 2)        1.0         (29, 2)       1.0
  (9, 2)        1.0         (30, 2)       1.0
  (10, 2)       1.0         (31, 2)       1.0
  (11, 2)       1.0         (32, 2)       1.0
  (12, 2)       1.0         (33, 2)       1.0
  (13, 2)       1.0         (34, 2)       1.0
  (14, 2)       1.0         (35, 2)       1.0
  (15, 2)       1.0         (36, 2)       1.0
  (16, 2)       1.0         (37, 2)       1.0
  (17, 2)       1.0         (38, 2)       1.0
  (18, 2)       1.0         (39, 2)       1.0
  (19, 2)       1.0         (40, 2)       1.0
  (20, 2)       1.0         (41, 2)       1.0
```

- Consulte la figura siguiente para una comprensión más sencilla.

	Row index	0		2	
Column index	Especies	setosa	versicolor	virginica	Sparse matrix expression
0	setosa	1	0	0	(0,0)
1	setosa	1	0	0	(1,0)
	setosa	1	0	0	
49	setosa	1	0	0	(49,0
50	versicolor	0	1	0	(50,1)
51	versicolor	0	1	0	(51,1)
	versicolor	0	1	0	
100	versicolor	0	1	0	
101	virginica	0	0	1	(101,2)
	virginica	0	0	1	(102,2)
	virginica	0	0	1	
150	virginica	0	0	1	(150,2)

- Uso de retención en la vida real que divide el conjunto de datos en conjuntos de datos de entrenamiento y de prueba
  - df_wine son los datos que miden los vinos producidos en Vinho Verde, adyacente al Océano Atlántico en el noroeste de Portugal. Midió y analizó el grado, el sabor y la acidez de 1.599 muestras de vino tinto y 4.898 de vino blanco para crear datos. Si los datos no se encuentran en la siguiente ruta, es posible importarlos desde local descargándolos directamente desde el repositorio de UCI.

- Uso de retención en la vida real que divide el conjunto de datos en conjuntos de datos de entrenamiento y de prueba

[84]:
```
1  df_wine = pd.read_csv('https://archive.ics.uci.edu/ml/machine-learning-databases/wine/wine.data',
2                        header=None)
3
4  df_wine.columns = ['Class label', 'Alcohol', 'Malic acid', 'Ash',
5                     'Alcalinity of ash', 'Magnesium', 'Total phenols',
6                     'Flavanoids', 'Nonflavanoid phenols', 'Proanthocyanins',
7                     'Color intensity', 'Hue', 'OD280/OD315 of diluted wines',
8                     'Proline']
9
10 print('Class labels', np.unique(df_wine['Class label']))
11 df_wine.head()
```

```
Class labels [1 2 3]
```

Línea 84

- Cuando no sea accesible para el conjunto de datos de vino del repositorio UCI Machine Learning, elimine la observación del siguiente código y lea el conjunto de datos de la ruta local:

- df_wine = pd.read_csv(‘wine.data’, header=None)

1.4. Preprocesamiento de datos para crear un buen conjunto de datos de entrenamiento

- Uso de retención en la vida real que divide el conjunto de datos en conjuntos de datos de entrenamiento y de prueba

[84]:

	Class label	Alcohol	Malic acid	Ash	Alcalinity of ash	Magnesium	Total phenols	Flavanoids	Nonflavanoid phenols	Proanthocyanins	Color intensity	Hue	OD280/OD315 of diluted wines	Proline
0	1	14.23	1.71	2.43	15.6	127	2.80	3.06	0.28	2.29	5.64	1.04	3.92	1065
1	1	13.20	1.78	2.14	11.2	100	2.65	2.76	0.26	1.28	4.38	1.05	3.40	1050
2	1	13.16	2.36	2.67	18.6	101	2.80	3.24	0.30	2.81	5.68	1.03	3.17	1185
3	1	14.37	1.95	2.50	16.8	113	3.85	3.49	0.24	2.18	7.80	0.86	3.45	1480
4	1	13.24	2.59	2.87	21.0	118	2.80	2.69	0.39	1.82	4.32	1.04	2.93	735

- Uso de retención en la vida real que divide el conjunto de datos en conjuntos de datos de entrenamiento y de prueba
  - La división de datos es posible utilizando la función train_test_split proporcionada en el módulo model_selection de scikit-learn. Primero, convierta las características del índice 1 al 13 a NumPy Array y asígnelas a la variable X. Con la función train_test_split, la conversión de datos se realiza en cuatro tuplas, así que asigne designando las variables apropiadas.

[85]:
```
from sklearn.model_selection import train_test_split

X, y = df_wine.iloc[:, 1:].values, df_wine.iloc[:, 0].values

X_train, X_test, y_train, y_test =\
    train_test_split(X, y,
                     test_size=0.3,
                     random_state=0,
                     stratify=y)
```

  - Divide aleatoriamente X e y en conjuntos de datos de entrenamiento y de prueba. test_size=0.3, por lo que el 30% de la muestra se asigna a X_test e y_test.
  - Con respecto al parámetro de estratificación, si se envía la etiqueta de clase Array y, la proporción de clases encontrada en los conjuntos de datos de entrenamiento y prueba se mantiene idéntica a la del conjunto de datos original.
  - Las proporciones más utilizadas en la vida real son 6:4, 7:3 y 8:2, según el tamaño del conjunto de datos. Es común y adecuado para conjuntos de datos grandes dividir el conjunto de datos de entrenamiento y el conjunto de datos de prueba en una proporción de 9:1 o 9,9:0,1..

- Organizar la escala entre características (variables)
  - Consulte el código práctico (Capítulo 5_Unidad 1_Análisis de datos basado en aprendizaje automático 1 (aprendizaje supervisado)) para obtener un código detallado.

[86]:
```
from sklearn.preprocessing import MinMaxScaler

mms = MinMaxScaler()
X_train_norm = mms.fit_transform(X_train)
X_test_nrom = mms.transform(X_test)
```

[87]:
```
from sklearn.preprocessing import StandardScaler

stdsc = StandardScaler()
X_train_std = stdsc.fit_transform(X_train)
X_test_std = stdsc.transform(X_test)
```

- Organizar la escala entre características (variables)
  - Consulte el código práctico (Capítulo 5_Unidad 1_Análisis de datos basado en aprendizaje automático 1 (aprendizaje supervisado)) para obtener un código detallado.

[88]:
```
from sklearn.preprocessing import RobustScaler
rbs = RobustScaler()
X_train_robust = rbs.fit_transform(X_train)
X_test_robust = rbs.transform(X_test)
```

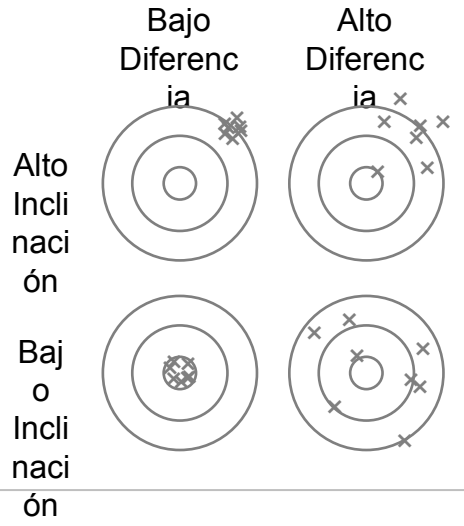
- Organizar la escala entre características (variables)
  - Consulte el código práctico (Capítulo 5_Unidad 1_Análisis de datos basado en aprendizaje automático 1 (aprendizaje supervisado)) para obtener un código detallado.

[89]:
```
# MaxAbsScaler divides the data into the maximum absolute value based on each feature.
# Thus, the maximum value of each feature becomes 1.
# The overall feature changes to [-1, 1] range.
```

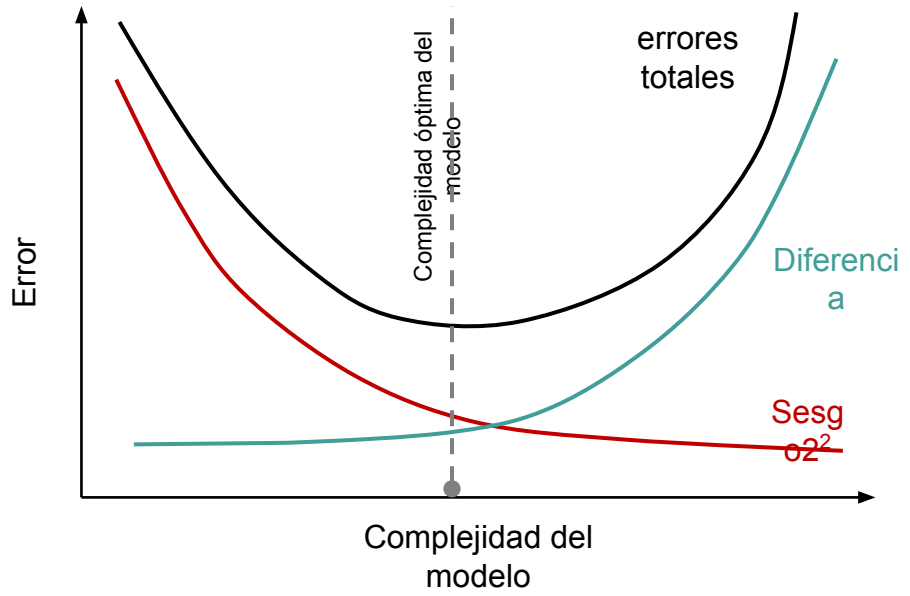
[90]:
```
from sklearn.preprocessing import MaxAbsScaler
mas = MaxAbsScaler()
X_train_maxabs = mas.fit_transform(X_train)
X_test_maxabs = mas.transform(X_test)
```

Limitar la complejidad del modelo mediante regularizaciones L1 y L2

- Sesgo y variación
  - Si los valores previstos se desvían mucho del valor objetivo real en general, se dice que hay un alto sesgo en el resultado. Cuando los valores predichos están dispersos muy lejos unos de otros, se dice que el resultado tiene una gran variación.
  - La siguiente figura expresa los resultados previstos del modelo en el objetivo. El sesgo alto se refiere al resultado cuando se desvía significativamente del centro del objetivo. Por otro lado, si los valores previstos se reúnen alrededor del centro del objetivo, el sesgo es bajo. El sesgo se refiere a la similitud general entre los valores predichos y los valores reales. En la figura, una varianza alta se produce cuando los valores predichos están muy separados entre sí. Por el contrario, una varianza baja se produce cuando los valores predichos están muy juntos. Por tanto, la varianza se refiere a la similitud general entre los valores predichos.



- Compensación
  - El sesgo y la varianza tienen una relación de compensación en la que cuando uno aumenta, el otro cae y viceversa. El modelo se vuelve complejo al inicio del aprendizaje; el costo general del error cae debido a la disminución del sesgo. Sin embargo, en algún momento, el modelo sigue aprendiendo y se vuelve mucho más complicado, lo que provoca una mayor variación y un mayor costo general de error. En otras palabras, el modelo se sobreajusta a los datos de entrenamiento. Una forma de evitar el sobreajuste es dejar de aprender en el momento adecuado. La regularización es un método para evitar el sobreajuste reduciendo la varianza. Aún así, puede aumentar el sesgo debido a la relación de compensación.



- Regresión de cresta
  - El modelo de regresión de cresta es una técnica para limitar la norma L2 de w, que es el vector de coeficientes de regresión. Se agrega una restricción para minimizar la suma de cuadrados de peso en la función de costo de la regresión lineal. Si el modelo de regresión lineal es el siguiente:

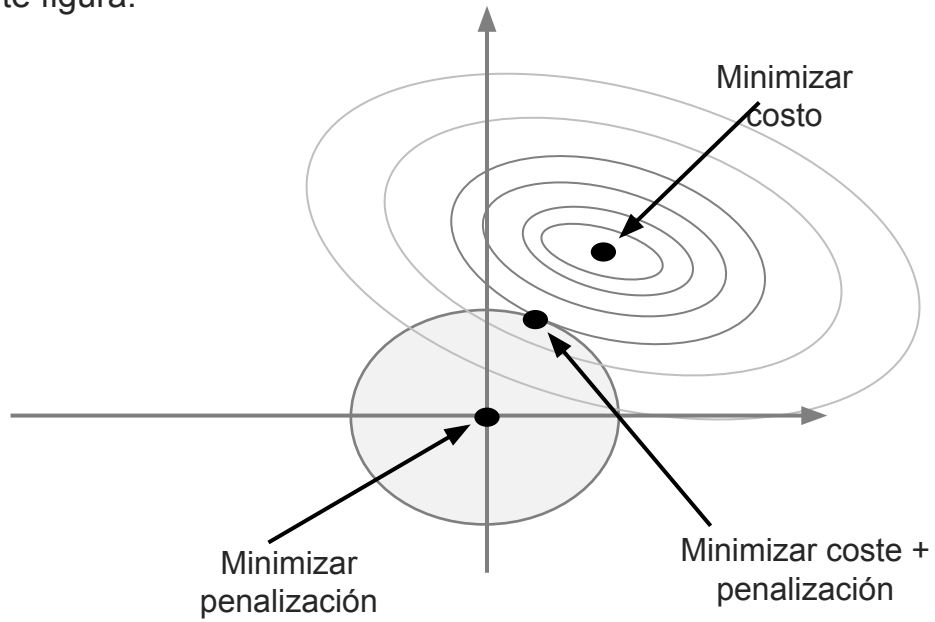
$$\hat{y} = w_0 + w_1 x_1 + w_2 x_2 + \cdots + w_n x_n = Xw \quad \left(\mathrm{X} = \begin{bmatrix} 1 \\ x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}, \mathrm{w} = \begin{bmatrix} w_0 \\ w_1 \\ w_2 \\ \vdots \\ w_n \end{bmatrix} \right)$$

  - Entonces, la función de costos del modelo de regresión de crestas es la siguiente. N es el número de datos y M es el número de elementos del vector de coeficientes de regresión. Se agrega una restricción al SSE (suma de errores al cuadrado) existente.

  - λ es un hiperparámetro para ajustar el peso de la SSE existente y la restricción añadida. Cuando λ es grande, la regularización se aplica con mayor fuerza y los coeficientes de regresión se vuelven más bajos. Cuando λ se hace más pequeño, la regularización se debilita. Cuando λ es igual a 0, la cláusula de restricción también se vuelve 0, igual que en el modelo de regresión lineal general.

- Regresión de cresta
  - El siguiente es un ejemplo de ecuación de modelo de regresión lineal simple:

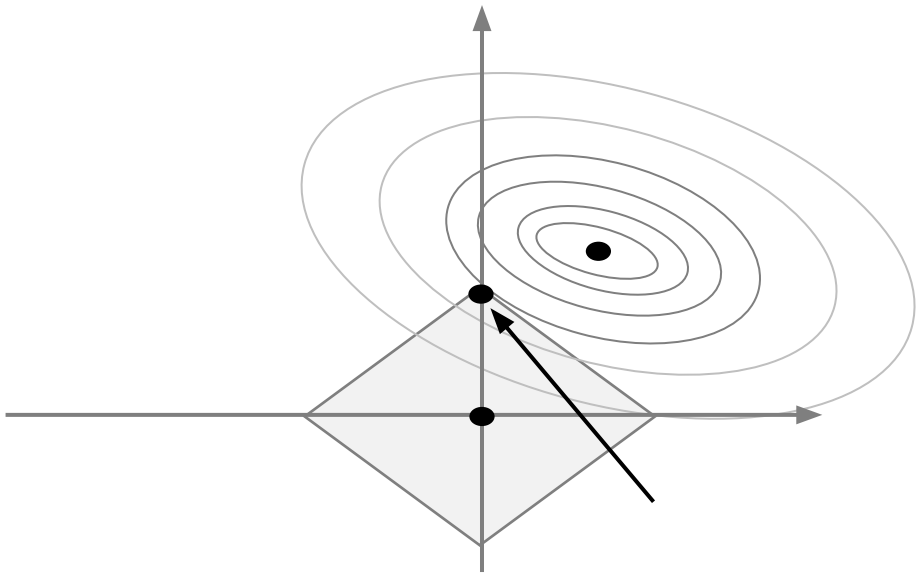
- Regresión de cresta
  - Al dibujar la función de costo SSE (w1, w2) en las coordenadas con el eje x y el eje y, se crea una elipse como se muestra en la siguiente figura:



  - En la figura anterior, la elipse dibujada con una línea sólida es la función de costo, que es la combinación de w1 y w2 con el mismo costo (SSE). El punto central de la elipse es donde el costo se vuelve 0. Hacia el exterior de la elipse se encuentran las combinaciones de w1 y w2 con un costo mayor, lo que representa el modelo con mayor error. El círculo de color se refiere a la restricción. El círculo se vuelve más pequeño cuando λ se hace más grande, y viceversa. El punto donde la función de costo (elipse) y la restricción (círculo de color) se encuentran es la solución óptima donde el costo del modelo de regresión Ridge es mínimo.

- Regresión de lazo
  - El modelo de regresión Lasso (Operador de selección y contracción mínima absoluta) es una técnica para limitar la L1norn del vector de coeficientes de regresión w. Se agrega una restricción para minimizar la suma de los valores absolutos de las ponderaciones en la función de costos de la regresión lineal. La siguiente figura muestra la función de costos del modelo de regresión de Lasso:

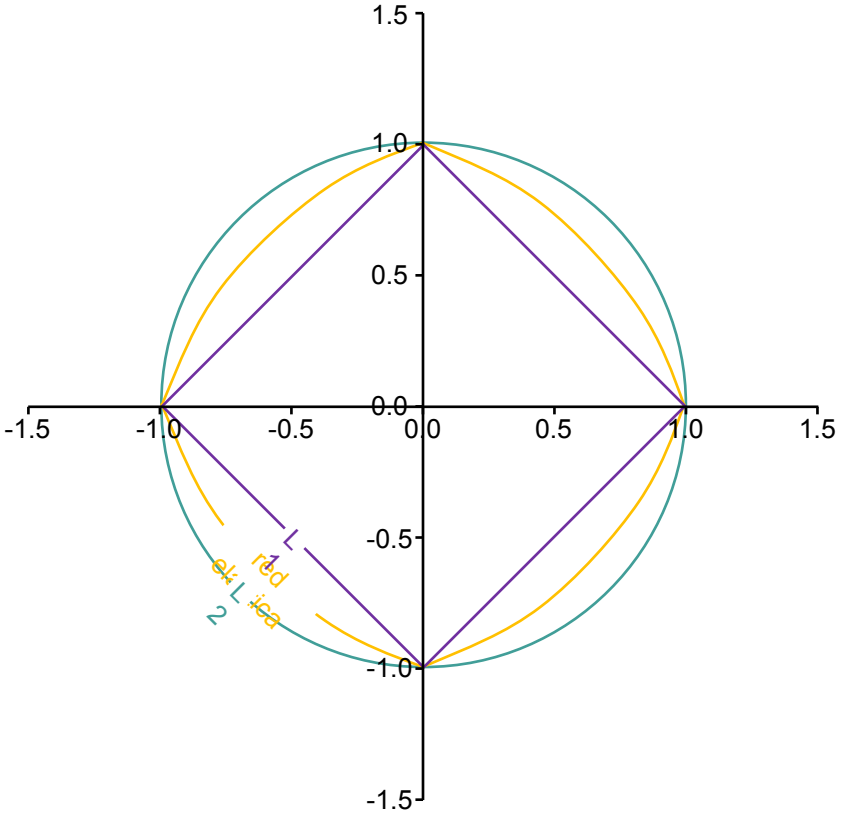
- Regresión de lazo
  - Al dibujar la función de costos del modelo de regresión de Lasso (w1, w2) en las coordenadas con el eje x y el eje y, se crea un rombo como se muestra en la siguiente figura:



  - Dado que la restricción del modelo de regresión de Lasso es un rombo, es muy posible que el punto que cumple con la función de costos sea el vértice del rombo. Los vértices del rombo son siempre los puntos donde w1 o w2 son 0. Por lo tanto, el modelo de regresión de Lasso da como resultado un peso de 0.

- Regresión de red elástica
  - El modelo de regresión Elastic-net aplica tanto la norma L2 como la norma L1 al vector de coeficientes de regresión. La restricción es tanto la suma de los cuadrados de los pesos como la suma de los valores absolutos de los mismos. La siguiente figura muestra la función de costo de Elastic-net. Hay dos hiperparámetros en Elastic-net, que son λ1 y λ2.

- Regresión de red elástica
  - Elastic-net aplica L2norn y L1norn al mismo tiempo, por lo que la restricción está en algún punto intermedio. Reduce un peso mayor mientras convierte un peso sin importancia en 0.



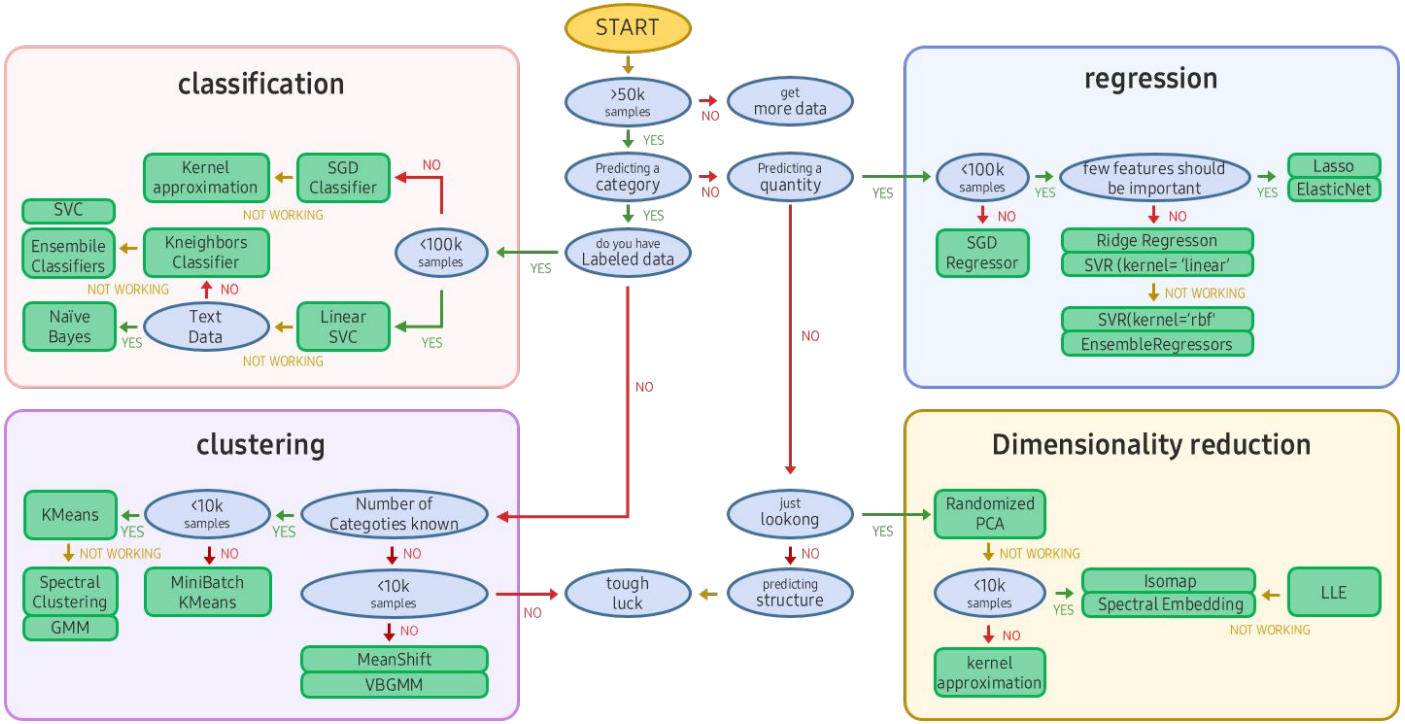
Unidad 1.

Análisis de datos basado en Machine Learning

- 1.1. ¿Qué es Machine Learning?
- 1.2. Python scikit-aprende library para Machine Learning
- 1.3. Preparación y división del conjunto de datos.
- 1.4. Preprocesamiento de datos para crear un buen conjunto de datos de entrenamiento
- 1.5. Practicando para encontrar un método óptimo para resolver problemas con scikit-learn

Encontrar un método óptimo para resolver problemas con scikit-learn

- Práctica
  - Utilice la siguiente metodología de resolución de problemas y considere qué algoritmo aplicar. Realice el preprocesamiento y el proceso general con respecto a los datos del iris y compárelos con el código de resultado que se proporciona a continuación.



- Práctica

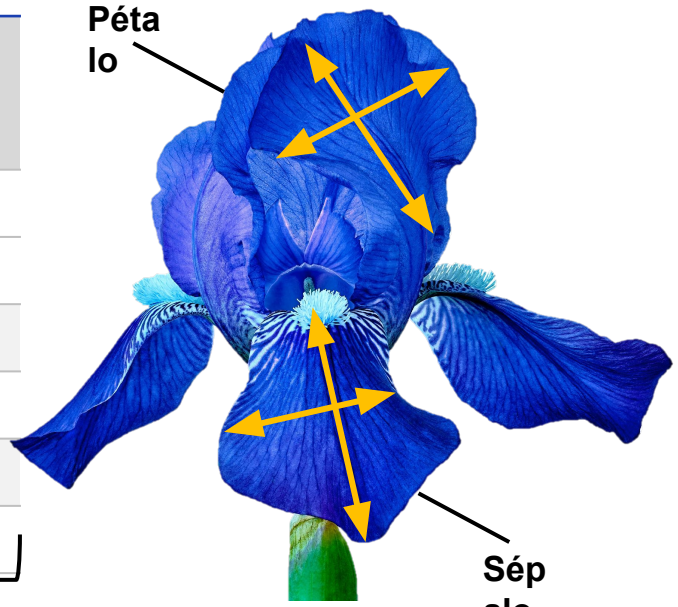
Muestra (instancia, observación)

	Longitud del sépalo	Ancho del sépalo	Longitud del pétalo	Ancho del pétalo	etiqueta de clase
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3.0	1.4	0.2	setosa
			…		
50	6.4	3.5	4.5	1.2	versicolor
			…		
150	5.9	3.0	5.0	1.8	virginica

Característica (Propiedad,

valor de medición, dimensión)

Etiqueta de clase (objetivo)



- Consideraciones en Machine Learning

1) Definir el problema del negocio y buscar soluciones o mejores planes alternativos.

2) Comprobar si es posible definirlo como problema supervisado o no supervisado.

3) Verifique qué método utilizar para medir el rendimiento del modelo.

4) Verificar si el índice de desempeño está vinculado con el objetivo comercial y confirmar si los participantes del proyecto llegaron a un acuerdo con respecto al desempeño.

- En este problema de práctica, defina el problema como especie de iris (supervisada) y suponga que el resultado es satisfactorio si el desempeño predice una precisión de clasificación del 85% o más. as is if 85% or

- Comprender los datos del iris

```
[1]: 1 from sklearn.datasets import load_iris
     2 data=load_iris()
     3 data.keys()
```

```
[1]: dict_keys(['data', 'target', 'frame', 'target_names', 'DESCR', 'feature_names', 'filename', 'data_module'])
```

- Conocimiento de dominio de los datos del iris.
  - Nombre de los datos: IRIS
  - Número de datos: 150
  - Número de variables: 5
  - Comprender las variables

Longitud del sépalo	Información de longitud del sépalo.
Ancho del sépalo	Información de ancho del sépalo.
Longitud del pétalo	Información de longitud del pétalo.
Ancho del pétalo	Información de ancho del pétalo.
Especies	Especies de flores, clasificadas en setosa / versicolor / virginica

- Comprender los datos del iris

```
[2]: 1 print(data.DESCR)
```

```
.. _iris_dataset:

Iris plants dataset
--------------------

**Data Set Characteristics:**

:Number of Instances: 150 (50 in each of three classes)
:Number of Attributes: 4 numeric, predictive attributes and the class
:Attribute Information:
    - sepal length in cm
    - sepal width in cm
    - petal length in cm
    - petal width in cm
    - class:
            - Iris-Setosa
            - Iris-Versicolour
            - Iris-Virginica
```

- Comprender los datos del iris

```
:Summary Statistics:

============== ==== ==== ======= ===== ====================
                Min  Max   Mean    SD   Class Correlation
============== ==== ==== ======= ===== ====================
sepal length:   4.3  7.9   5.84   0.83    0.7826
sepal width:    2.0  4.4   3.05   0.43   -0.4194
petal length:   1.0  6.9   3.76   1.76    0.9490  (high!)
petal width:    0.1  2.5   1.20   0.76    0.9565  (high!)
============== ==== ==== ======= ===== ====================

:Missing Attribute Values: None
:Class Distribution: 33.3% for each of 3 classes.
:Creator: R.A. Fisher
:Donor: Michael Marshall (MARSHALL%PLU@io.arc.nasa.gov)
:Date: July, 1988

The famous Iris database, first used by Sir R.A. Fisher. The dataset is taken
from Fisher's paper. Note that it's the same as in R, but not as in the UCI
Machine Learning Repository, which has two wrong data points.

This is perhaps the best known database to be found in the
pattern recognition literature.  Fisher's paper is a classic in the field and
is referenced frequently to this day.  (See Duda & Hart, for example.)  The
data set contains 3 classes of 50 instances each, where each class refers to a
type of iris plant.  One class is linearly separable from the other 2; the
latter are NOT linearly separable from each other.
```

•Preprocesamiento de datos del iris y EDA

[3]:
```
1 import pandas as pd
2 import seaborn as sns
3 import numpy as np
4 import matplotlib.pyplot as plt
```

[4]:
```
1 iris=pd.DataFrame(data=data.data, columns=data.feature_names)
2 iris
```

Línea 3-1 ~ 4

- Importe el library necesario para practicar.

- Preprocesamiento de datos del iris y EDA

[3]:
```
1 import pandas as pd
2 import seaborn as sns
3 import numpy as np
4 import matplotlib.pyplot as plt
```

[4]:
```
1 iris=pd.DataFrame(data=data.data, columns=data.feature_names)
2 iris
```

Línea 4-1

- Convierta los datos de la variable actual a DataFrame..

- Preprocesamiento de datos del iris y EDA

[4]:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2
...	...	...	...	...
145	6.7	3.0	5.2	2.3
146	6.3	2.5	5.0	1.9
147	6.5	3.0	5.2	2.0
148	6.2	3.4	5.4	2.3
149	5.9	3.0	5.1	1.8

150 rows × 4 columns

- Preprocesamiento de datos del iris y EDA

[5]:
```
1 feature=pd.DataFrame(data=data.data, columns=data.feature_names)
```

[6]:
```
1 data.target
```

[6]:
```
array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2,
       2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
       2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2])
```

[7]:
```
1 target=pd.DataFrame(data.target, columns=["species"])
```

- Preprocesamiento de datos del iris y EDA

```
[8]: 1 target
```

[8]:

	species
0	0
1	0
2	0
3	0
4	0
...	...
145	2
146	2
147	2
148	2
149	2

150 rows × 1 columns

- Preprocesamiento de datos del iris y EDA

```
[9]: 1 iris = pd.concat([feature, target], axis=1)
     2
     3 iris.rename({"sepal length (cm)":"sepal_length", "sepal width (cm)":"sepal_width",
     4              "petal length (cm)":"petal_length", "petal width (cm)":"petal_width"},
     5             axis=1, inplace=True)
     6 iris.columns
```

```
[9]: Index(['sepal_length', 'sepal_width', 'petal_length', 'petal_width',
            'species'],
           dtype='object')
```

Línea 9-1

- Concatenar característica y objetivo.

- Preprocesamiento de datos del iris y EDA

```
[9]: 1 iris = pd.concat([feature, target], axis=1)
     2
     3 iris.rename({"sepal length (cm)":"sepal_length", "sepal width (cm)":"sepal_width",
     4              "petal length (cm)":"petal_length", "petal width (cm)":"petal_width"},
     5             axis=1, inplace=True)
     6 iris.columns
```

```
[9]: Index(['sepal_length', 'sepal_width', 'petal_length', 'petal_width',
            'species'],
           dtype='object')
```

Línea 9-3 ~ 9-5 Cambie el nombre de la columna.

- Preprocesamiento de datos del iris y EDA

```
[10]: 1 iris["species"]=iris.species.map(lambda x: data.target_names[x])
      2 iris.head()
```

[10]:

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

Línea 10

- Cambie el valor objetivo.

- Preprocesamiento de datos del iris y EDA

```
[11]: 1 iris.isna().sum()
```

```
[11]: sepal_length    0
      sepal_width     0
      petal_length    0
      petal_width     0
      species         0
      dtype: int64
```

Línea 11

- Verifique el valor que falta.

- Preprocesamiento de datos del iris y EDA
  - Análisis estadístico básico
    - Realice análisis estadísticos básicos para comprender mejor los datos al comprender el tamaño de los datos (cantidad de datos), la forma de los datos (forma de matriz), el tipo de datos, la distribución de los datos y la relación entre las características. También mejorará el rendimiento de Machine Learning. by of of

```
[12]: 1 iris.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   sepal_length  150 non-null    float64
 1   sepal_width   150 non-null    float64
 2   petal_length  150 non-null    float64
 3   petal_width   150 non-null    float64
 4   species       150 non-null    object
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
```

- Preprocesamiento de datos del iris y EDA

```
[13]: 1 iris.describe()
```

[13]:

	sepal_length	sepal_width	petal_length	petal_width
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.057333	3.758000	1.199333
std	0.828066	0.435866	1.765298	0.762238
min	4.300000	2.000000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000
50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000

Línea 13

- petal_length tiene la mayor desviación estándar. En comparación con otras características, petal_width parece tener un rango de valores más estrecho. Sería mejor realizar la regularización después de verificar el rendimiento del modelo debido a las diferencias de escala entre las características.

- Preprocesamiento de datos del iris y EDA
  - Análisis de correlación
    - Utilice corr para analizar la relación entre características.

```
[14]: 1 iris.corr(numeric_only=True)
```

[14]:

	sepal_length	sepal_width	petal_length	petal_width
sepal_length	1.000000	-0.117570	0.871754	0.817941
sepal_width	-0.117570	1.000000	-0.428440	-0.366126
petal_length	0.871754	-0.428440	1.000000	0.962865
petal_width	0.817941	-0.366126	0.962865	1.000000

Línea 14

- El coeficiente de correlación de petal_length y petal_width es 0,962865, que es extremadamente alto. Dado que las características altamente correlacionadas pueden inducir problemas de multicolinealidad, se recomienda seleccionar una de las dos variables para usar.

- Preprocesamiento de datos del iris y EDA
  - Análisis de agregación

```
[15]: 1 iris.groupby('species').size()
```

```
[15]: species
      setosa        50
      versicolor    50
      virginica     50
      dtype: int64
```

Línea 15

- La cantidad de datos en cada objetivo se contó utilizando la función de agregación "tamaño" y se confirmó que se encontraron 50 datos por igual en cada característica. Seleccione entre "tamaño" y "recuento" según el propósito del análisis. El "tamaño" cuenta la cantidad de datos, incluidos los valores faltantes. Por otro lado, el 'recuento' cuenta la cantidad de datos sin valores faltantes. En este caso, no hay diferencia al usar "tamaño" y "recuento" porque a los datos del iris no les falta ningún valor.

- Preprocesamiento de datos del iris y EDA
  - Visualización de datos
    - Anteriormente, se realizaba un análisis estadístico básico de los datos. Aún así, no es fácil de entender debido a que hay demasiados números y la lectura incorrecta de los puntos decimales daría lugar a errores importantes. Si es así, la visualización de datos como un gráfico proporciona una comprensión intuitiva para el lector. Además, la visualización de datos es eficaz para explicar los resultados del análisis de datos.

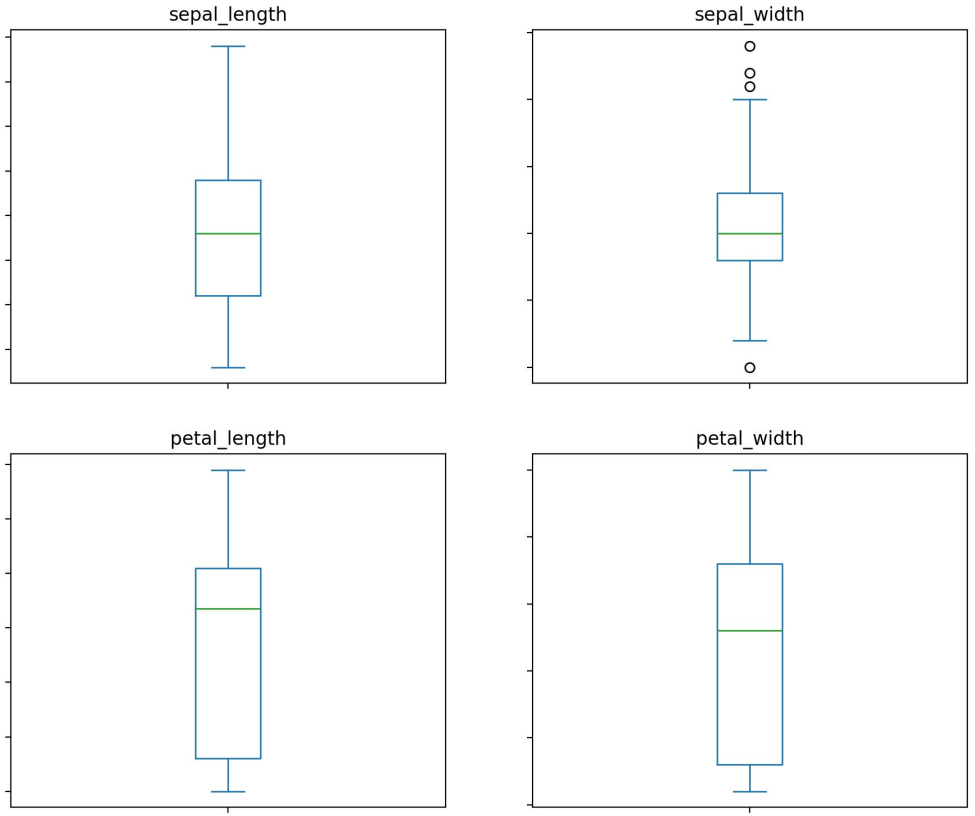
- Preprocesamiento de datos del iris y EDA
  - Visualización de estadísticas básicas y valores atípicos

[16]:

```
def boxplot_iris(feature_names, dataset):
    i=1
    plt.figure(figsize=(11,9))
    for col in feature_names:
        plt.subplot(2,2,i)
        plt.axis('on')
        plt.tick_params(axis='both', left=True,
                        top=False, right=False,
                        bottom=True, labelleft=False,
                        labeltop=False, labelright=False,
                        labelbottom=False)
        dataset[col].plot(kind='box', subplots=True, sharex=False)
        plt.title(col)
        i += 1
    plt.show()
```

- Preprocesamiento de datos del iris y EDA

```
[17]: 1 boxplot_iris(iris.columns[:-1], iris)
```



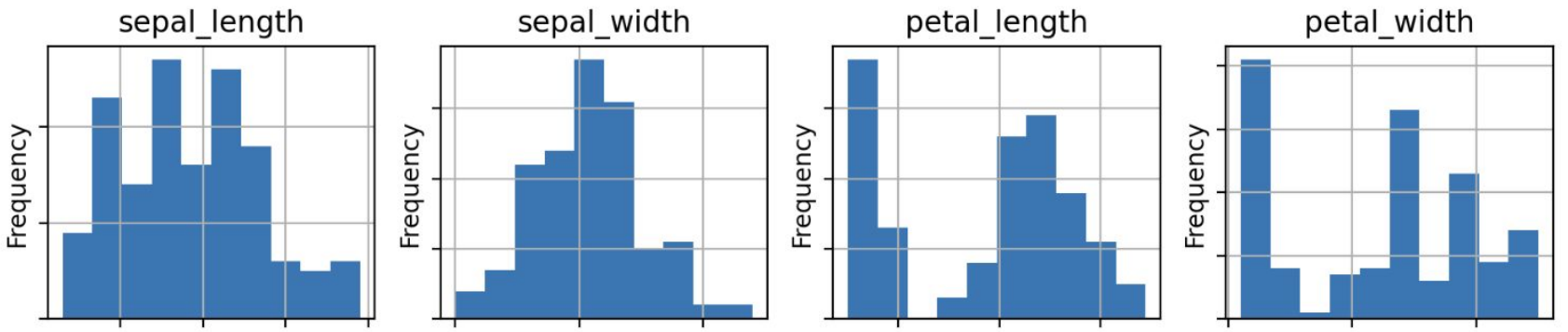
- Preprocesamiento de datos del iris y EDA
  - Visualizando la distribución de datos

[18]:

```
def histogram_iris(feature_names, dataset):
    i=1
    plt.figure(figsize=(11,2))
    for col in feature_names:
        plt.subplot(1,4,i)
        plt.axis('on')
        plt.tick_params(axis='both', left=True,
                        top=False, right=False,
                        bottom=True, labelleft=False,
                        labeltop=False, labelright=False,
                        labelbottom=False)
        dataset[col].plot(kind='hist', subplots=True, grid=True)
        plt.title(col)
        i += 1
    plt.show()
```

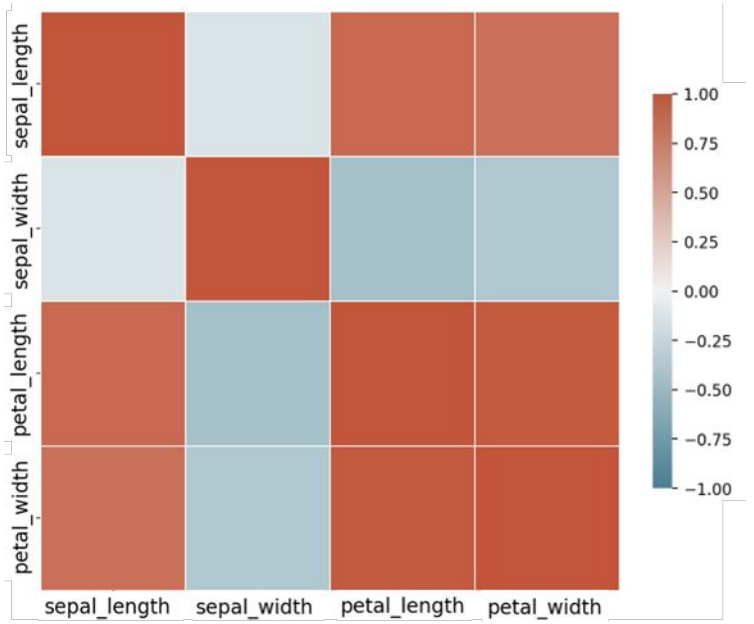
- Preprocesamiento de datos del iris y EDA
  - La frecuencia del intervalo de clase mediana es alta para petal_width y se vuelve más baja a medida que se desvía del centro. En el diagrama de caja, la longitud del cuadro de sepal_width es corta porque se agregaron muchos datos en la mediana. En el caso de petal_length, la frecuencia del intervalo de clase mediano es alta, pero hay muchos datos en el intervalo de clase izquierdo. En el diagrama de caja, el cuadro de longitud_pétalo es largo hasta el final porque había muchos datos en valores más bajos.

```
[19]: 1 histogram_iris(iris.columns[:-1], iris)
```



- Visualizando la correlación

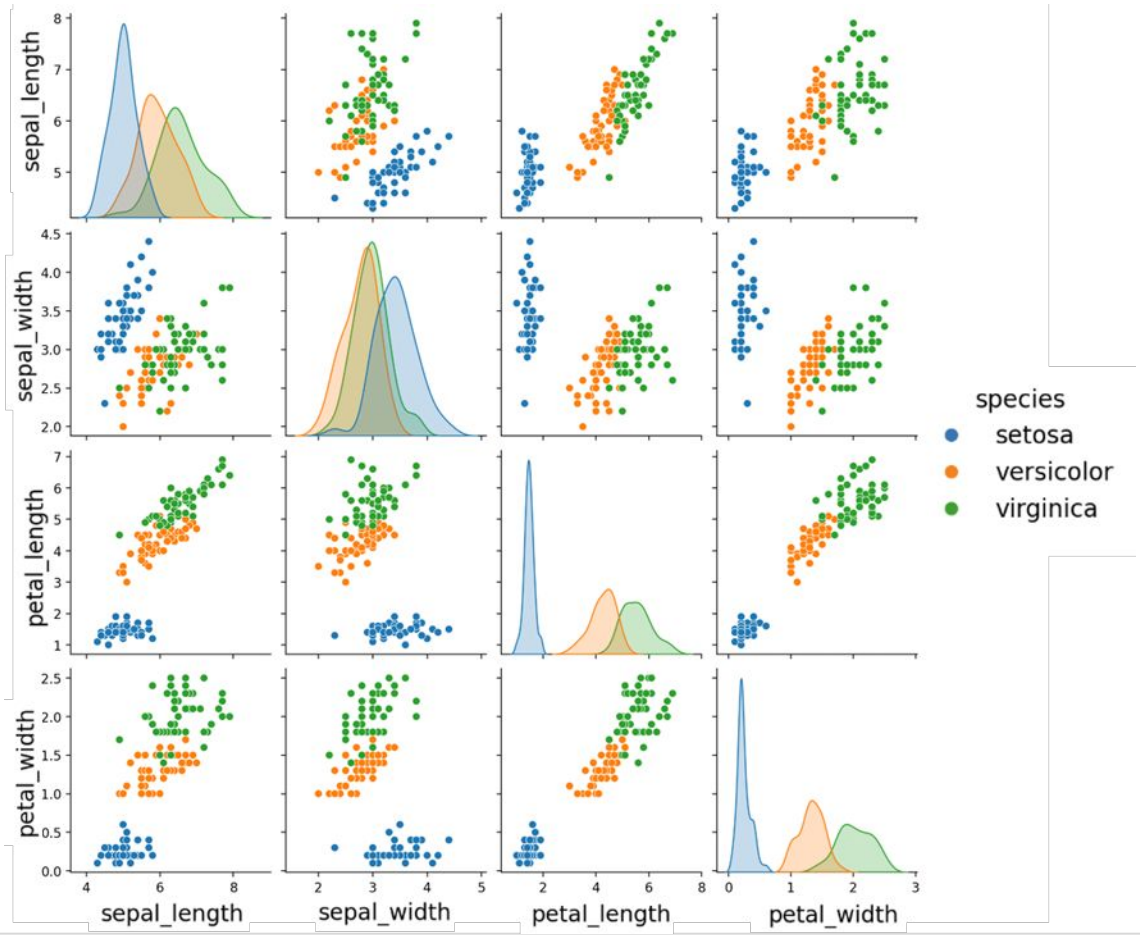
[20]:
```
1 corr=iris.corr(numeric_only=True)
2 cmap=sns.diverging_palette(220,20, as_cmap=True)
3 plt.figure(figsize=(11,9))
4 sns.heatmap(corr, cmap=cmap, vmax=1.0, vmin=-1.0, center=0, square=True,
5             linewidths=.5, cbar_kws={"shrink":.5})
6 plt.show()
```



- Visualización de la correlación entre características y distribución de datos mediante diagrama de pares

[21]:
```
1 sns.pairplot(iris, hue="species")
2 plt.plot()
```

- Visualización de la correlación entre características y distribución de datos mediante diagrama de pares



- Visualización de la correlación entre características y distribución de datos mediante diagrama de pares
  - setosa se agrega al desviarse claramente de otras clases. La clasificación es posible trazando una línea imaginaria y setosa se clasificará como un modelo lineal. Para versicolor y virginica, parece difícil clasificarlos trazando una línea porque están mezclados en el gráfico, completo con las características sepal_width y sepal_length. Sin embargo, aunque parezca un poco vago, se pueden clasificar en otros gráficos.

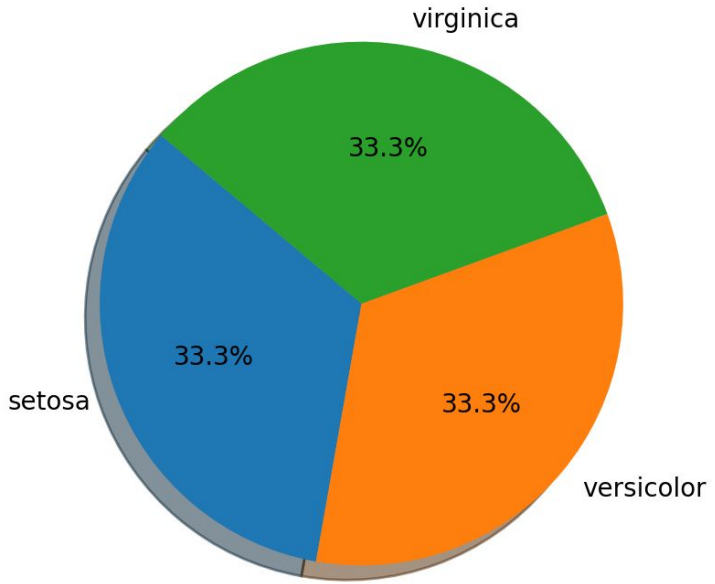
- Visualizando la proporción de clases del objetivo.

[22]:

```
def piechar_iris(feature_names, target, dataset):
    i=1
    plt.figure(figsize=(11,9))
    for colName in [target]:
        labels=[];sizes=[];
        df=dataset.groupby(colName).size()
        for key in df.keys():
            labels.append(key)
            sizes.append(df[key])
        plt.subplot(2,2,i)
        plt.axis('on')
        plt.tick_params(axis='both', left=True,
                        top=False, right=False,
                        bottom=True, labelleft=True,
                        labeltop=True, labelright=False,
                        labelbottom=False)
        plt.pie(sizes, labels=labels, autopct='%1.1f%%',
                shadow=True, startangle=140)
        plt.axis('equal')
        i += 1
    plt.show()
```

- Visualizando la proporción de clases del objetivo.

```
[23]: 1 piechar_iris(iris.columns[:-1], iris.species, iris)
```



Línea 15

- Los datos están organizados uniformemente en cada clase objetivo.

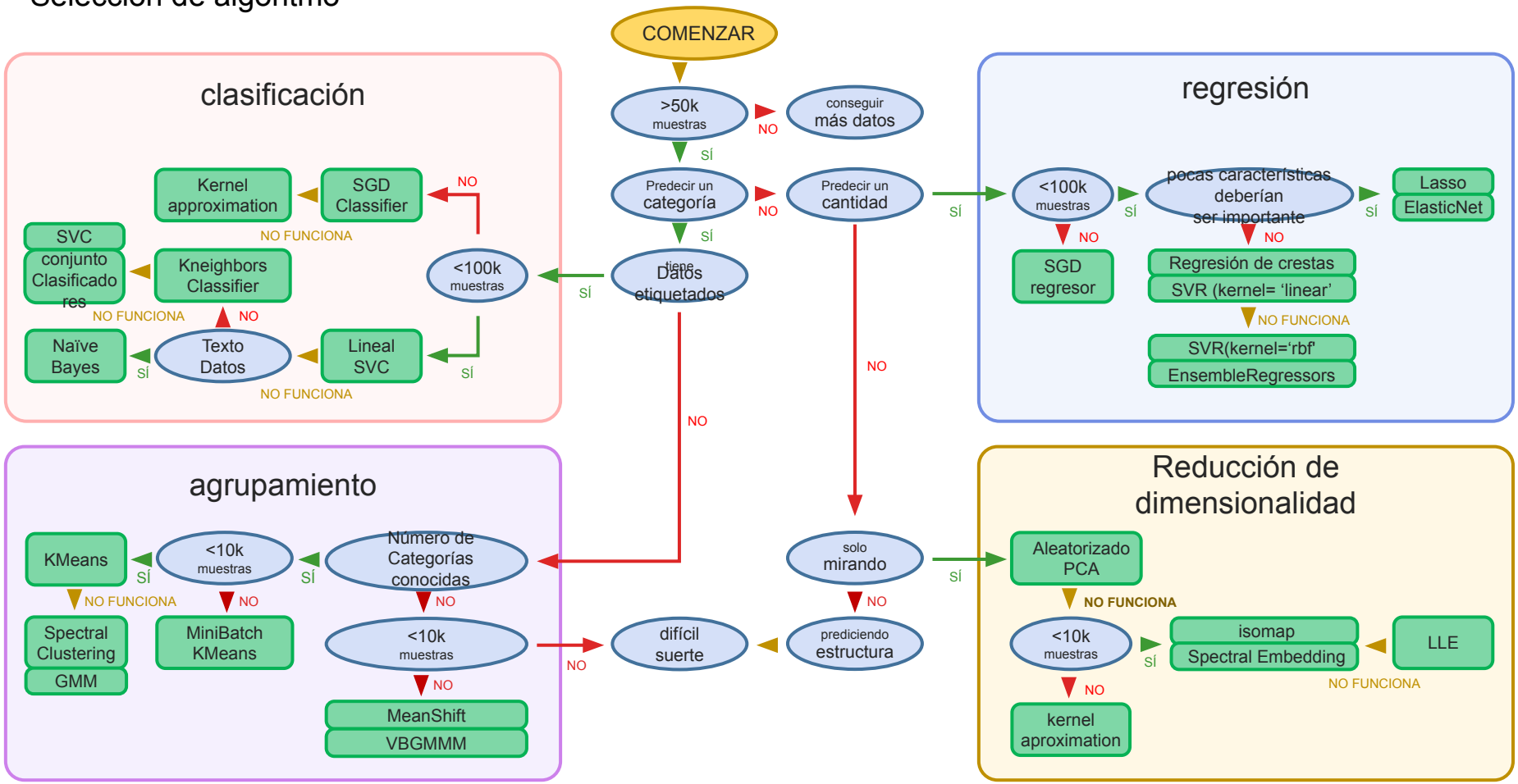
- Dividir el conjunto de datos en datos de prueba de rendimiento y entrenamiento

```
[24]: 1 from sklearn.model_selection import train_test_split

[25]: 1 X_train, X_test, y_train, y_test=train_test_split(iris.iloc[:,:-1], iris.iloc[:,-1],
      2                                                  test_size=0.33, random_state=42)
```

- Antes de iniciar Machine Learning, divida el conjunto de datos en datos de prueba de entrenamiento y rendimiento. El objetivo final de Machine Learning es crear un modelo generalizado para que pueda predecir con precisión nuevos datos. Si se evalúa el desempeño con datos utilizados en el aprendizaje, la posibilidad de hacerlo bien es alta ya que el modelo ya está familiarizado con la característica de datos dada. Para una evaluación confiable, separe el conjunto de datos de prueba de rendimiento del conjunto de datos de entrenamiento. Debido a que se trata de la separación de datos, se le conoce como método de reserva.
- Divide los conjuntos de datos de entrenamiento y de prueba de rendimiento con la función train_test_split de sklearn. Clasifica los datos de entrenamiento como ‘train’ y los datos de prueba de rendimiento como ‘test’. X es la característica del conjunto de datos e y es el objetivo. Para el análisis de datos estructurados, indica el DataFrame con letras mayúsculas y la Series con minúsculas. La opción test_size=0.33 separa el 33% de los datos totales como conjunto de prueba. random_state=42 es una opción utilizada para inducir resultados reproducibles para el problema de práctica. Si no se designa random_state, el conjunto de datos para la conversión será diferente cada vez.

- Selección de algoritmo



- Selección de algoritmo
  - setosa se agrega al desviarse claramente de otras clases. La clasificación es posible trazando una línea imaginaria y setosa se clasificará como un modelo lineal. Para versicolor y virginica, parece difícil clasificarlos trazando una línea porque están mezclados en el gráfico, completo con las características sepal_width y sepal_length. Sin embargo, aunque parezca un poco vago, se pueden clasificar en otros gráficos.

- Selección de algoritmo

[26]:
```
from sklearn.tree import DecisionTreeClassifier
```

[27]:
```
model = DecisionTreeClassifier(criterion='gini',splitter='best',max_depth=None,min_samples_split=2,
                               min_samples_leaf=1,min_weight_fraction_leaf=0.0,max_features=None,
                               random_state=42,max_leaf_nodes=None,min_impurity_decrease=0.0,
                               class_weight=None)
```

- # regularización: restringe el grado de libertad del árbol de decisión of
- # Reducir max_ Depth restringiría el modelo y reduciría el riesgo de sobreajuste.
- # # min_samples_split: cantidad mínima de muestra requerida por el nodo para dividir
- # # min_weight_fraction_leaf: Idéntico a min_samples_leaf, pero es la relación con el peso en toda la muestra.
- # # max_leaf_nodes: número máximo de nodos hoja
- # # max_features: la mayor cantidad de funciones que se utilizarán para dividir por cada nodo
- # Aumentar el parámetro que comienza con min_ o disminuir el parámetro que comienza con max_ aumentaría la restricción del modelo.

- Selección de algoritmo
  - Impureza de Gini o entropía
  - La diferencia entre la impureza de Gini y la entropía es vaga en la vida real. Ambos crean un árbol similar.
  - El cálculo de la impureza de Gini es más rápido, por lo que se recomienda como valor predeterminado.
  - Sin embargo, al crear un árbol diferente, la impureza de Gini tiende a aislar la clase más frecuente a un lado, mientras que la entropía da como resultado un árbol más equilibrado.

- Aprendizaje modelo
  - Realice el aprendizaje del modelo con los datos de entrenamiento para verificar el rendimiento del modelo. El modelo actual está configurado con el hiperparámetro predeterminado excepto random_state.

```
[28]: 1 model.fit(X_train, y_train)

[28]: DecisionTreeClassifier
      DecisionTreeClassifier(random_state=42)
```

- Puntaje
  - Evalúe el rendimiento utilizando el conjunto de datos de prueba de rendimiento. En scikit-learn, la puntuación se refiere a la precisión. Dado que el conjunto de datos del iris está bien estructurado para la práctica, generalmente muestra un alto rendimiento en cualquier modelo.

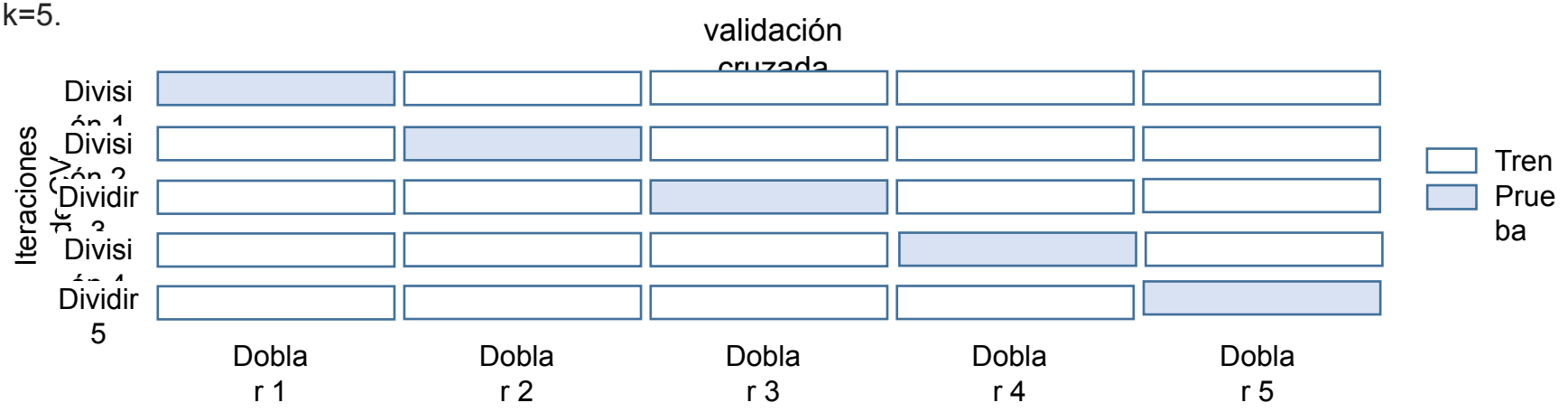
```
[29]: 1 model.score(X_test, y_test)

[29]: 0.98
```

- Estrategia de generalización del modelo.
  - Como Machine Learning muestra el rendimiento del modelo basado en datos, se requiere una cantidad suficiente de datos para un buen rendimiento. Una cantidad insuficiente de datos puede provocar un sobreajuste, lo que significa que el modelo muestra una menor capacidad de predicción con respecto a los datos no vistos porque se ajusta solo a las características de los datos de entrenamiento. La siguiente es la estrategia de generalización para que el modelo proporcione un alto rendimiento con respecto a datos invisibles.
- Conjunto de validación
  - El conjunto de datos de prueba de rendimiento que se dividirá con Train_test_split es para la evaluación final del rendimiento del modelo. Dado que es necesario verificar el rendimiento del modelo durante el aprendizaje del modelo, guarde algunos de los datos del conjunto de datos de entrenamiento y utilícelos como conjunto de validación. Se puede encontrar una posibilidad de sobreajuste durante el aprendizaje utilizando el conjunto de validación. También se utiliza para encontrar hiperparámetros.



- Validación cruzada
  - Esta estrategia consiste en crear múltiples conjuntos de validación para que todos los datos puedan ser incluidos en el aprendizaje al menos una vez. Divide el conjunto de datos en un número aleatorio n=k (k-fold). Utiliza el primer pliegue como conjunto de validación y los otros k-1 pliegues como conjunto de entrenamiento para medir el rendimiento.
  - Utiliza el segundo pliegue como conjunto de prueba y los demás pliegues como conjunto de entrenamiento para el aprendizaje, y luego mide el rendimiento. Repite el mismo proceso para todos los demás pliegues, de modo que todos los datos puedan ser incluidos en el entrenamiento. Obtén k resultados de evaluación del rendimiento y luego calcula el promedio para predecir el rendimiento del modelo. La siguiente figura es un ejemplo de cuando k=5.



- puntuación_val_cruzada
  - La validación cruzada se puede realizar fácilmente utilizando la función cross_val_score de scikit-learn.

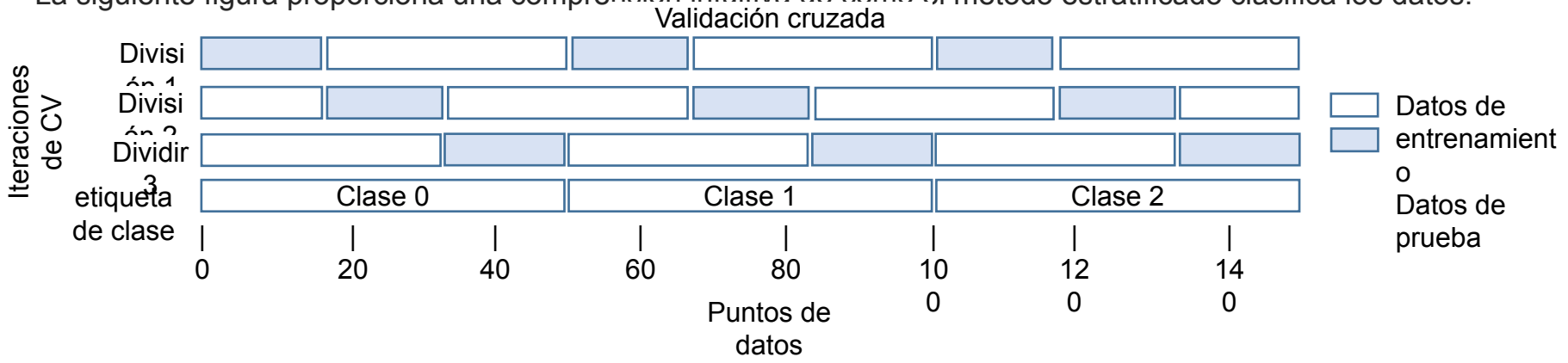
[30]:
```
1 import numpy as np
2 from sklearn.model_selection import cross_val_score, KFold
3 cv=KFold(n_splits=10,shuffle=True,random_state=42)
4 results=cross_val_score(model, X_train, y_train, cv=cv)
5 fin_result=np.mean(results)
```

[31]:
```
1 for i, score in enumerate(results):
2     print(f"{i}th cross validation score: {score}")
3 print(f"\ncross validation final score: {fin_result}")
```

```
0th cross validation score: 0.9
1th cross validation score: 1.0
2th cross validation score: 0.8
3th cross validation score: 1.0
4th cross validation score: 0.8
5th cross validation score: 0.9
6th cross validation score: 1.0
7th cross validation score: 0.9
8th cross validation score: 1.0
9th cross validation score: 1.0

cross validation final score: 0.93
```

- estratificado
  - La división aleatoria del conjunto de trenes y el conjunto de validación daría como resultado una clase objetivo inconstante cuando se resista. Si es así, la distribución de datos difiere en los conjuntos de entrenamiento y validación, lo que afecta el aprendizaje. Para Machine Learning, existe una premisa en la que la distribución de datos de entrenamiento y la distribución de datos de la vida real son las mismas. Si no se sigue la premisa, el rendimiento del modelo de aprendizaje cae. Entonces, para evitar tal problema, se utiliza el método estratificado para distribuir uniformemente la proporción de clases objetivo.
  - La siguiente figura proporciona una comprensión intuitiva de cómo el método estratificado clasifica los datos.



  - La validación cruzada es posible enviando la instancia de StratifiedKFold a la opción cv de cross_val_score.

- estratificado

[32]:
```
from sklearn.model_selection import StratifiedKFold
cv=StratifiedKFold(n_splits=10,shuffle=True,random_state=42)
results=cross_val_score(model, X_train, y_train, cv=cv)
fin_result=np.mean(results)
for i, score in enumerate(results):
    print(f"{i}th cross validation score: {score}")
```

```
0th cross validation score: 0.9
1th cross validation score: 0.9
2th cross validation score: 0.8
3th cross validation score: 0.9
4th cross validation score: 1.0
5th cross validation score: 1.0
6th cross validation score: 0.9
7th cross validation score: 0.8
8th cross validation score: 1.0
9th cross validation score: 1.0
```

[33]:
```
print(f"cross validation final score: {fin_result}")
```

```
cross validation final score: 0.9199999999999999
```

- Curva de aprendizaje
  - !pip instalar scikit-plot

```
[34]: 1 !pip install scikit-plot
```

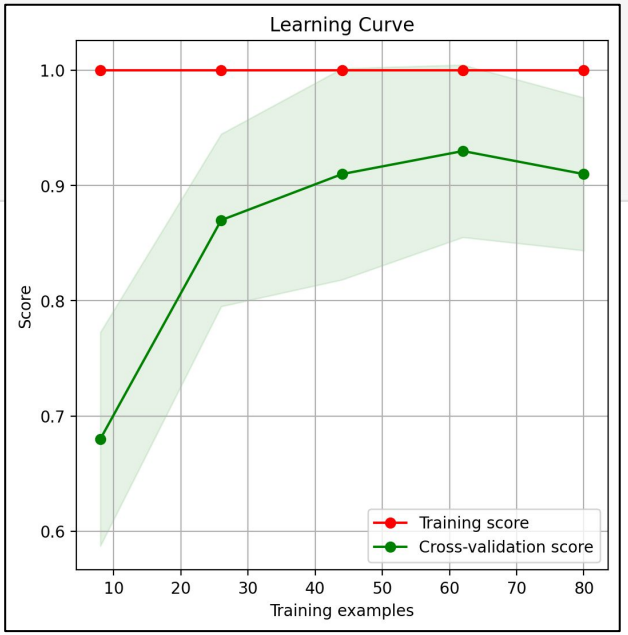
```
Requirement already satisfied: scikit-plot in c:\users\jangrae\anaconda3\lib\site-packages (0.3.7)
Requirement already satisfied: matplotlib>=1.4.0 in c:\users\jangrae\anaconda3\lib\site-packages (from scikit-plot) (3.10.0)
Requirement already satisfied: scikit-learn>=0.18 in c:\users\jangrae\anaconda3\lib\site-packages (from scikit-plot) (1.6.1)
Requirement already satisfied: scipy>=0.9 in c:\users\jangrae\anaconda3\lib\site-packages (from scikit-plot) (1.15.3)
Requirement already satisfied: joblib>=0.10 in c:\users\jangrae\anaconda3\lib\site-packages (from scikit-plot) (1.4.2)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\jangrae\anaconda3\lib\site-packages (from matplotlib>=1.4.0->scikit-plot) (1.3.1)
Requirement already satisfied: cycler>=0.10 in c:\users\jangrae\anaconda3\lib\site-packages (from matplotlib>=1.4.0->scikit-plot) (0.11.0)
Requirement already satisfied: fonttools>=4.22.0 in c:\users\jangrae\anaconda3\lib\site-packages (from matplotlib>=1.4.0->scikit-plot) (4.55.
3)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\jangrae\anaconda3\lib\site-packages (from matplotlib>=1.4.0->scikit-plot) (1.4.8)
Requirement already satisfied: numpy>=1.23 in c:\users\jangrae\anaconda3\lib\site-packages (from matplotlib>=1.4.0->scikit-plot) (2.1.3)
Requirement already satisfied: packaging>=20.0 in c:\users\jangrae\anaconda3\lib\site-packages (from matplotlib>=1.4.0->scikit-plot) (24.2)
Requirement already satisfied: pillow>=8 in c:\users\jangrae\anaconda3\lib\site-packages (from matplotlib>=1.4.0->scikit-plot) (11.1.0)
Requirement already satisfied: pyparsing>=2.3.1 in c:\users\jangrae\anaconda3\lib\site-packages (from matplotlib>=1.4.0->scikit-plot) (3.2.0)
Requirement already satisfied: python-dateutil>=2.7 in c:\users\jangrae\anaconda3\lib\site-packages (from matplotlib>=1.4.0->scikit-plot) (2.
9.0.post0)
Requirement already satisfied: six>=1.5 in c:\users\jangrae\anaconda3\lib\site-packages (from python-dateutil>=2.7->matplotlib>=1.4.0->scikit-
plot) (1.17.0)
Requirement already satisfied: threadpoolctl>=3.1.0 in c:\users\jangrae\anaconda3\lib\site-packages (from scikit-learn>=0.18->scikit-plot) (3.
5.0)
```

- Curva de aprendizaje
  - !pip instalar scikit-plot
    - La línea verde es el resultado de la validación cruzada. El sobreajuste ocurre cuando la línea verde sube hacia la derecha pero luego comienza a caer. La línea roja son datos utilizados para la capacitación que están validados. La línea roja puede caer momentáneamente si hay demasiados datos. Este fenómeno es sólo temporal y el gráfico convergerá en el largo plazo.
    - La opción CV no está designada, por lo que se aplica 3 veces de forma predeterminada. Hay 100 datos en el conjunto de entrenamiento y el 33% de ellos se usa para validación cruzada, por lo que el valor máximo del eje x es 66. La curva de entrenamiento se desconecta a medida que aumenta la línea verde, por lo que se desconoce si hay más datos. En este punto, es imposible saber si hay suficientes datos. La curva de entrenamiento se dibuja de forma diferente según el algoritmo, incluso cuando se utilizan los mismos datos. Por tanto, lo que se sabe de esta curva de entrenamiento es que el rendimiento del modelo de árbol de decisión actual sería mejor si hubiera más datos.

- Curva de aprendizaje
  - !pip instalar scikit-plot

[35]:
```
1 import numpy as np
2 import scipy
3 scipy.interp = np.interp
4 import scikitplot as skplt
```

[36]:
```
1 import matplotlib.pyplot as plt
2 skplt.estimators.plot_learning_curve(model,
3                                      X_train, y_train,
4                                      figsize=(6,6))
5 plt.show()
```



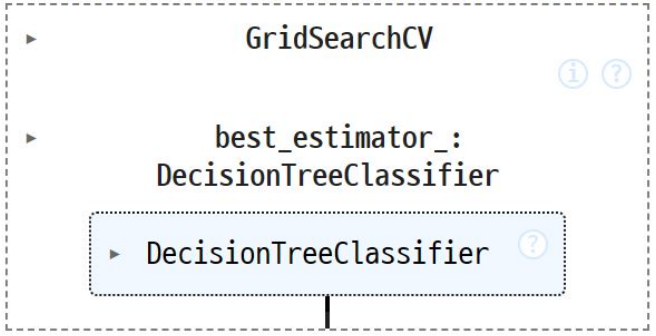
- Curva de aprendizaje
  - Si hay suficientes datos, se mantiene una distribución de datos idéntica incluso cuando los conjuntos de entrenamiento y validación se dividen aleatoriamente. El método de validación cruzada se requiere cuando no hay datos suficientes. Dibuje una curva de aprendizaje para determinar si hay suficientes datos. La curva de aprendizaje muestra cómo cambia el rendimiento cuando se aumenta ligeramente la cantidad de datos de entrenamiento configurando el eje x como el número de datos de entrenamiento y el eje y como la puntuación de rendimiento. La puntuación de la prueba se calcula mediante validación cruzada interna.
  - La curva de aprendizaje se puede dibujar utilizando scikitplot library, que admite scikit-learn. Instale el library por separado de scikit-learn para usarlo.
  - scikitplot no se proporciona en anaconda de forma predeterminada, por lo que debe instalarse utilizando las herramientas de administración de paquetes. Ejecute el siguiente código con Jupyter Notebook para instalar library. Tenga en cuenta los diferentes nombres al instalar e importar library.

- Estrategia de optimización del modelo.

[37]:
```
estimator=DecisionTreeClassifier()
from sklearn.model_selection import GridSearchCV
parameters={'max_depth': [4,6,8,10,12],
            'criterion': ['gini', 'entropy'],
            'splitter': ['best', 'random'],
            'min_weight_fraction_leaf': [0.0,0.1,0.2,0.3],
            'random_state': [7,23,42,78,142],
            'min_impurity_decrease': [0.0,0.05,0.1,0.2]}
model2=GridSearchCV(estimator=estimator,
                    param_grid=parameters,
                    cv=KFold(10), verbose=1,
                    n_jobs=-1, refit=True)
model2.fit(X_train, y_train)
```

```
Fitting 10 folds for each of 1600 candidates, totalling 16000 fits
```

[37]:



- Estrategia de optimización del modelo.

#hiperparámetro

- En Machine Learning, la máquina aprende los datos y encuentra los parámetros por sí misma. Los hiperparámetros se refieren a parámetros que deben ser designados directamente por un humano ya que la máquina no puede encontrarlos. En scikit-learn, es posible configurar hiperparámetros para crear una instancia de un algoritmo.

#Búsqueda de hiperparámetros usando GridSearchCV

- En general, los hiperparámetros se encuentran mediante la experiencia del analista. Scikit-learn proporciona la función GridSearchCV que encuentra hiperparámetros. Es una función que enumera todos los casos relacionados con combinaciones de hiperparámetros en una cuadrícula y aprende y realiza medidas de rendimiento para cada combinación. Puede parecer que se trabaja sin ningún plan. Sin embargo, la máquina realiza automáticamente el trabajo según el rango de hiperparámetros designado por el analista. Aunque lleva algo de tiempo, facilita la búsqueda de hiperparámetros.

- Similar a los algoritmos, también puede utilizar GridSearchCV. Al crear una instancia, envíe el modelo del algoritmo instanciado como argumento a la opción del estimador. Para param_grid, envíe el diccionario que contiene hiperparámetros para probar como argumento.

- The total number of cases that can possibly be made with the parameters from the practice problem is 1600. Since k=10 in the K-fold cross-validation, 10 cross-validations were performed for each case; a total of 16,000 training were done. The following table shows hyperparameter combinations in the practice problem.
  - Los parámetros óptimos y el rendimiento optimizado encontrados con GridSearchCV se registran en los atributos best_params_y best_score_.
  - Si la opción de reajuste está configurada en Verdadero, entrene el modelo con los hiperparámetros óptimos y regístrelo en el atributo best_estimator_.

	0	1	2	3	4	···	1595	1596	1597	1598	1599
criterio	gini	gini	gini	gini	gini	···	entropía	entropía	entropía	entropía	entropía
profundidad_max	4	4	4	4	4	···	12	12	12	12	12
min_impuridad_disminución	0	0	0	0	0	···	0.2	0.2	0.2	0.2	0.2
hoja_fracción_aleatoria	0	0	0	0	0	···	0.3	0.3	0.3	0.3	0.3
estado_aleatorio	7	7	23	23	42	···	42	78	78	142	142
disidente	mejor	aleatorio	mejor	aleatorio	mejor	···	aleatorio	mejor	aleatorio	mejor	aleatorio

6 filas * 1600 columnas

```
[38]: 1 model2.best_estimator_
```

```
[38]: DecisionTreeClassifier
DecisionTreeClassifier(max_depth=4, random_state=23, splitter='random')
```

```
[39]: 1 model2.best_params_
```

```
[39]: {'criterion': 'gini',
 'max_depth': 4,
 'min_impurity_decrease': 0.0,
 'min_weight_fraction_leaf': 0.0,
 'random_state': 23,
 'splitter': 'random'}
```

```
[40]: 1 model2.best_score_
```

```
[40]: np.float64(0.9700000000000001)
```

- Criterios de evaluación y evaluación del modelo.
  - Utilice X_test e y_test de Holdout para la evaluación final del modelo. Para una evaluación precisa, es importante conocer diferentes tipos de criterios de evaluación.

```
[40]: 1 model2.best_score_

[40]: np.float64(0.9700000000000001)

[41]: 1 from sklearn.metrics import accuracy_score
      2 pred=model2.predict(X_test)
      3 pred[:20]

[41]: array([np.str_('versicolor'), np.str_('setosa'), np.str_('virginica'),
             np.str_('versicolor'), np.str_('versicolor'), np.str_('setosa'),
             np.str_('versicolor'), np.str_('virginica'), np.str_('versicolor'),
             np.str_('versicolor'), np.str_('virginica'), np.str_('setosa'),
             np.str_('setosa'), np.str_('setosa'), np.str_('setosa'),
             np.str_('versicolor'), np.str_('virginica'), np.str_('versicolor'),
             np.str_('versicolor'), np.str_('virginica')], dtype=object)

[42]: 1 accuracy_score(y_test, pred)

[42]: 0.98
```

- Criterios de evaluación y evaluación del modelo.
  - #Limitaciones de precisión
    - Hasta ahora, la precisión era el único criterio para validar el modelo, pero existe una limitación. Se requiere más que precisión para evaluar el modelo adecuadamente.

      Ex Si hay un modelo que predice "setosa" sólo incluso cuando se ingresan varios tipos de datos, el rendimiento del modelo sería dudoso.

    - Sin embargo, supongamos que hay 48 setosas, 1 versicolor y 1 virginica en el conjunto de prueba. Al realizar una evaluación utilizando este conjunto de pruebas, el problema es que tendría un 96% de precisión. Sin embargo, no es porque el rendimiento del modelo sea excelente. Sería necesario comprobar también otros criterios de evaluación para evaluar con precisión el rendimiento del modelo.

- Matriz de confusión
  - La siguiente matriz de confusión se puede expresar con clasificación binaria.

	Clase positiva prevista	Clase negativa prevista
Positivo real	TP (verdadero positivo)	FN (falso negativo)
Negativo real	FP (falso positivo)	TN (Verdadero Negativo)

  - Las puntuaciones de evaluación, incluida la precisión, la recuperación, la f-score1 y otras, se pueden realizar basándose en los conceptos mencionados anteriormente (TP, FP, TN, FN).
  - Utilice la matriz de confusión para analizar los resultados previstos tanto correctos como incorrectos. La matriz de confusión puede validar el desempeño de manera diferente para ver qué tan bien se acertaron los objetivos previstos y reales.

- Clasificación de etiquetas múltiples

	Setosa prevista	Versicolor predicho	virginica predicha
Setosa real	Setosa real y setosa prevista	Setosa real pero versicolor predicho	Setosa real pero virginica predicha
versicolor real	Versicolor real pero setosa predicha	Versicolor real y versicolor previsto	Versicolor real pero virginica predicha
virginica real	Virginia real pero setosa predicha	Virginia real pero versicolor predicha	Virginia real y virginica predicha

- Dado que los datos del iris son un problema de clasificación de etiquetas múltiples, no se pueden expresar en cuatro conceptos diferentes únicamente como se proporcionó anteriormente. Entonces, cree tres índices para cada setosa, versicolor, virginica considerando cada uno como un problema de clasificación binaria. Tomemos como ejemplo a setosa.

- Matriz de confusión del conjunto de datos del iris.

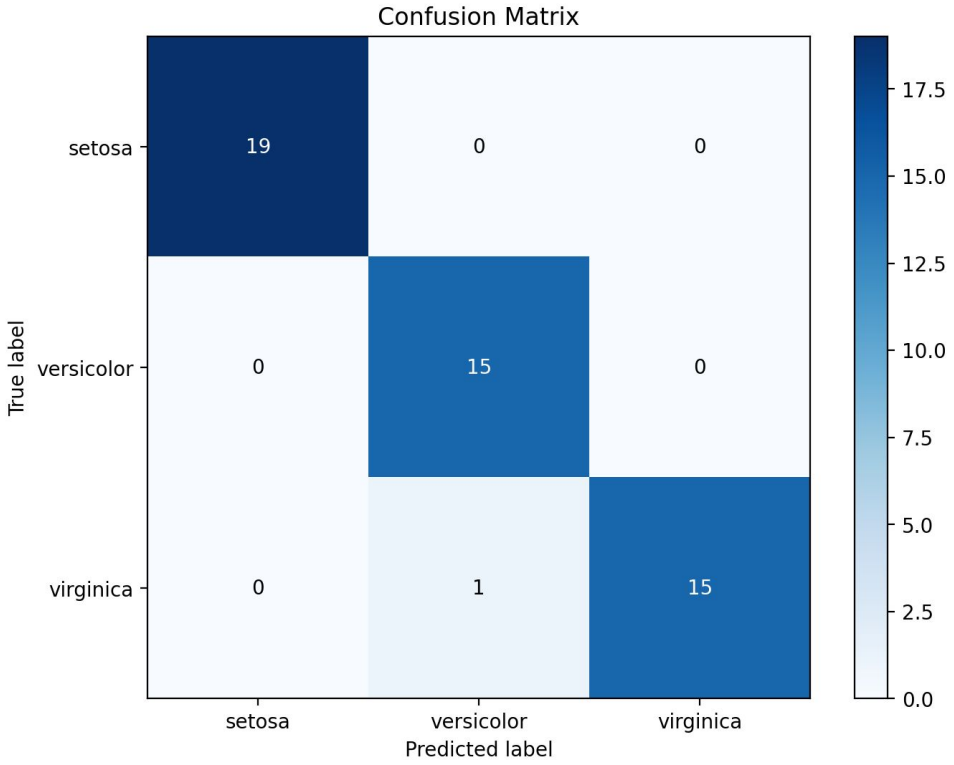
```
[43]: 1 from sklearn.metrics import confusion_matrix
      2 pred=model2.predict(X_test)
      3 confusion_matrix(y_test, pred)
```

```
[43]: array([[19,  0,  0],
             [ 0, 15,  0],
             [ 0,  1, 15]])
```

- Con scikit-learn, es posible calcular fácilmente la matriz de confusión utilizando la función confusion_matrix. Envíe los argumentos a la clase real y luego a la clase prevista.

- Matriz de confusión del conjunto de datos del iris.

[44]:
```
1 import scikitplot as skplt
2 skplt.metrics.plot_confusion_matrix(y_test, pred, figsize=(8,6))
3 plt.show()
```



- Matriz de confusión del conjunto de datos del iris.
  - Utilice el diagrama de scikit para visualizar la matriz de confusión en un mapa de calor más intuitivo. El scikit-learn no tenía etiquetas para los ejes x e y. Por otro lado, scikit-plot tiene etiquetas para el eje para facilitar la interpretación de los resultados.

- precision / recall / fall-out / f-score
  - Las puntuaciones de evaluación difieren en cada clase objetivo en un problema de clasificación de etiquetas múltiples.

	Setosa prevista	Setosa no prevista (versicolor o virginica previsto) or
Setosa real	TP TP (verdadero positivo)	FN (falso negativo)
No versicolor real (versicolor o virginica) or	FP (falso positivo)	TN (Verdadero Negativo)

  - Califique los resultados de la evaluación según los TP, TN, FP y FN de la matriz de confusión. Estos cuatro conceptos sólo son posibles en problemas de clasificación binaria. Para problemas de clasificación de etiquetas múltiples que tienen N clases de objetivos, como datos de iris, considere cada clase de objetivos como una clasificación binaria y obtenga N matrices de confusión.

    datos del iris

    Ex Considere cada setosa, versicolor y virginica como un problema de clasificación binaria y cree tres matrices de confusión.

    A continuación se muestra la matriz de confusión de setosa.

- precisión
  - La precisión es la proporción de la clase predicha correcta.

[45]:
```
1 from sklearn.metrics import precision_score
2 precisions=precision_score(y_test, pred, average=None)
3
4 for target, score in zip(data.target_names, precisions):
5     print(f"{target} precision:{score}")
```

```
setosa precision: 1.0
versicolor precision: 0.9375
virginica precision: 1.0
```

Línea 45

- En la clasificación de etiquetas múltiples, el promedio no puede ser "binario".
- "binario" es el valor predeterminado promedio.

- recall
  - También llamada sensibilidad, la recuperación es la proporción de predicción correcta entre la clase objetivo real.

[46]:
```
from sklearn.metrics import recall_score
recalls=recall_score(y_test, pred, average=None)
for target, score in zip(data.target_names, recalls):
    print(f"{target}sensitivity:{score}")
```

```
setosa sensitivity: 1.0
versicolor sensitivity: 1.0
virginica sensitivity: 0.9375
```

- fall-out
  - Las consecuencias son la proporción incorrecta entre la clase real, no el objetivo. También expresado como 1-especificidad.

  - Scikit-learn no proporciona cómo calcular las consecuencias.

- f-score
  - *Precision* y *recall* tienen una relación de compensación. La f-score es la media armónica ponderada de precisión y recuperación. Si la f-score es inferior a 1, se le da más peso a la precisión; si es mayor que 1, se proporciona más peso para recuperar. La f-score se utiliza para comprender con precisión el rendimiento del modelo cuando la clase de datos está desequilibrada.

  - Para un peso uniforme de precisión y recuperación(recall), β se establece en 1 la mayor parte del tiempo, lo que se conoce específicamente como f1-score.

$$F_1 = 2 \cdot \frac{precision \times recall}{precision + recall}$$

- f-score
  - Medida #F1: tanto la precisión como la recuperación tienen el mismo peso. La f-score1 se utiliza para calcular el promedio a partir de la media armónica de precisión y recuperación (sensibilidad) y la precisión y recuperación de ponderación.
  - #a (precision), b (recall) 2ab/a+b
  - Medida #F0.5: precision tiene más peso que recall. Se aplica 0,5 veces más peso al recall en comparación con la precisión.
  - Medida #F2: recall tiene más peso. Recall tiene 2 veces más peso que la precisión.

- f-score

[47]:

```
1  from sklearn.metrics import fbeta_score, f1_score
2
3  fbetas=fbeta_score(y_test, pred, beta=1, average=None)
4
5  for target, score in zip(data.target_names, fbetas):
6      print(f"{target} fbetas score: {score}")
7
8  f1s=f1_score(y_test, pred, average=None)
9
10 for target, score in zip(data.target_names, f1s):
11     print(f"{target} f1 score: {score}")
```

```
setosa fbetas score: 1.0
versicolor fbetas score: 0.967741935483871
virginica fbetas score: 0.967741935483871
setosa f1 score: 1.0
versicolor f1 score: 0.967741935483871
virginica f1 score: 0.967741935483871
```

- exactitud

- classification_report
  - Utilice la función Classification_Report de scikit-learn para calcular por lotes la precisión, la recuperación y la f-score1.

```
[48]: 1 from sklearn.metrics import classification_report
      2 print(classification_report(y_test, pred))
```

```
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        19
  versicolor       0.94      1.00      0.97        15
   virginica       1.00      0.94      0.97        16

    accuracy                           0.98        50
   macro avg       0.98      0.98      0.98        50
weighted avg       0.98      0.98      0.98        50
```

- curva ROC
  - La curva ROC tiene TPR (tasa de verdaderos positivos) en el eje y y FPR (tasa de falsos positivos) en el eje x. TPR es retiro y FPR se refiere a consecuencias.

- curva ROC

```
[49]: 1 pred_proba=model2.predict_proba(X_test)

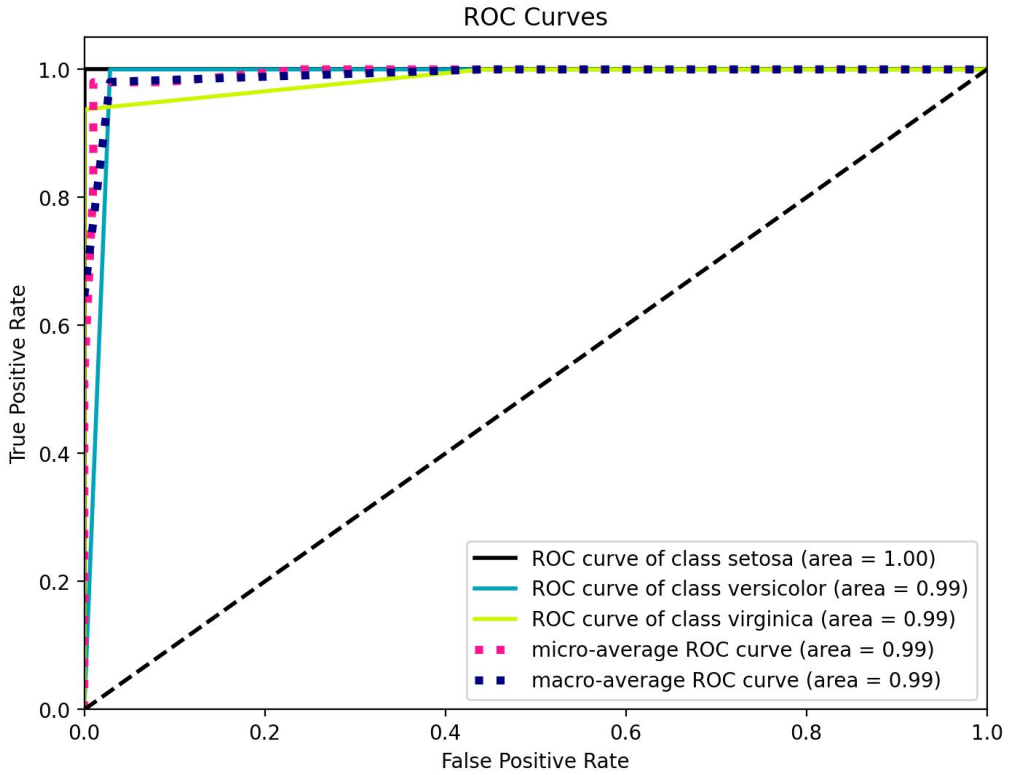
[50]: 1 pred_proba

[50]: array([[0.        , 0.93939394, 0.06060606],
             [1.        , 0.        , 0.        ],
             [0.        , 0.        , 1.        ],
             [0.        , 0.93939394, 0.06060606],
             [0.        , 0.93939394, 0.06060606],
             [1.        , 0.        , 0.        ],
             [0.        , 0.93939394, 0.06060606],
             [0.        , 0.        , 1.        ],
             [0.        , 0.93939394, 0.06060606],
             [0.        , 0.93939394, 0.06060606],
             [0.        , 0.0625    , 0.9375    ],
             [1.        , 0.        , 0.        ],
             [1.        , 0.        , 0.        ],
             [1.        , 0.        , 0.        ],
             [1.        , 0.        , 0.        ],
             [0.        , 0.93939394, 0.06060606],
             [0.        , 0.0625    , 0.9375    ],
```

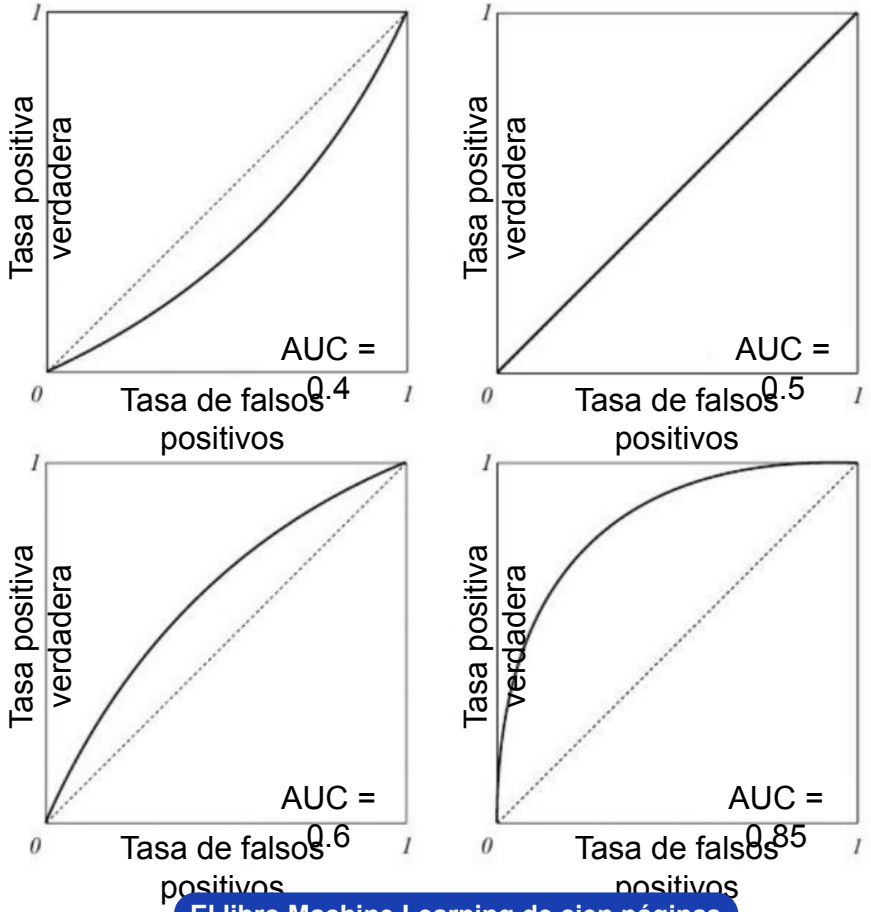
- curva ROC

[51]:

```
1 import scikitplot as skplt
2 skplt.metrics.plot_roc(y_test, pred_proba, figsize=(8,6))
3 plt.show()
```



- AUC (área bajo la curva)



El libro Machine Learning de cien páginas

- Mal rendimiento: volver al paso anterior
  - modelo final
  - Guardar modelo: lleva días saber si hay muchos datos. Es extremadamente ineficiente entrenar el modelo cada vez para realizar predicciones, por lo que la mejor manera es guardar el modelo para reutilizarlo. Utilice 'pickle'

[52]:
```
model.fit(iris.iloc[:,:-1], iris.iloc[:,-1])
```

[52]:
```
DecisionTreeClassifier
DecisionTreeClassifier(random_state=42)
```

[53]:
```
import pickle
```

[54]:
```
with open('final_model.pickle', 'wb') as fp:
    pickle.dump(model2, fp)
```

[55]:
```
f=open('final_model.pickle', 'rb')
model3=pickle.load(f)
```

[56]:
```
predicted_species=model3.predict(iris.iloc[:,:-1])
```

[57]:
```
iris['predicted_species']=predicted_species
iris.to_csv('FinalResult.csv', index=False)
```

- Mal rendimiento: volver al paso anterior
  - modelo final

```
[52]: model.fit(iris.iloc[:,:-1], iris.iloc[:,-1])
```

```
[52]: DecisionTreeClassifier
      DecisionTreeClassifier(random_state=42)
```

```
[53]: import pickle
```

```
[54]: with open('final_model.pickle', 'wb') as fp:
          pickle.dump(model2, fp)
```

```
[55]: f=open('final_model.pickle', 'rb')
      model3=pickle.load(f)
```

```
[56]: predicted_species=model3.predict(iris.iloc[:,:-1])
```

```
[57]: iris['predicted_species']=predicted_species
      iris.to_csv('FinalResult.csv', index=False)
```

Línea 55

- Modelo de importación.

- Mal rendimiento: volver al paso anterior
  - modelo final

```
[52]: model.fit(iris.iloc[:,:-1], iris.iloc[:,-1])
```

```
[52]: DecisionTreeClassifier
      DecisionTreeClassifier(random_state=42)
```

```
[53]: import pickle
```

```
[54]: with open('final_model.pickle', 'wb') as fp:
          pickle.dump(model2, fp)
```

```
[55]: f=open('final_model.pickle', 'rb')
      model3=pickle.load(f)
```

```
[56]: predicted_species=model3.predict(iris.iloc[:,:-1])
```

```
[57]: iris['predicted_species']=predicted_species
      iris.to_csv('FinalResult.csv', index=False)
```

Línea 56

- Predicción final

- Mal rendimiento: volver al paso anterior
  - modelo final

```
[52]: model.fit(iris.iloc[:,:-1], iris.iloc[:,-1])
```

```
[52]: DecisionTreeClassifier
      DecisionTreeClassifier(random_state=42)
```

```
[53]: import pickle
```

```
[54]: with open('final_model.pickle', 'wb') as fp:
          pickle.dump(model2, fp)
```

```
[55]: f=open('final_model.pickle', 'rb')
      model3=pickle.load(f)
```

```
[56]: predicted_species=model3.predict(iris.iloc[:,:-1])
```

```
[57]: iris['predicted_species']=predicted_species
      iris.to_csv('FinalResult.csv', index=False)
```

Línea 57

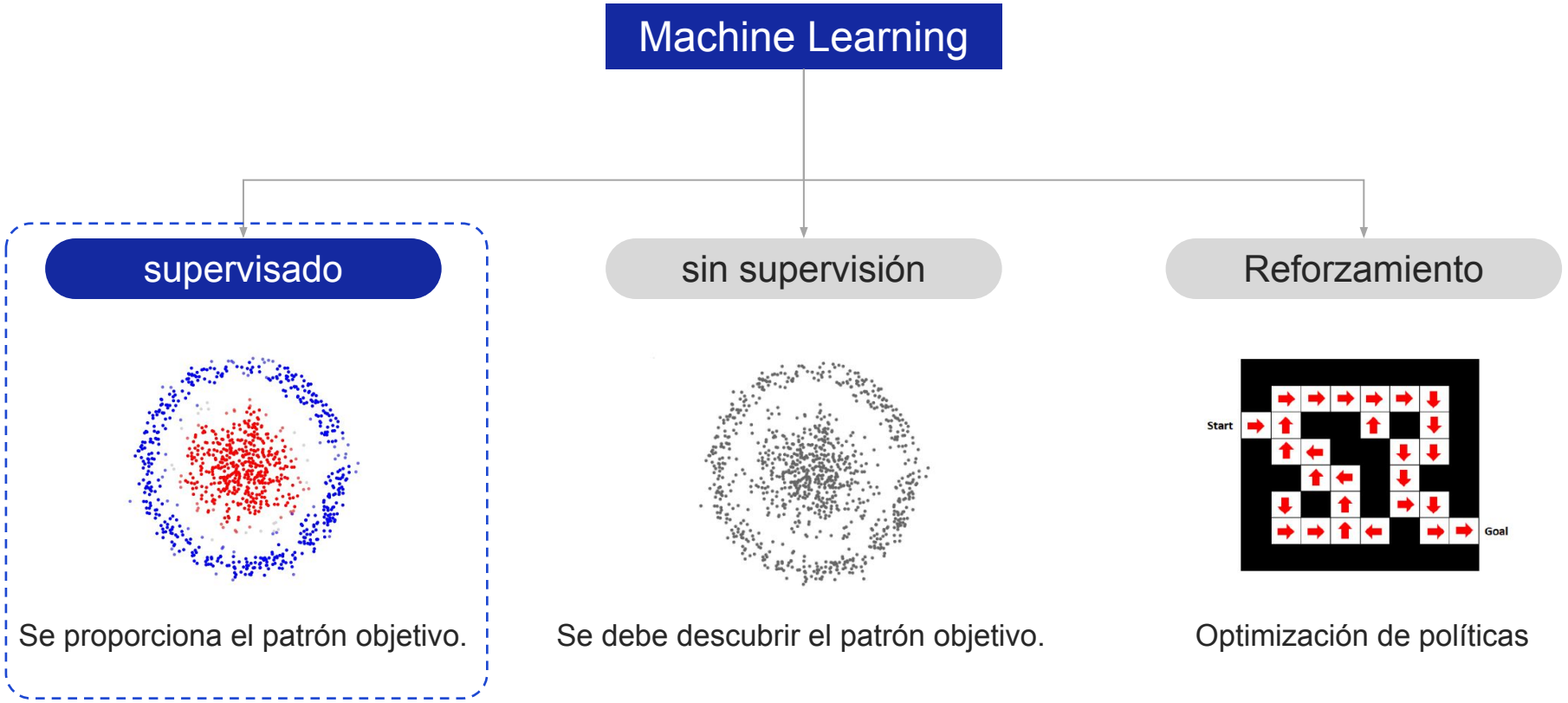
- Guardar csv.

Unidad 2.

Aplicación del modelo de aprendizaje supervisado para la predicción numérica

- 2.1. Capacitación y pruebas en Machine Learning
- 2.2. Conceptos básicos de la regresión lineal
- 2.3. Diagnóstico de regresión lineal
- 2.4. Otros tipos de regresión
- 2.5. Práctica del modelo de aprendizaje supervisado para la predicción numérica

Machine Learning Tipos

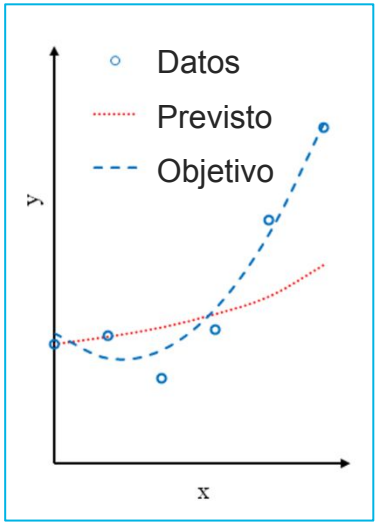


Capacitación y pruebas en Machine Learning

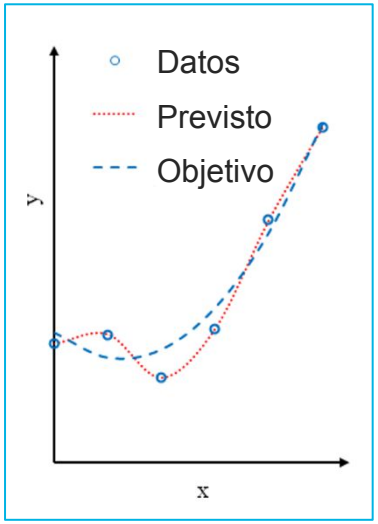
- ¿Qué es el aprendizaje (humano)?
  - El verdadero aprendizaje no es sólo tener buena memoria.
  - El material aprendido debe ponerse a prueba.
- Entrenamiento y pruebas en Machine Learning
  - Se debe poner a prueba un modelo entrenado para generalizar.
  - El resultado de la prueba se mide por los errores.

Tipos de errores

- Error de sesgo (error de ajuste insuficiente)
  - Asociado a modelos simples/rígidos/sesgados.
  - La predicción no puede dar cuenta del patrón de datos detallado.
  - Para reducir este tipo de error, aumente la complejidad del modelo.

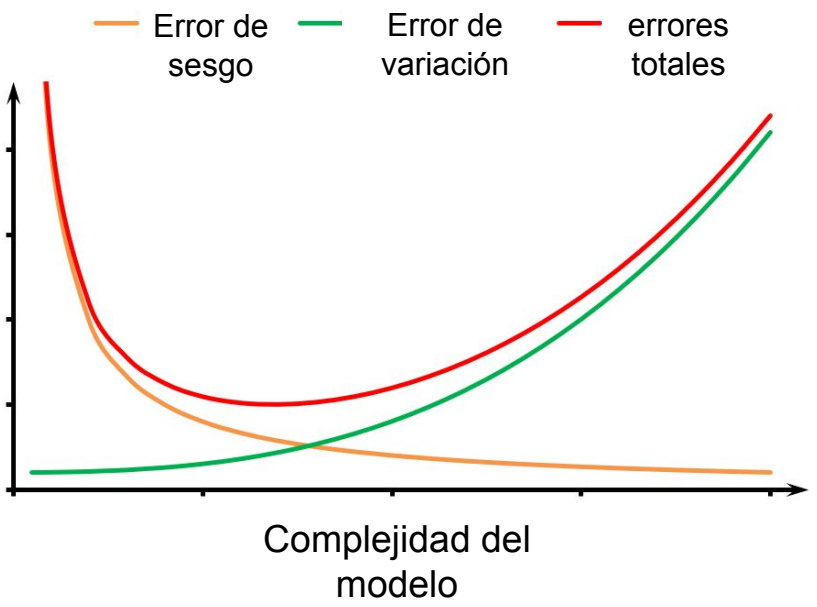


- Error de varianza (error de sobreajuste)
  - Asociado a modelos demasiado complejos y sensibles al ruido.
  - El rendimiento de la predicción es bueno durante el entrenamiento, pero empeora cuando se prueba con un conjunto de datos diferente.
  - Para reducir este tipo de error, aumente la cantidad de datos de entrenamiento (*) o disminuya la complejidad del modelo.



(*) Por método de aumento de datos, por ejemplo.

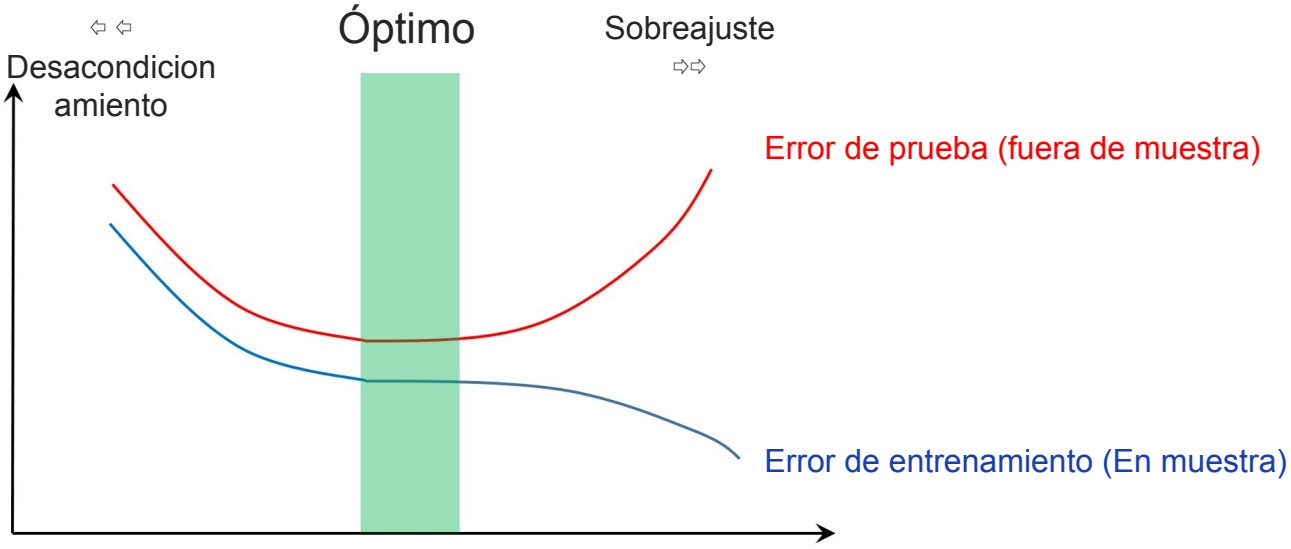
- error total



- The goal is to minimize the **Total error** = **Bias error** + **Variance error**.
- Se requiere suficiente complejidad para "optimizar" el modelo.

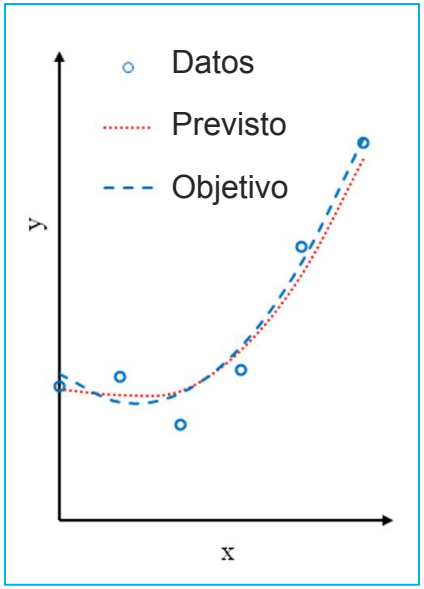
Minimizar errores

- Modelo Machine Learning optimizado
  - El rendimiento de la predicción debe ser bueno tanto en el entrenamiento como en las pruebas.
  - Dado un algoritmo Machine Learning, existe un modelo con la complejidad suficiente (*).



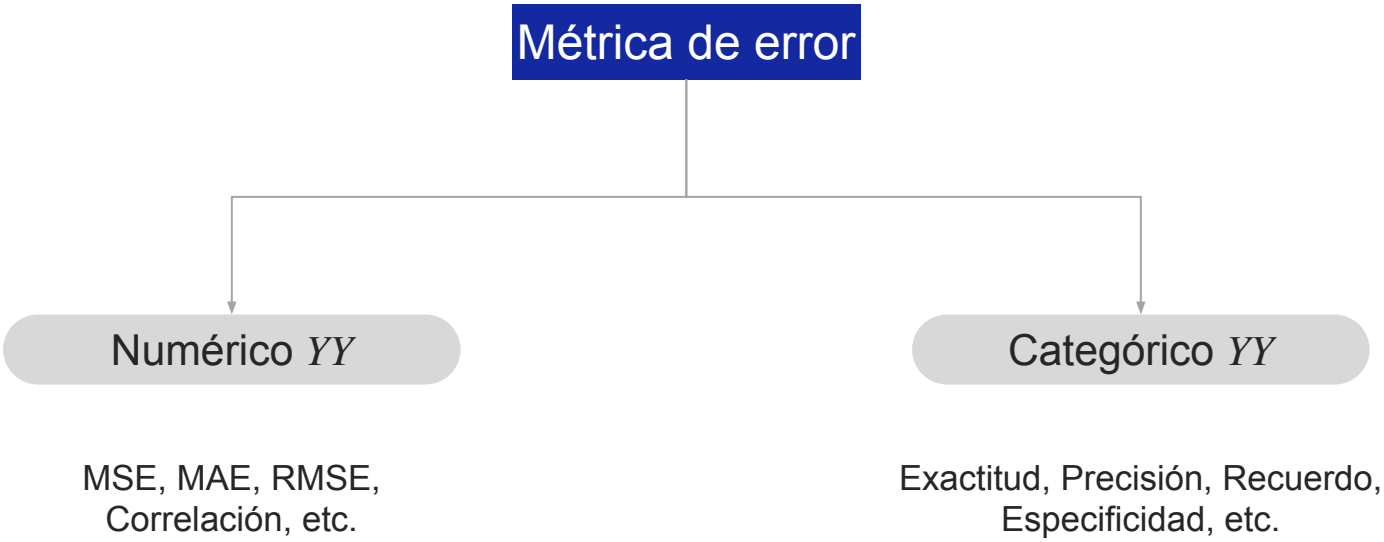
(*) La complejidad suele estar controlada por el número de parámetros.

- Modelo Machine Learning optimizado
  - El rendimiento de la predicción debe ser bueno tanto en el entrenamiento como en las pruebas.
  - Dado un algoritmo Machine Learning, existe un modelo con la complejidad suficiente (*).



(*) La complejidad suele estar controlada por el número de parámetros.

Machine Learning Tipos

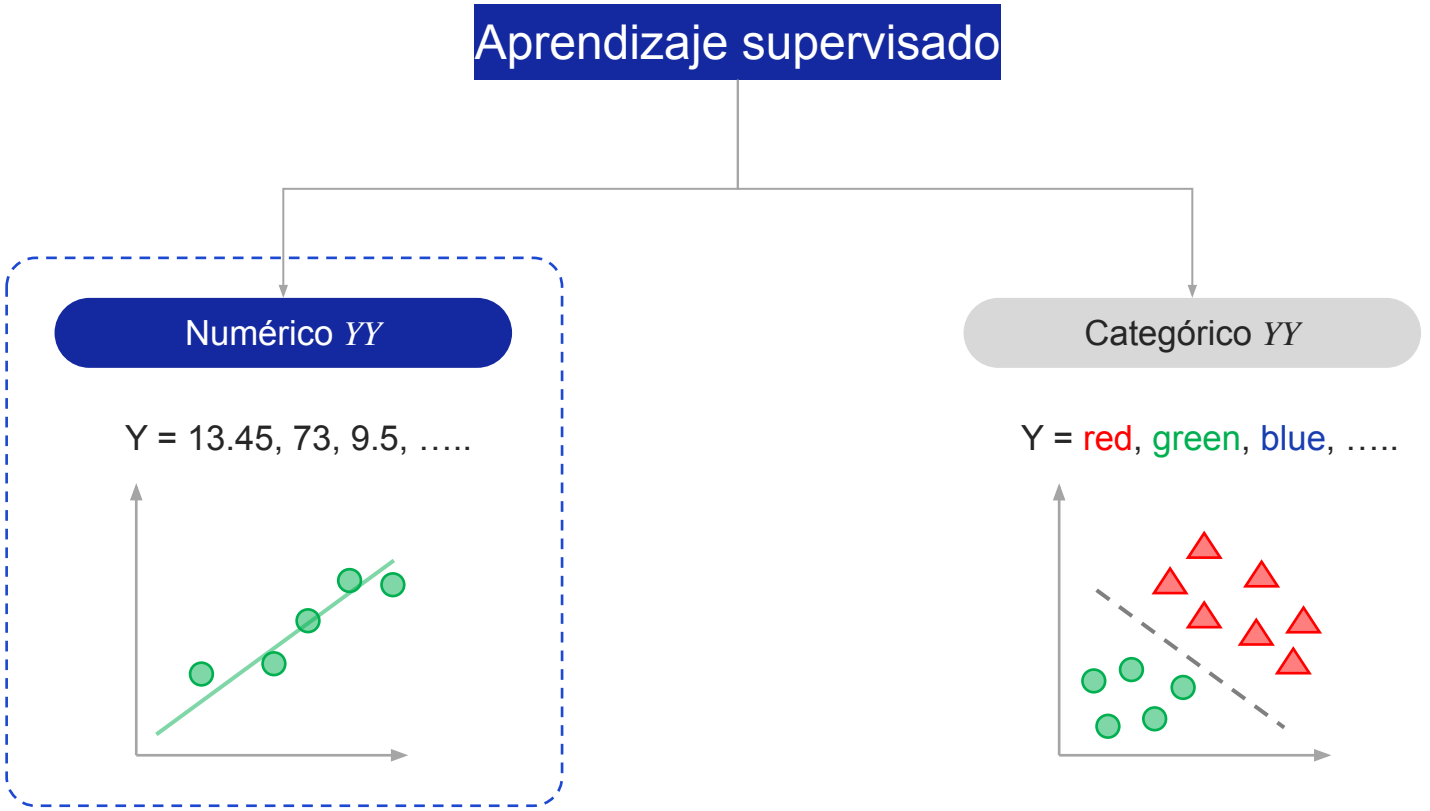


Unidad 2.

Aplicación del modelo de aprendizaje supervisado para la predicción numérica

- 2.1. Capacitación y pruebas en Machine Learning
- 2.2. Conceptos básicos de la regresión lineal
- 2.3. Diagnóstico de regresión lineal
- 2.4. Otros tipos de regresión
- 2.5. Práctica del modelo de aprendizaje supervisado para la predicción numérica

Conceptos básicos de la regresión lineal



- Acerca de la regresión lineal
  - Existen una o más variables explicativas: X_1 , X_2 ,..., X_k
  - Existe una variable de respuesta: Y
  - Las variables X_i y Y están conectadas por una relación lineal: $Y=\beta_0+\beta_1 X_1+\beta_2 X_2+\cdots+\beta_k X_k+ \varepsilon$

- Propósito de la regresión lineal

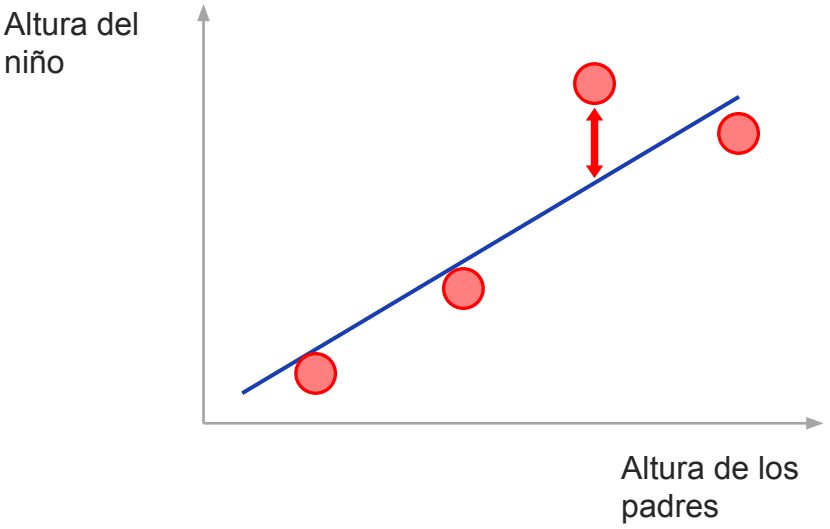
a) Mediante el modelado, descubra qué variables explicativas tienen el mayor impacto en la variable de respuesta.

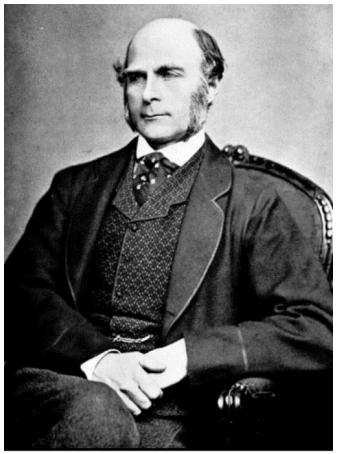
Ex Si el precio de la propiedad inmobiliaria es la variable respuesta Y, ¿cuáles son las variables explicativas estadísticamente más significativas? Zona, ubicación, antigüedad, distancia al centro de negocios, etc.Y

b) Predecir la respuesta dadas las condiciones de las variables explicativas.

Ex ¿Cuál es el precio de un departamento de 10 años de antigüedad, con un área de 100 m^2 y ubicado a 3 km del centro de negocios? ← “predecir” el valor que aún no es de conocimiento público.

- Antecedentes históricos
  - El término “regresión” fue acuñado por Francis Galton, biólogo del siglo XIX.th
  - Las alturas de los descendientes tienden a retroceder hacia la media.





Francisco Galton

- Ventajas
  - Sólida formación estadística y matemática.
  - Fuente de conocimientos
  - entrenamiento rapido

- Contras
  - Muchos supuestos: linealidad, normalidad, independencia de las variables explicativas, etc.
  - Sensible a valores atípicos
  - Propenso a la multicolinealidad

- Suposiciones
  - La variable respuesta puede explicarse mediante una combinación lineal de las variables explicativas.
  - No debería haber multicolinealidad.
  - Los residuos deben distribuirse normalmente centrados alrededor de 0.
  - Los residuos deben distribuirse con una variación constante.
  - Los residuos deben distribuirse aleatoriamente sin ningún patrón.

Análisis residual

- modelo lineal

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_k X_k + \varepsilon$$

variable de respuesta

Variables explicativas

- Los valores de las variables están dados por los datos.

- modelo lineal

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_k X_k + \varepsilon$$

Coeficientes de regresión

- Los coeficientes de regresión son parámetros del modelo: capturan los patrones de datos.

- modelo lineal

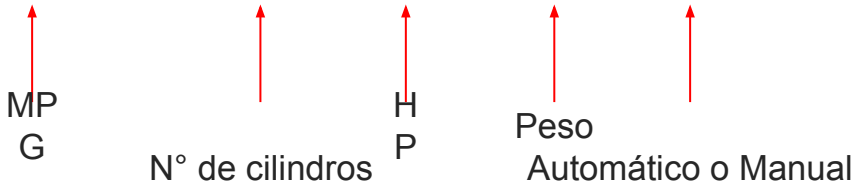
$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_k X_k + \varepsilon$$

- El término de error ε debe tener una media de cero y una varianza constante.

- modelo lineal

Ex

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_3 + \beta_4 X_4 + \varepsilon$$



Las MPG pueden explicarse por otras variables.

- Interpretación de los coeficientes de regresión.

$$\Delta Y = \beta_1 \Delta X_1 + \cdots + \beta_i \Delta X_i + \cdots + \beta_k \Delta X_k$$

- Si $X_1, X_1, \ldots, X_k$ cambia por $\Delta X_1, \Delta X_2, \ldots, \Delta X_k$, entonces el cambio en Y es ΔY.

- Interpretación de los coeficientes de regresión.

$$\Delta Y = \beta_1 \Delta X_1 + \cdots + \beta_i \Delta X_i + \cdots + \beta_k \Delta X_k$$

1

- β_i puede ser interpretado como el cambio en Y cuando X_i es incrementado por una unidad (ΔX_i=1).

- Interpretación de los coeficientes de regresión.

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_k X_k + \varepsilon$$

Intercept ar

0

- El interceptador β_0 es el valor de Y cuando todos los X_i = 0. Es como un “baseline.”

- Interpretación de los coeficientes de regresión.

Ex Encuesta salarial a los empleados de una empresa.

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \varepsilon$$

Salario — Experiencia — Calificación

- β_0 β0 puede interpretarse como el salario base cuando no se tiene experiencia ni calificación.
- β_1 β1 puede interpretarse como el cambio en el salario cuando la experiencia aumenta en una unidad.
- β_2 β2 puede interpretarse como el cambio en el salario cuando la calificación aumenta en una unidad.

- Solución de mínimos cuadrados ordinarios (OLS)

$$y_j = \beta_0 + \beta_1 x_{j,1} + \beta_2 x_{j,2} + \cdots + \beta_K x_{j,k} + \varepsilon_j$$



- Ahora podemos escribir la relación lineal en términos de los valores de datos reales.

- Solución de mínimos cuadrados ordinarios (OLS)

$$\boldsymbol{Y} = \boldsymbol{X}\,\boldsymbol{\beta} + \boldsymbol{\varepsilon}$$

  - Una notación compacta usando matrices.

- Solución de mínimos cuadrados ordinarios (OLS)

$$Y = X\beta + \varepsilon$$

  - Una notación compacta usando matrices.

- Solución de mínimos cuadrados ordinarios (OLS)

$$Y = X\beta + \varepsilon$$

  - Una notación compacta usando matrices.

- Solución de mínimos cuadrados ordinarios (OLS)

$$Y = X\beta + \varepsilon$$

  - Una notación compacta usando matrices.

- Solución de mínimos cuadrados ordinarios (OLS)

$$Y = X\beta + \varepsilon$$

  - Una notación compacta usando matrices.

- Solución de mínimos cuadrados ordinarios (OLS)
  - Como tenemos un sistema de ecuaciones lineales sobredeterminado, la solución exacta no existe.
  - Podemos minimizar $|\varepsilon|2$ y obtener la “mejor” solución $\beta.\varepsilon|^2\beta$.
  - La condición de minimización para $|\varepsilon|2$ viene dada por la derivada:$\varepsilon|^2$

$$\frac{d|\boldsymbol{\varepsilon}|^2}{d\beta} = 0$$

- Solución de mínimos cuadrados ordinarios (OLS)
  - La condición de minimización para |ε|2 viene dada por la derivada que se puede desarrollar de la siguiente manera:$\varepsilon|^2$

$$\frac{d|\boldsymbol{\varepsilon}|^2}{d\beta} = \frac{d|\boldsymbol{Y} - \boldsymbol{X\beta}|^2}{d\beta}$$

$$= \frac{d\{(\boldsymbol{Y} - \boldsymbol{X\beta})^T(\boldsymbol{Y} - \boldsymbol{X\beta})\}}{d\beta}$$

$$= \frac{d\{\boldsymbol{Y}^T\boldsymbol{Y} - \boldsymbol{\beta}^T\boldsymbol{X}^T\boldsymbol{Y} - \boldsymbol{Y}^T\boldsymbol{X\beta} + \boldsymbol{\beta}^T\boldsymbol{X}^T\boldsymbol{X\beta}\}}{d\beta}$$

$$= -2\boldsymbol{X}^T\boldsymbol{Y} + 2\boldsymbol{X}^T\boldsymbol{X\beta} = 0$$

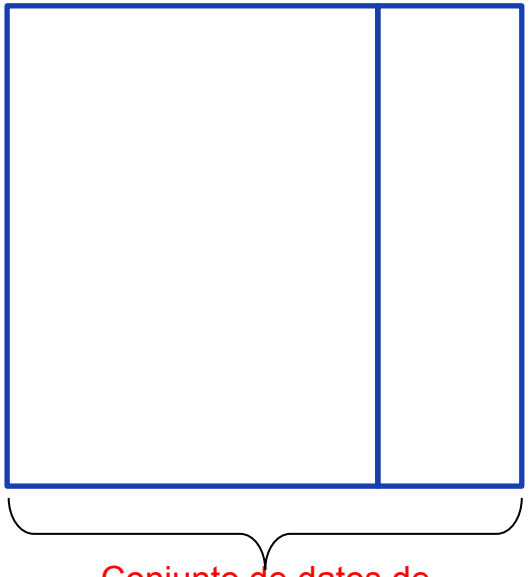
- Solución de mínimos cuadrados ordinarios (OLS)
  - La solución β de la diapositiva anterior viene dada por la siguiente expresión:β

$$\boldsymbol{\beta} = \underbrace{[(\boldsymbol{X}^T\boldsymbol{X})^{-1}\boldsymbol{X}^T]}_{\text{Pseudo-inverso}}\,\boldsymbol{Y}$$

  - La expresión matricial dentro del paréntesis cuadrado se llama "pseudoinversa".

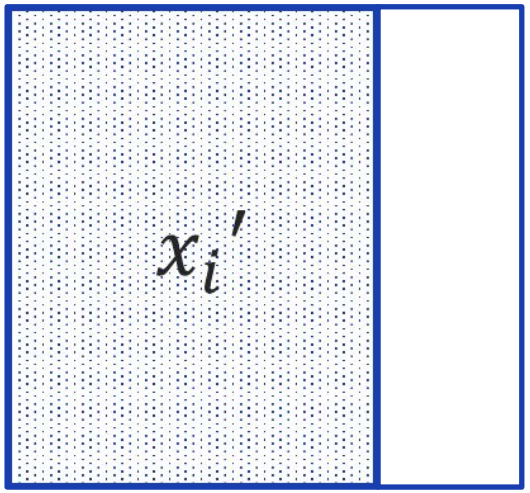
- Entrenamiento y predicción de regresión (pruebas)

1) Paso de entrenamiento: utiliza el conjunto de datos de entrenamiento y obtén un conjunto de parámetros del modelo $\{\beta i\}$.



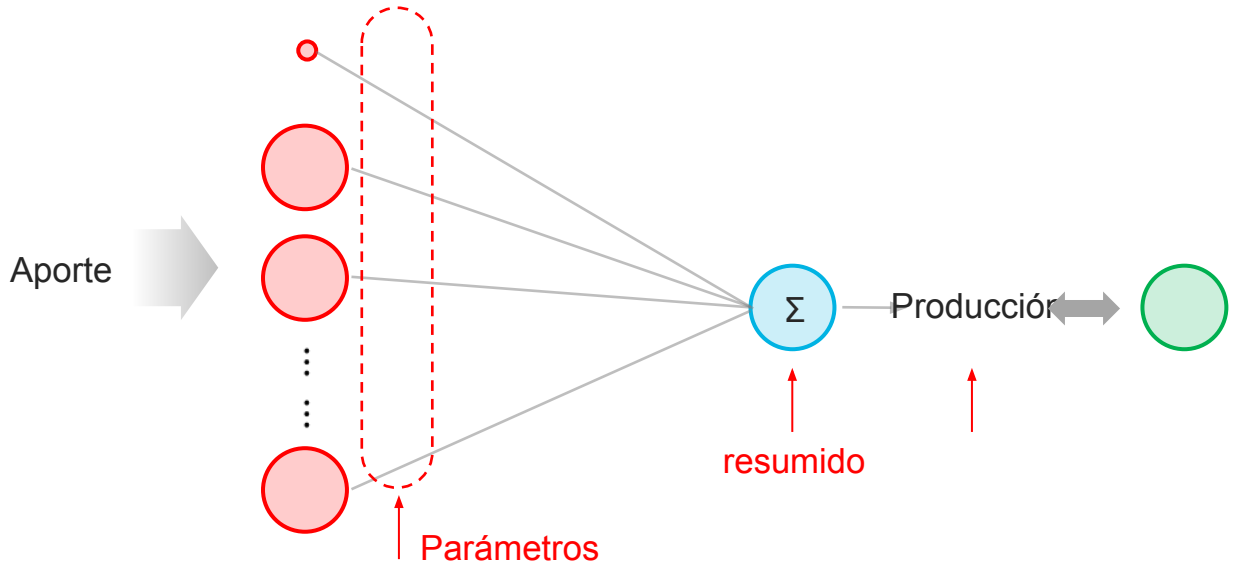
- Entrenamiento y predicción de regresión (pruebas)

2) Paso de predicción: cuando se proporciona un nuevo conjunto de {x_i'}, calcula el valor de y', que era desconocido.



- El valor predicho de y' se denota como $\hat{y}$, que es una expectativa condicional $\hat{y} = E[y|data]$.
- Dado los valores x_1', x_2', ..., x_k', calcula $\hat{y} = \beta_0 + \beta_1 x_1' + \beta_2 x_2' + \cdots + \beta_K x_k'$.

- Entrenamiento y predicción de regresión (pruebas)



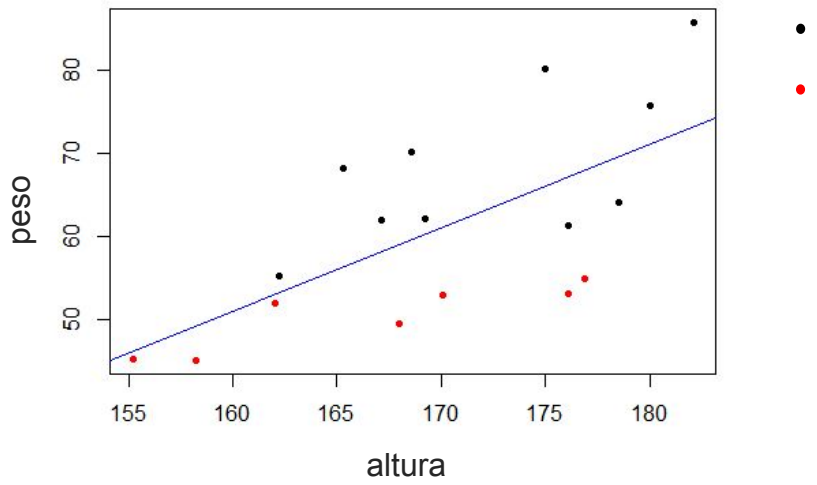
- Vista esquemática del paso de predicción.
- Se observa cierta discrepancia entre el valor predicho $\hat{y}$ y el valor real y.

- Métricas de error

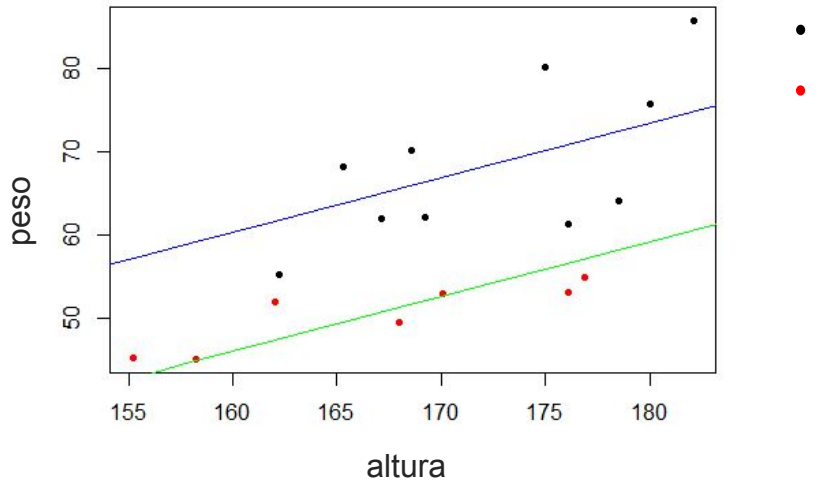
- Intervalo de confianza (95%) cuando solo hay una variable explicativa

  - $quantile(\alpha,kquantile(\alpha,k)$ es el cuantil t de Student del grado de libertad $k.k.$

- Papel de las variables categóricas
  - Sin variable categórica: peso ~ altura

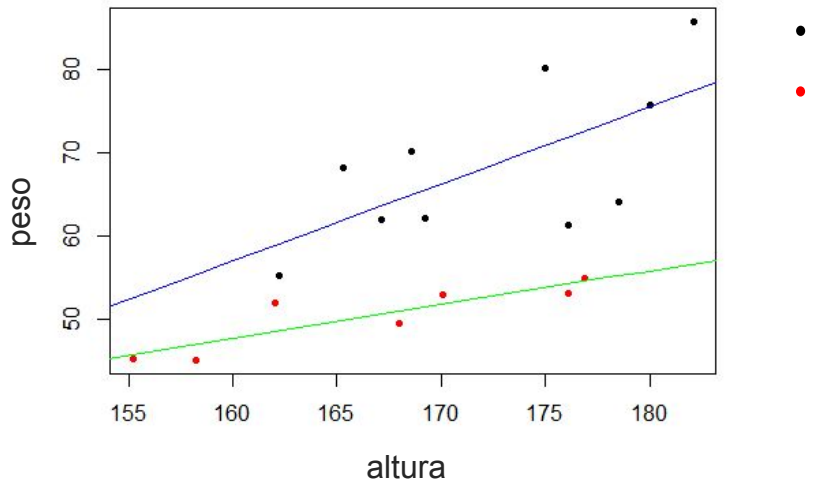


- Papel de las variables categóricas
  - Con una variable categórica género: peso ~ altura + sexo



  - La variable categórica debe convertirse primero en variable ficticia.
  - Sube o baja efectivamente la intercepción.

- Papel de las variables categóricas
  - Con una variable categórica género que “interactúa”: peso ~ altura + género + altura □ género ×



  - Tanto la intersección como la pendiente dependen de la variable categórica.
  - Mejora aún más las métricas de error.

Ejercicio de codificación n.° 0301



Siga los pasos de práctica en el archivo 'ex_0301.ipynb'.

Unidad 2.

Aplicación del modelo de aprendizaje supervisado para la predicción numérica

- 2.1. Capacitación y pruebas en Machine Learning
- 2.2. Conceptos básicos de la regresión lineal
- 2.3. Diagnóstico de regresión lineal
- 2.4. Otros tipos de regresión
- 2.5. Práctica del modelo de aprendizaje supervisado para la predicción numérica

Diagnóstico de regresión lineal

- Métodos de diagnóstico de regresión lineal:

  1) Métricas de error: MSE, RMSE, MAE, MAPE, etc.

  2) Coeficiente de determinación o "r-squared" $R2$

  3) F-test para la validez global del modelo lineal

  4) t-test para la validez de los coeficientes de regresión individuales

  5) Correlación entren Y e $\hat{Y}$

  6) Factor de inflación de la varianza o Variance inflation factor (VIF)

- Modelización: optimización de los criterios de información: AIC o BIC

- Análisis residual y de apalancamiento.

- Métricas de error

  - ¡Cuanto más pequeño, mejor!

- Coeficiente de determinación o $R2R2$

  - R^2R2 está acotado por encima de and por debajo: 0<$R2$<1.
  - R^2R2 cercano a uno significa que la variable respuesta está bien explicada.
  - A medida que se añaden más variables explicativas, $R2$ tiende a aumentar espuriamente: se introduce $R2$ ajustado.
  - If solo hay una variable explicativa X, entonces:

- Prueba F para la significancia general del modelo lineal
  - Null hypothesis $\boldsymbol{H0}$: $\beta 1=\beta 2=\cdots=\beta K=0$
  - Hipótesis alternativa $\boldsymbol{H1}$: Al menos βi es distinto de cero.

⇦ F-test statistic =

⇦ ⇦ Si el valor p está por debajo de una referencia (digamos 0,05), entonces $\boldsymbol{H0}$ se rechaza a favor de $\boldsymbol{H1}$.$\boldsymbol{H0}$ $\boldsymbol{H1}$.

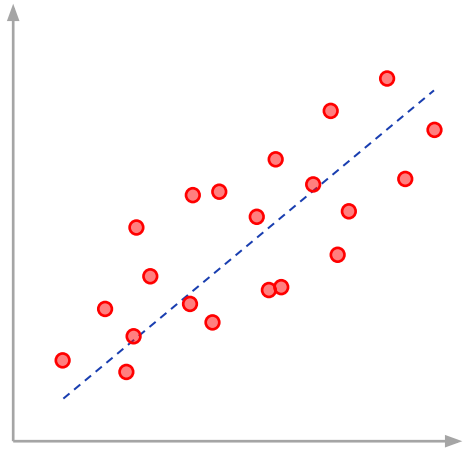
En este caso, el modelo lineal tiene una importancia global.

- Prueba t para determinar la significancia de los coeficientes de regresión individuales
  - Null hypothesis $\boldsymbol{H0}$: βi=0
  - Alternate hypothesis $\boldsymbol{H1}$: βi≠0

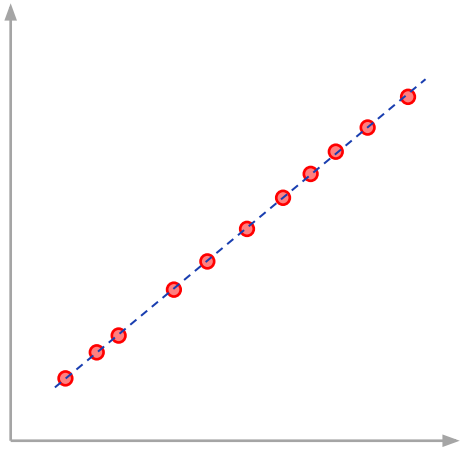
⇦ t-test statistic =

⇦ ⇦ Si el valor p está por debajo de una referencia (digamos 0,05), entonces $\boldsymbol{H0}$ se rechaza a favor de $\boldsymbol{H1}$.$\boldsymbol{H0}$ $\boldsymbol{H1}$.

En este caso, se justifica la inclusión de la variable explicativa Xi.



<



Correlación positiva débil

Fuerte correlación positiva

- Factor de inflación de varianza (VIF)
  - Puede medir la "seriedad" de la multicolinealidad.
  - In general, VIF>10 se considera grave. Sin embargo, este punto de referencia es bastante subjetivo.
  - En caso de que se detecte una multicolinealidad grave, es necesaria la simplificación del modelo.

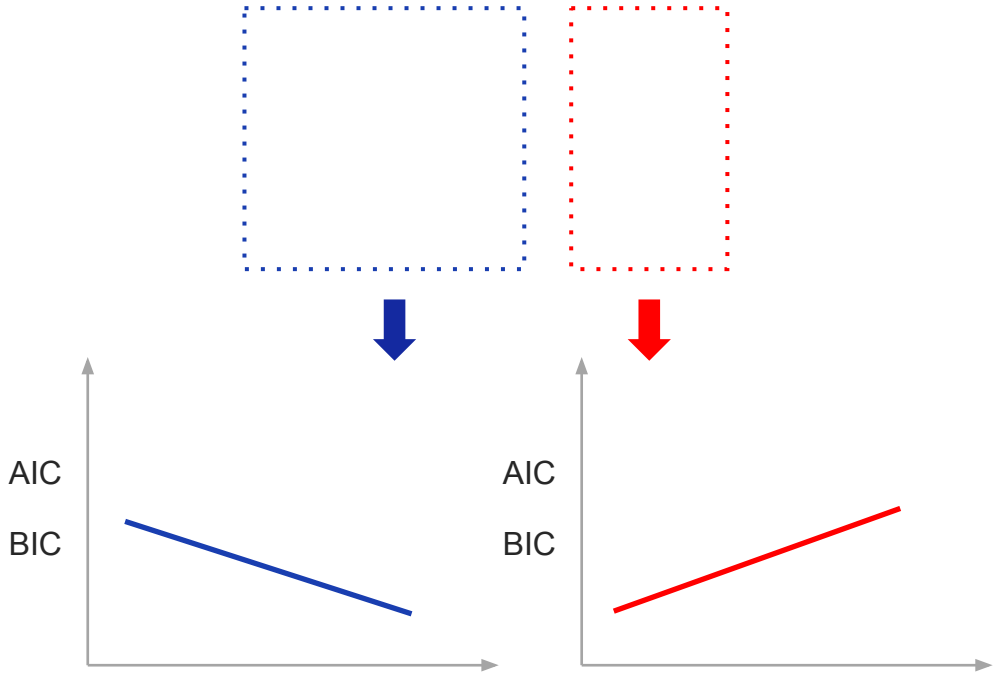
- Factor de inflación de varianza (VIF)
  - VIF es calculado individualmente por cada variable independiente X_i.

    1) Pon X_i como la respuesta y luego haz una regresión usando la expresión:

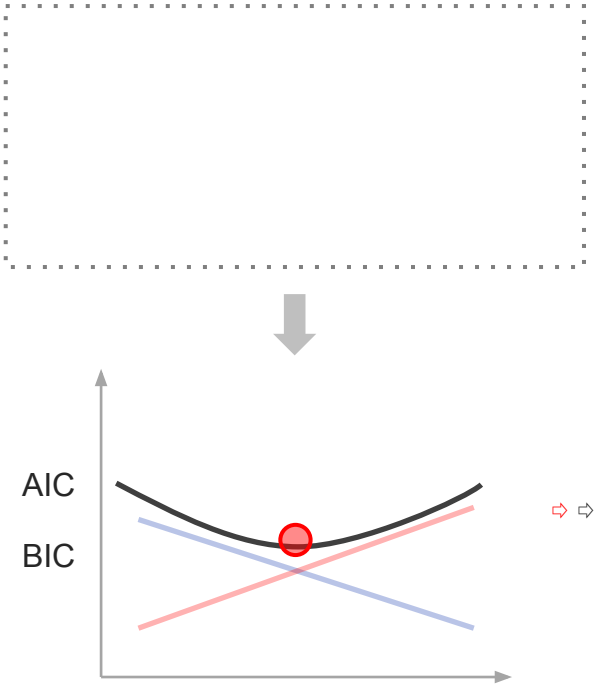
    2) Usando el correspondiente R_i^2 calcula $VIFi$:

- Criterios de información y modelización.
  - Akaike Information Criteria (AIC) con p=número de parámetros:
  - Bayes Information Criteria (BIC) con p=número de parámetros:
  - En las expresiones anteriores tenemos:

- Criterios de información y modelización.

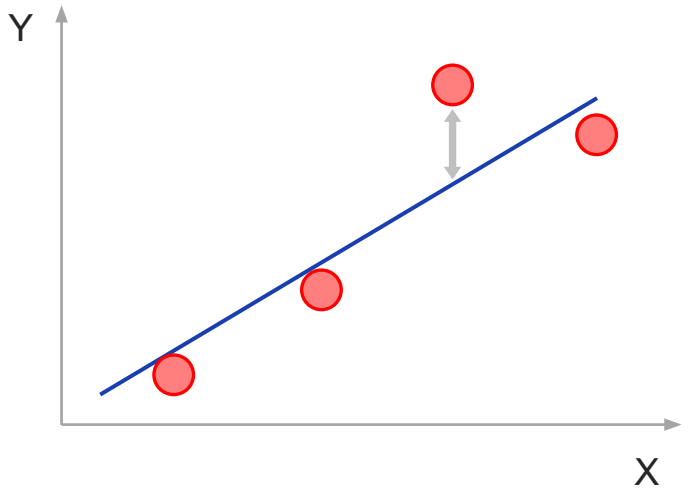


- Criterios de información y modelización.



⇨ ⇨ Hay un mínimo.

- Análisis residual



- El residuio es la diferencia entre el valor predicho $\hat{y}$ y el valor real y.
- Podemos detectar fácilmente valores atípicos en Y que se desvían sustancialmente de la tendencia principal.

- Análisis residual
  - Razones para el análisis residual:

    1) Detectar valores atípicos en Y.

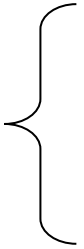
    2) Verificar los supuestos de regresión lineal.

    Los residuos deben distribuirse normalmente centrados alrededor de 0.

    Los residuos deben distribuirse con una variación constante.

    Los residuos deben distribuirse aleatoriamente sin ningún patrón.

- Análisis de apalancamiento
  - Aprovechamiento de la i-ésima observación
  - Existe una "regla de la suma".



  - El apalancamiento (leverage) indica qué tan alejado está X del centro => Detección de valores atípicos en X.

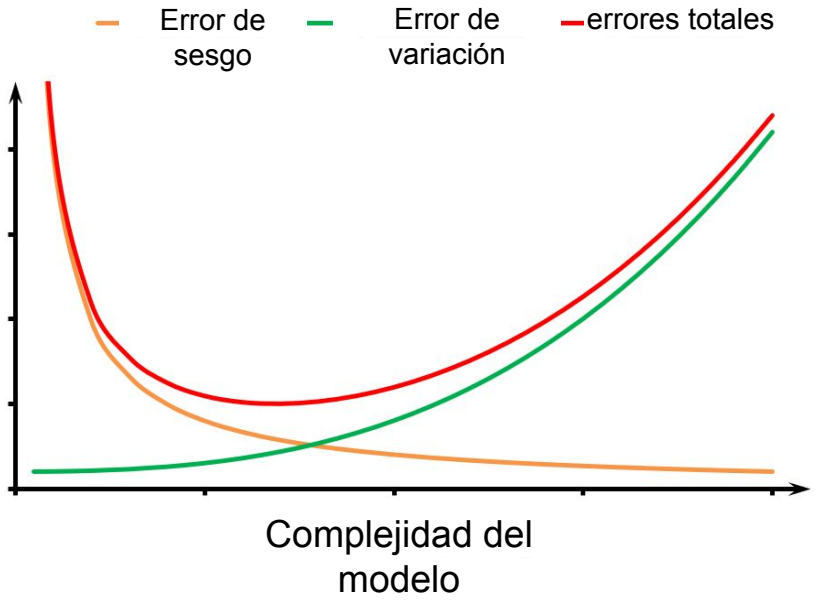
Unidad 2.

Aplicación del modelo de aprendizaje supervisado para la predicción numérica

- 2.1. Capacitación y pruebas en Machine Learning
- 2.2. Conceptos básicos de la regresión lineal
- 2.3. Diagnóstico de regresión lineal
- 2.4. Otros tipos de regresión
- 2.5. Práctica del modelo de aprendizaje supervisado para la predicción numérica

Regresión regularizada

- Compensación entre sesgo y varianza



- Relación de compensación entre el error de sesgo y el error de varianza..
- El obetivo debería ser minimizar el **Total error** = **Bias error** + **Variance error**.

- Regresión de cresta
  - Útil cuando la regresión lineal habitual se sobreajusta (error de sesgo << error de varianza).<<
  - Recordemos que la solución MCO consiste en minimizar $|\varepsilon|2.\varepsilon|^2$.
  - En la regresión Ridge, minimizamos la siguiente "función de pérdida":

- In the loss function, $\lambda \sum_{i=0}^{k} \beta_i^2$ is included as penalty: "L2 regularization."

- Regresión de cresta
  - Una λ positiva y mayor restringe más los coeficientes β_i, disminuyendo el error de varianza (sobreajuste).

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_k X_k + \varepsilon$$

  - Sin embargo, una λ demasiado grande puede hacer que el modelo sea demasiado "sesgado".
  - Incluso cuando es grande, los coeficientes β_i nunca llegan a ser exactamente iguales a cero. λ

- regresión de lazo
  - Útil cuando la regresión lineal habitual se sobreajusta (error de sesgo << error de varianza).<<
  - En la regresión de Lasso, minimizamos la siguiente "función de pérdida":

$$L = |\varepsilon|^2 + \lambda \sum_{i=0}^{k} |\beta_i|$$

  - En la función de pérdida, se incluye $\lambda \sum_{i=0}^{k} |\beta_i|$ como penalización: "Regularización L1 ."
  - Un λ positivo y mayor restringe aún más los coeficientes β_i disminuyendo el error de varianza (sobreajuste).
  - Sin embargo, un λ demasiado grande puede hacer que el modelo sea demasiado "sesgado"
  - Cuando λ es grande, los coeficientes β_i pueden llegar a ser exactamente iguales a cero.

Regresión polinomial

- Regresión polinómica
  - Útil cuando la regresión lineal habitual no es adecuada (error de sesgo >> error de varianza).
  - Podemos modelar la relación entre X y Y usando los polinomios:X Y

  - Observamos que solo hay una variable explicativa $X.X$.
  - Cuando la potencia polinómica es demasiado alta, puede producirse un sobreajuste.

Regresión de Poisson

- regresión de Poisson
  - Útil cuando queremos modelar la respuesta Y que representa recuentos o frecuencias.Y
  - Suponemos que Y sigue la distribución de Poisson.

Unidad 2.

Aplicación del modelo de aprendizaje supervisado para la predicción numérica

- 2.1. Capacitación y pruebas en Machine Learning
- 2.2. Conceptos básicos de la regresión lineal
- 2.3. Diagnóstico de regresión lineal
- 2.4. Otros tipos de regresión
- 2.5. Práctica del modelo de aprendizaje supervisado para la predicción numérica

Práctica del modelo de aprendizaje supervisado para la predicción numérica

- Importar módulos.

```
[ ]: import numpy as np
     import pandas as pd
     import matplotlib.pyplot as plt
     import seaborn as sns
     import os
     from sklearn.linear_model import LinearRegression
     %matplotlib inline
```

Ejercicio de codificación n.° 0302



Siga los pasos de práctica en el archivo 'ex_0302.ipynb'.

Ejercicio de codificación n.° 0303



Siga los pasos de práctica en el archivo 'ex_0303.ipynb'.

Ejercicio de codificación n.° 0304



Siga los pasos de práctica en el archivo 'ex_0304.ipynb'.

Ejercicio de codificación n.º 0305



Siga los pasos de práctica en el archivo 'ex_0305.ipynb'.

Ejercicio de codificación n.º 0306



Siga los pasos de práctica en el archivo 'ex_0306.ipynb'.

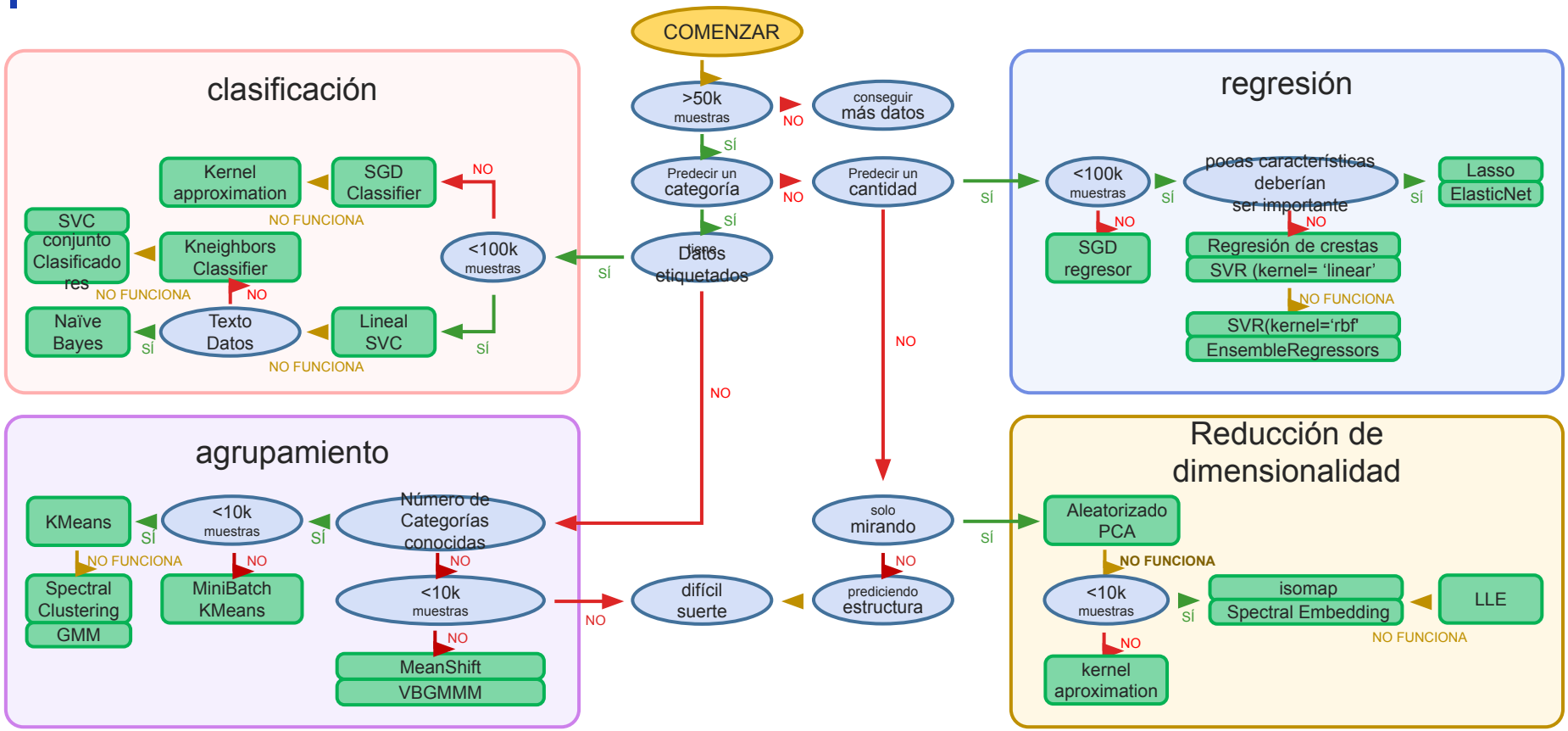
Unidad 3.

Aplicación del modelo de aprendizaje supervisado para la clasificación

- 3.1. Capacitación y pruebas en Machine Learning
- 3.2. Conceptos básicos de la regresión logística
- 3.3. Métricas de rendimiento de regresión logística

Capacitación y pruebas en Machine Learning

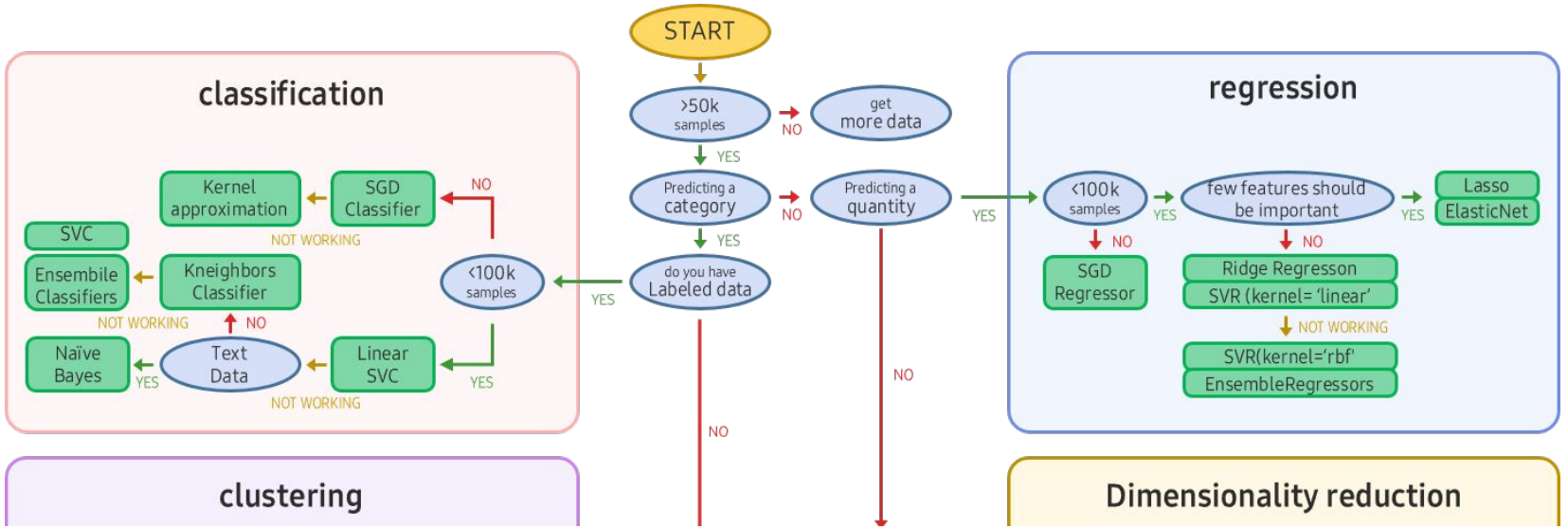
Seleccione un algoritmo Machine Learning adecuado para los datos.



Seleccione un algoritmo Machine Learning adecuado para los datos.

- El aprendizaje supervisado son tareas Machine Learning que intentan predecir o clasificar los valores de variables objetivas (o variables de respuesta) en datos invisibles, ya que el conjunto de datos de entrenamiento tiene las variables objetivas (o variables de respuesta) que se van a predecir (Y). (or (or
- Se llama aprendizaje supervisado porque se le dice al algoritmo que prediga un objetivo determinado.
- El aprendizaje supervisado se clasifica principalmente en problemas de clasificación y regresión según los tipos de variables objetivas (o variables de respuesta). (or
- En otras palabras, la clasificación se aplica si las variables objetivas (o variables de respuesta) son discretas o nominales, tales como ‘hombre/mujer’, ‘correo basura/correo deseado’, ‘positivo/neutral/negativo’, ‘Seúl/Busán/Gyeonggi/Gangwon’, etc. Por otro lado, la regresión se aplica si las variables objetivas (o variables de respuesta) son numéricas, tales como 0~10, -500~500, -3.5~3.5, -∞~∞, etc.
- Cuando los datos proporcionados no tienen marcas de clasificación o variables objetivas (o variables de respuesta), y si no se intenta predecir el valor objetivo, se aplica el aprendizaje no supervisado. (or
- El aprendizaje no supervisado se clasifica en agrupamiento, asociación, reducción de dimensiones y otros según el propósito y los métodos del análisis. Se explicarán más detalles en otros capítulos.

Seleccione un algoritmo Machine Learning adecuado para los datos.



- Como se muestra en la figura anterior, las redes neuronales se pueden aplicar si hay respuestas correctas y es con fines de clasificación o tiene muchos datos. De lo contrario, es posible utilizar un árbol de decisión o un algoritmo SVC.
- Además, se puede utilizar el algoritmo Naïve Bayes si los datos están en texto.
- El método KNN se utiliza si no se trata de datos de texto. También es posible utilizar el método de conjunto con un mejor rendimiento.

Machine Learning para clasificación

- La clasificación se utiliza si las variables objetivas (o variables de respuesta) se pueden clasificar en ciertas categorías, como tipo discreto o nominal. Es el más común y el que se encuentra con más frecuencia en el análisis de datos basado en Machine Learning.
- Los algoritmos Machine Learning para clasificación se pueden aplicar a amplios problemas diarios y comerciales.

  Ejemplos de uso frecuente

  Ex

  (1) Clasificación de correos no deseados

  (2) Predicción de quiebra corporativa

  (3) Predicción de pérdida de clientes

  (4) Clasificación de la calificación crediticia de los clientes

  (5) Predicción de la aparición de una determinada enfermedad (por ejemplo, cáncer, enfermedad cardíaca, etc.)

  (6) Predicción de la reacción del cliente ante un evento de marketing específico.

  (7) Predicción de la compra del cliente

- Los ejemplos proporcionados anteriormente son muy pocas aplicaciones de la vida real; Hay infinitos campos de negocios en los que se aplica el tipo de clasificación Machine Learning. Este tipo de Machine Learning se ha aplicado continuamente con nuevos métodos comerciales y de I+D.

Tipos de algoritmo Machine Learning para clasificación (1/2)

- Existen muchos algoritmos Machine Learning de tipo clasificación diferentes. Algunos de ellos pueden usarse para problemas de regresión. La siguiente tabla proporciona descripciones de los métodos de clasificación utilizados con frecuencia.

Tipo	Concepto	Nota
K-vecino más cercano	Un método de clasificación que utiliza la regla de la mayoría en los k valores de variables objetivas (o variables de respuesta) más cercanos en función de la distancia entre la coordinación de datos distintos de ciertos datos.	Aprendizaje perezoso
Bayes ingenuo	Un método basado en el teorema de Bayes que realiza la clasificación hacia la mayor probabilidad expresando la probabilidad condicional de variables objetivas (o variables de respuesta) como una multiplicación de la probabilidad previa y la función de verosimilitud. Se supone que todos los valores observados ocurren estadísticamente independientemente de otros valores observados. (Conocido como modelo ingenuo ya que el supuesto se da sin confianza).	Modelo de demostrabilidad (probabilidad condicional basada en el teorema de Bayes)
Regresión logística	Un método para estimar la probabilidad de variables objetivas a través de la estimación de máxima verosimilitud asumiendo que la probabilidad de que el valor de la variable objetivo esté en una determinada categoría tiene la forma de una función logística cuando se proporciona el valor explicativo.	Modelo de demostrabilidad (estimación de máxima verosimilitud)
Árbol de decisión	Un método para crear reglas de clasificación dividiendo las ramas hacia una menor impureza o entropía en el orden de las variables más asociadas con las variables objetivas.	Divide y vencerás

Tipos de algoritmo Machine Learning para clasificación (2/2)

- Existen muchos algoritmos Machine Learning de tipo clasificación diferentes. Algunos de ellos pueden usarse para problemas de regresión. La siguiente tabla proporciona descripciones de los métodos de clasificación utilizados con frecuencia.

Tipo	Concepto	Nota
Red neuronal artificial	Un método inspirado en la red neuronal humana. Este método de análisis, compuesto por el nodo de entrada, el nodo oculto y el nodo de salida, se utiliza para resolver problemas complicados de clasificación o predicción de valores de caja negra.	Prueba de caja negra
Máquina de vectores de soporte	Un método para clasificar ciertos datos encontrando un plano que maximice el margen entre datos en diferentes categorías.	Lineal y no lineal (truco del kernel)
Bosque aleatorio	Un método de conjunto para decidir el resultado final de la clasificación creando varios árboles de decisión basados en datos dados y agregando los resultados previstos de cada árbol de decisión mediante votación.	modelo de conjunto

Unidad 3.

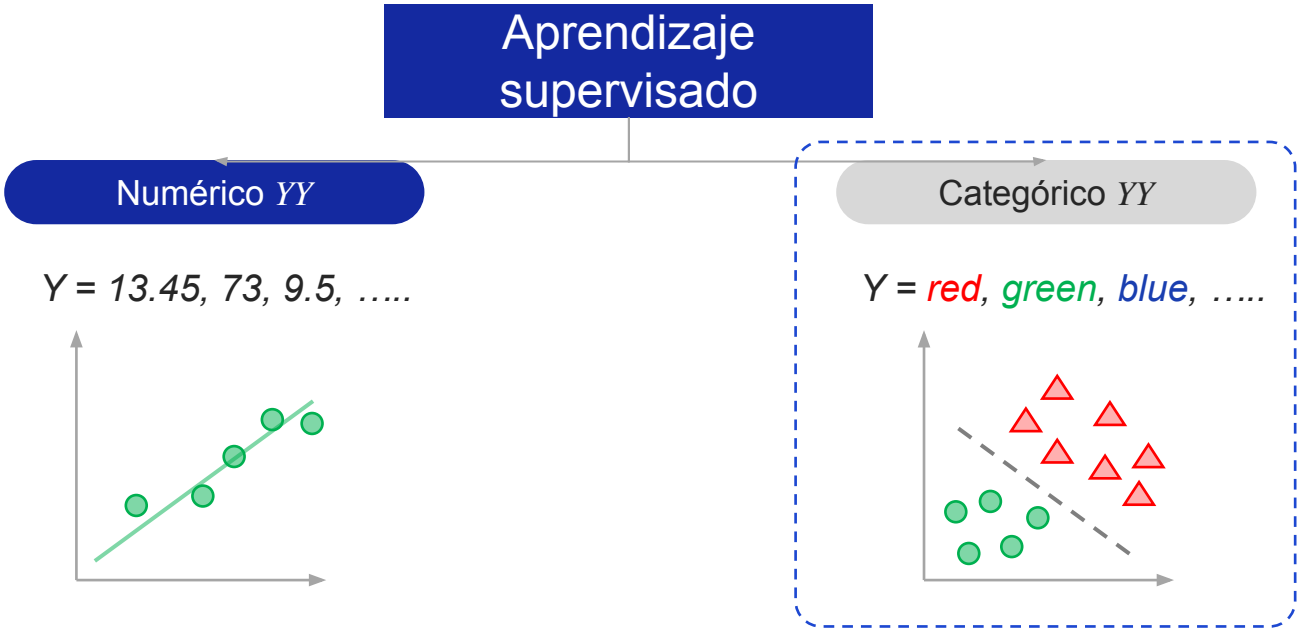
Aplicación del modelo de aprendizaje supervisado para la clasificación

- 3.1. Capacitación y pruebas en Machine Learning
- 3.2. Conceptos básicos de la regresión logística
- 3.3. Métricas de rendimiento de regresión logística

Conceptos básicos de la regresión logística

- ¿Qué es el análisis de regresión logística?

- En contraste con el análisis de regresión lineal general utilizado para la predicción numérica, el análisis de regresión logística se utiliza para clasificar la categoría de las variables objetivas (y) que se desean predecir.En otras palabras, lo que se predice no es el valor de y en sí como variable objetiva. Es P(Y=i), que es la probabilidad de que la variable objetiva y pertenezca a una categoría determinada (i).



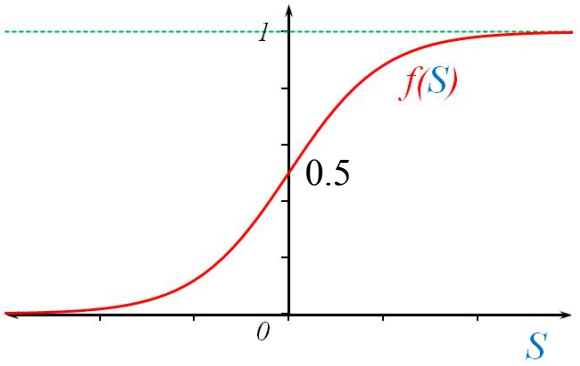
- Acerca de la regresión lineal
  - Existen una o más variables explicativas: $X_1, X_2, \ldots, X_k$
  - Existe una variable de respuesta: Y
  - La variable de respuesta Y es binaria, donde los valores posibles son {0,1}, $\{False, True\}$, $\{No, Yes\}$, etc.
  - Uno de los algoritmos de clasificación más básicos

- Ventajas
  - Simple y relativamente fácil de implementar.
  - Fuente de conocimientos intuitivos
  - entrenamiento rapido

- Contras
  - No se encuentra entre los algoritmos de clasificación más precisos.
  - Asume que las variables explicativas son independientes sin multicolinealidad.

- Acerca de la regresión logística
  - Las combinaciones lineales de variables X_i se denominan “Logit” y se denotan aquí como $S.X_i$ S.

  - La probabilidad condicional de que Y sea igual a 1 se denota como $P(Y = 1|\{x_i\})$.
  - La función "sigmoide" o "logística" conecta la probabilidad con el logit mediante la siguiente relación:



- Acerca de la regresión logística
  - Si p = $probability$ de Y=1, podemos definir:

  - La función logística es la inversa del logit (y viceversa):

$$S = Log\left(\frac{p}{1-p}\right) \quad \Leftrightarrow \quad p = \frac{e^{S}}{1+e^{S}}$$

- Acerca de la regresión logística
  - Para la regresión logística, las categorías de la variable objetiva (Y) a predecir son 0 y 1 (modelo de regresión logística binomial). La posibilidad de que la categoría de la variable objetiva Y sea 1 se expresa como p(Y=1) = P(Y). Comience expresando la descripción como una ecuación de regresión. El lado izquierdo de la ecuación se conoce comúnmente como "odds" (probabilidades relativas).
  - Las "probabilidades" significan la tasa de dividendos, que se clasifica en probabilidades de éxito/fracaso. Si la probabilidad de éxito es alta, pasa a ser 1; si es bajo, pasa a ser 0. En otras palabras, es una tasa de éxito. Agregar un registro a las probabilidades se convierte en logit y esta función es logística.
  - El lado izquierdo de la ecuación es una razón de probabilidad; el lado derecho es una función exponencial que tiene un rango (0, ∞). Agregue un logaritmo en ambos lados de la ecuación para obtener un rango con valores (−∞, ∞) en ambos.
  - Al observar la ecuación en detalle, el $\beta_0 + \beta_1 X$ del lado derecho es un modelo lineal con un rango (−∞, ∞); el lado izquierdo también tiene un rango (−∞, ∞). El log(P(Y))/(1-P(Y)) en el lado izquierdo de la ecuación se llama función logit.
  - O bien, agregue "exp" a ambos lados de la ecuación y organícela con respecto a P(Y) para obtener la siguiente ecuación.

•Acerca de la regresión logística

- En resumen, el análisis de regresión logística es un algoritmo que establece un modelo utilizando la función logística de la ecuación anterior y prediciendo los parámetros β_0, β_1 a partir de la probabilidad P(Y=1) = P(Y), donde la variable objetiva Y tiene el valor categórico 1 en los datos de entrenamiento.
- Generalmente, se utiliza la estimación de máxima verosimilitud (Maximum Likelihood Estimation) para predecir los parámetros β_0 y β_1 Este es un método analítico que transforma la fórmula. Dado que el cálculo directo es difícil, la estimación se realiza asignando un valor inicial determinado y ejecutando cálculos repetidos para ajustar el valor mediante cálculo numérico.

- Acerca de la regresión logística
  - La siguiente figura muestra una gráfica de la función logística. Como se observa en el gráfico, el rango del eje X va de $-\infty$ a ∞ . El eje Y representa la probabilidad de ocurrencia de la variable objetiva Y, que oscila entre 0 y 1. Por lo tanto, la función P(Y=1) muestra una curva en forma de S de 0 a 1 a medida que aumenta el valor de x.

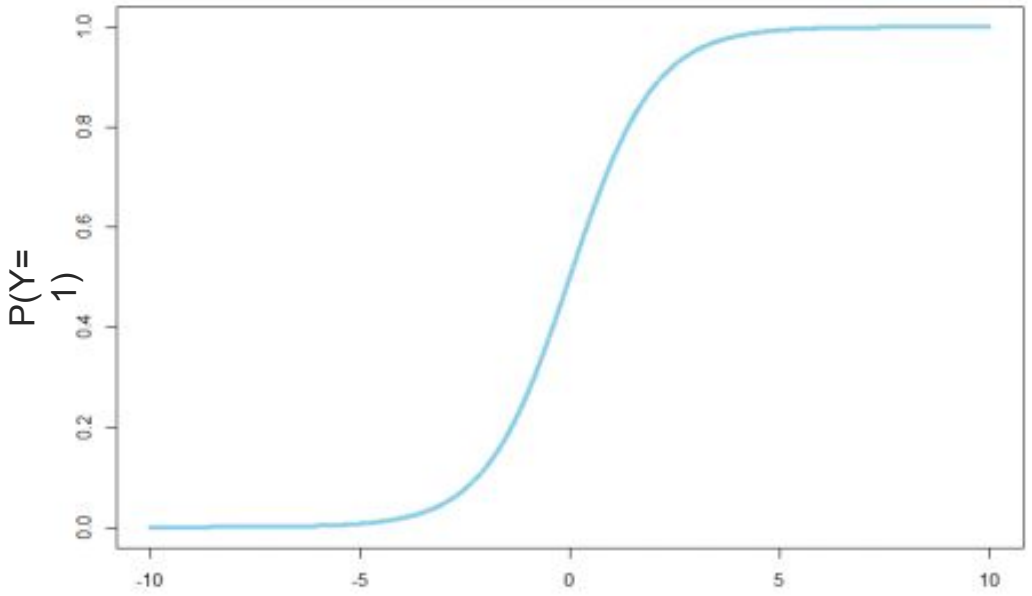
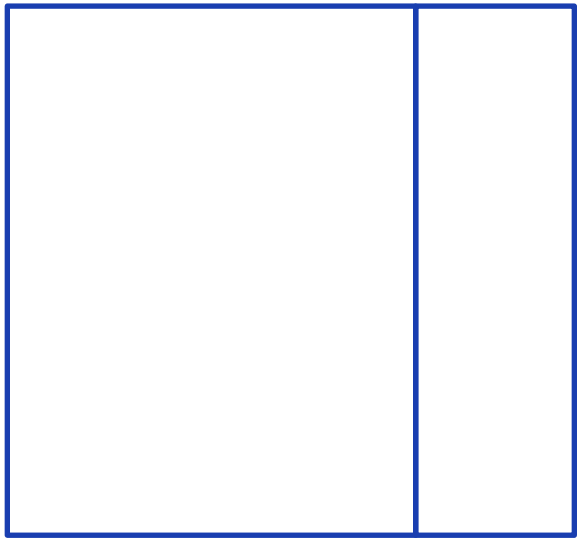


Gráfico de regresión logística

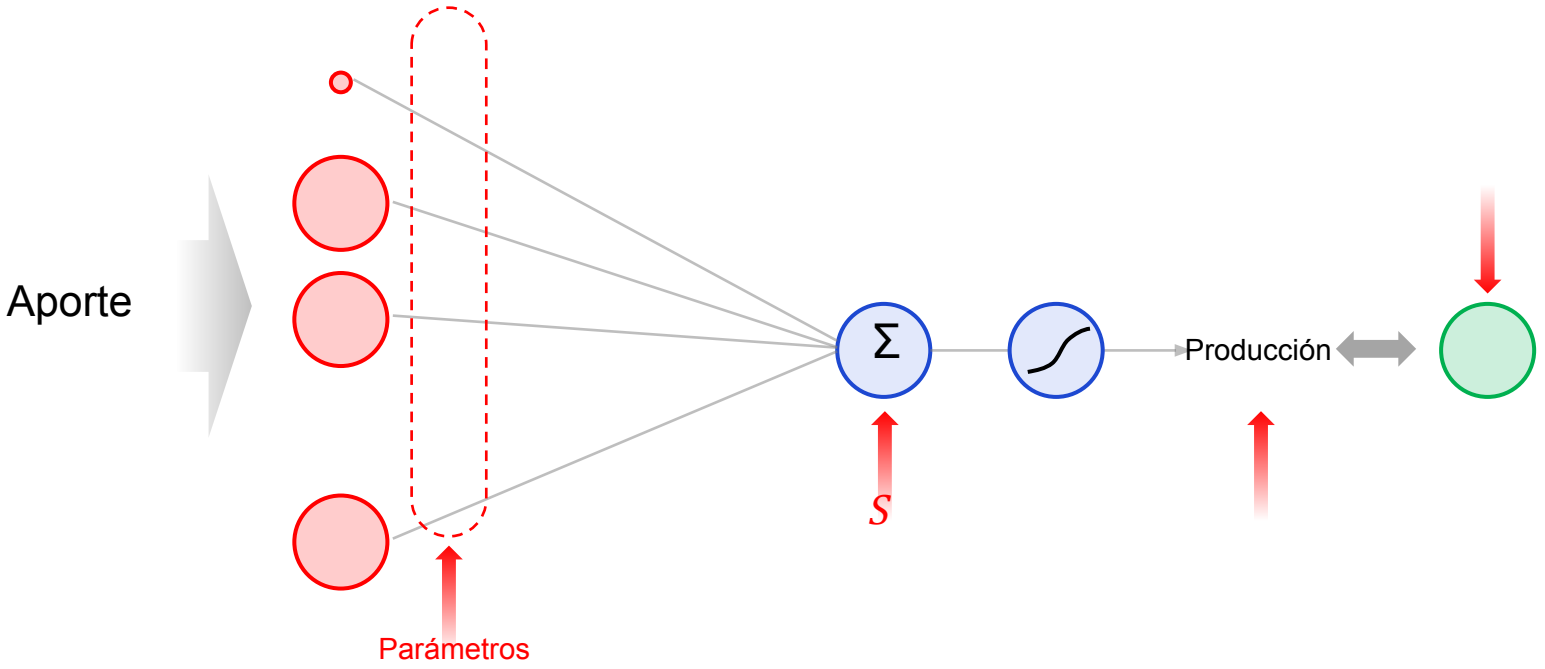
- Entrenamiento y predicción de regresión logística (pruebas)

2) Paso de predicción: cuando se proporciona un nuevo conjunto de $\{x_i'\}$, se calcula el valor de y', que era desconocido.



Compare la probabilidad condicional con un límite.

- Entrenamiento y predicción de regresión logística (pruebas)



- Entrenamiento por algoritmo de descenso de gradiente.
  - Podemos obtener el conjunto de parámetros $\{\beta_i\}$ minimizando una función objetivo $L(\boldsymbol{\beta})$.
  - Minimizamos la diferencia entre el valor real Y y el valor predicho $\hat{Y}$.
  - Podemos considerar a $L(\boldsymbol{\beta})$ como una función de "pérdida" (loss) o de "costo" (cost).
  - $L(\boldsymbol{\beta})$ es una función del conjunto de parámetros $\{\beta_i\}$, el cual puede ser representado por un vector $\boldsymbol{\beta}$.

- Entrenamiento por algoritmo de descenso de gradiente.
  - $L(\boldsymbol{\beta})$ es el negativo de la verosimilitud logarítmica (Negative Log-Likelihood)

  - En la relación anterior, $\boldsymbol{x_i}$ e y_i epresentan los valores proporcionados por el conjunto de datos de entrenamiento
  - Aquí, hemos asumido la conversión de $y_i = \{0,1\}$ to $y_i = \{-1, +1\}$.

- Entrenamiento por algoritmo de descenso de gradiente.
  - $L(\boldsymbol{\beta})$ es el negativo de la verosimilitud logarítmica (negative log-likelihood)

- Entrenamiento por algoritmo de descenso de gradiente.
  - El gradiente de $L(\boldsymbol{\beta})$ se denota como $\nabla L(\boldsymbol{\beta})$, que también es una función del conjunto de parámetros $\{\beta_i\}$ o $\boldsymbol{\beta}$.
  - La expresión para el gradiente $\nabla L(\boldsymbol{\beta})$ is:
  - El crecimiento de $L(\boldsymbol{\beta})$ es más pronunciado a lo largo de la dirección de $\nabla L(\boldsymbol{\beta})$.

    $\Leftrightarrow$ El descenso de $L(\boldsymbol{\beta})$ es más pronunciado a lo largo de la dirección de $-\nabla L(\boldsymbol{\beta})$.

- Entrenamiento por algoritmo de descenso de gradiente.
  - La expresión por la gradiente $\nabla L(\boldsymbol{\beta})$ es:
  - La gradiente $\nabla L(\boldsymbol{\beta})$ es obtenida calculando la derivada parcial de $L(\boldsymbol{\beta})$:

- Entrenamiento por algoritmo de descenso de gradiente.
  - $L(\boldsymbol{\beta})$ se minimiza iterativamente en pequeños “pasos” empujando $\boldsymbol{\beta}$ en la dirección $-\nabla L(\boldsymbol{\beta})$.

    1) $\boldsymbol{\beta}$ es inicializado aleatoriamente.

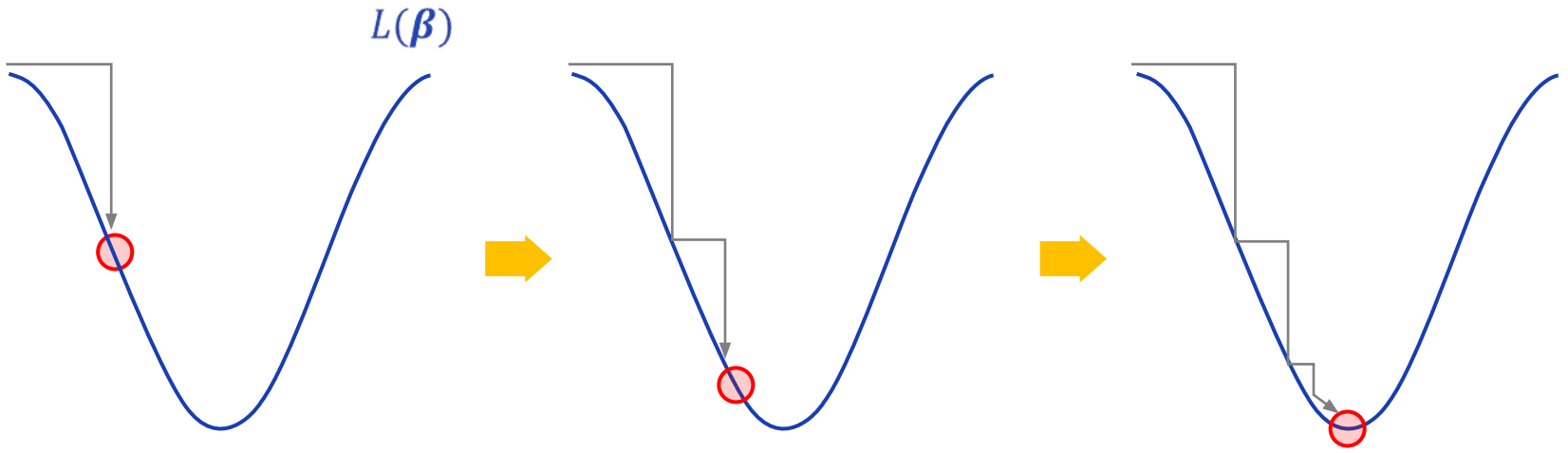
    2) Calcula la gradiente $\nabla L(\boldsymbol{\beta})$.

    3) Actualiza $\boldsymbol{\beta}$ en un paso: $\boldsymbol{\beta} \leftarrow \boldsymbol{\beta} - \eta \nabla L(\boldsymbol{\beta})$.

    La velocidad de convergencia está controlada por la "Tasa de aprendizaje" (learning rate) η.

    4) Repite desde el paso 2) un número fijo de veces (épocas o epochs).

- Entrenamiento por algoritmo de descenso de gradiente.
  - $L(\beta)$ se minimiza iterativamente en pequeños “pasos” empujando β en la dirección $-\nabla L(\beta)$.



- Entrenamiento por algoritmo de descenso de gradiente.
  - La función de gradiente en Python

[1]:

```
def sigmoid(x):
    s=1.0/(1.0 + np.exp(-x))
    return s
def gradient(X, Y, beta):
    z = np.dot(X.beta.T) * Y
    ds = -Y * (1 - sigmoid(z)) * X
    return ds.sum(axis=0)
```

- Entrenamiento por algoritmo de descenso de gradiente.
  - La función de gradiente en Python

[2]:

```
def train(self, input_X, input_Y, n_epochs):
    ones_column = np.ones((input_X.shape[0], 1))
    X = np.concatenate((ones_column, input_X), axis=1)
    Y = (2 * input_Y - 1).reshape(-1, 1)
    for n in range(n_epochs):
        self.beta = self.beta - self.rate * gradient(X, Y, self.beta)
    return self.beta
```

Unidad 3.

Aplicación del modelo de aprendizaje supervisado para la clasificación

- 3.1. Capacitación y pruebas en Machine Learning
- 3.2. Conceptos básicos de la regresión logística
- 3.3. Métricas de rendimiento de regresión logística

Métricas de rendimiento de regresión logística

- Matriz de confusión
  - En el método de clasificación Machine Learning, el método más común para evaluar el resultado del modelo de análisis es el cálculo métrico, incluida la precisión de la clasificación, utilizando una matriz de confusión.
  - La matriz de confusión se refiere a la matriz que crea una tabla cruzada de la categoría de clasificación prevista a partir del modelo de análisis y la categoría de datos clasificada real.

•Matriz de confusión

Valor categórico previsto

Actual valor categórico	Y	N
Y	O (TP: Verdadero Positivo)	X (FN: falso negativo)
N	X (FP: falso positivo)	O (TN: Verdadero Negativo)

- Matriz de confusión
  - La matriz de confusión se puede hacer con una mesa transversal de 2X2, junto con mesas transversales de 3X3 y superiores. Por conveniencia, en este capítulo sólo usaremos una matriz de confusión de 2X2.
  - En la matriz de confusión de la diapositiva anterior, los casos "O" colocados en diagonal significan que los valores categóricos previstos y reales son los mismos. En otras palabras, la clasificación Machine Learning predijo correctamente los resultados.
  - Por otro lado, si los valores categóricos previstos y reales difieren, el modelo Machine Learning ha realizado predicciones incorrectas.
  - Las categorías que más interesan al análisis son las categorías positivas; las demás se llaman categorías negativas. Dependiendo de la precisión de la predicción (verdadera o falsa) con respecto a las categorías positivas y negativas, la clasificación precisa de las categorías interesadas se denomina TP (Verdadero Positivo). La clasificación precisa de categorías desinteresadas se llama TN (Verdadero Negativo). .
  - La clasificación inexacta de categorías desinteresadas en categorías interesadas se denomina FP (falso positivo). La clasificación inexacta de categorías interesadas en categorías no interesadas se denomina FN (falso negativo)..
  - Existen varias métricas basadas en diferentes combinaciones de TP, TN, FP y FN de la matriz de confusión para evaluar el resultado del análisis de los métodos de clasificación Machine Learning.

- Métrico
  - Las métricas principales calculadas a partir de la matriz de confusión incluyen la exactitud (accuracy), la tasa de error (error rate = 1 - exactitud), la sensibilidad (sensitivity, también conocida como recall, hit ratio o tasa de verdaderos positivos), la especificidad (specificity), la tasa de falsos positivos (FP rate) y la precisión (precision), entre otras. De estas, la exactitud, la sensibilidad y la precisión son las métricas utilizadas con mayor frecuencia.
  - Además, existen medidas F (o puntuaciones F1) que combinan sensibilidad, precisión y estadísticas Kappa, donde los valores previstos y reales del modelo de análisis son exactamente iguales. Las fórmulas de cálculo y las definiciones de varias métricas se encuentran en la siguiente tabla.

•Métrico

Métrico	Calculation Formula	Definición
accuracy	(TP+TN) / (TP+TN+FP+FN)	Ratio of accurate prediction of actual classification category (Ratio of TP and TN from the entire prediction)
error rate	(FP+FN) / (TP+TN+FP+FN)	Ratio of inaccurate prediction of actual classification category (Identical to 1-accuracy)
sensitivity = TP Rate	(TP) / (TP+FN)	Ratio of accurate prediction to 'positive' from actual 'positive' categories (True Positive – also referred to as Recall, Hit Ratio, and TP Rate)
specificity	(TN) / (TN+FP)	Ratio of accurate prediction to 'negative' from actual 'negative' categories (True Negative)
FP Rate	(FP) / (TN+FP)	Ratio of inaccurate prediction to 'positive' from actual 'negative' categories = 1-specificity
precision	(TP) / (TP+FP)	Ratio of actual 'True Positive' from the ratio of predicted 'positive'
F-Measure (F1-Score)		Ranged between 0~1. It is the harmonic mean between precision and sensitivity (recall). If both precision and sensitivity are high, f-Measure also tends to have a larger value.
Kappa Statistic		The value after eliminating coincidental agreement between predicted and actual values of the model (Ranged between 0~1. When the value is closer to 1, the predicted and actual values of the model accurately coincide. The values do not coincide when the value gets closer to 0.)

- Métrico
  - Entre las métricas de la diapositiva anterior, la sensibilidad significa qué tan bien se predice que la categoría "positiva" real es "positiva". La precisión es el índice que muestra la proporción entre las categorías "positivas" reales y las categorías "positivas" pronosticadas. Por tanto, son métricas que explican directamente qué tan bien el modelo de análisis de clasificación Machine Learning clasifica valores categóricos interesados de variables objetivas.
  - La sensibilidad y la precisión son las métricas más importantes y utilizadas con frecuencia para clasificar los resultados Machine Learning en la vida real.

- Matriz de confusión

Ex

	Real 0	real 1
Predicho 0	120	5
Predicho 1	15	20

- La matriz de confusión es una tabla de contingencia que cuenta las frecuencias de lo real frente a lo previsto.

- Matriz de confusión: precisión

Ex

	Real 0	real 1
Predicho 0	120	5
Predicho 1	15	20

- La precisión es la relación entre la suma diagonal y la suma total.

- Matriz de confusión: Sensibilidad

Ex

	Real 0	real 1
Predicho 0	120	5
Predicho 1	15	20

- Matriz de confusión: especificidad

Ex

	Real 0	real 1
Predicho 0	120	5
Predicho 1	15	20

- Matriz de confusión: precisión

Ex

	Real 0	real 1
Predicho 0	120	5
Predicho 1	15	20

- Nota
  - La precisión por sí sola no es suficiente para realizar pruebas.

    Ex Si los fraudes constituyen sólo el 1% de todas las transacciones, la precisión de un sistema de detección de fraudes (FDS) que prediga la ausencia de fraude en todas las transacciones sería bastante alta, del 99%.

    Sin embargo, tal FDS sería inútil porque pasa por alto el 1% que realmente importa.
  - También deberíamos considerar otras métricas además de la precisión.

- Terminología

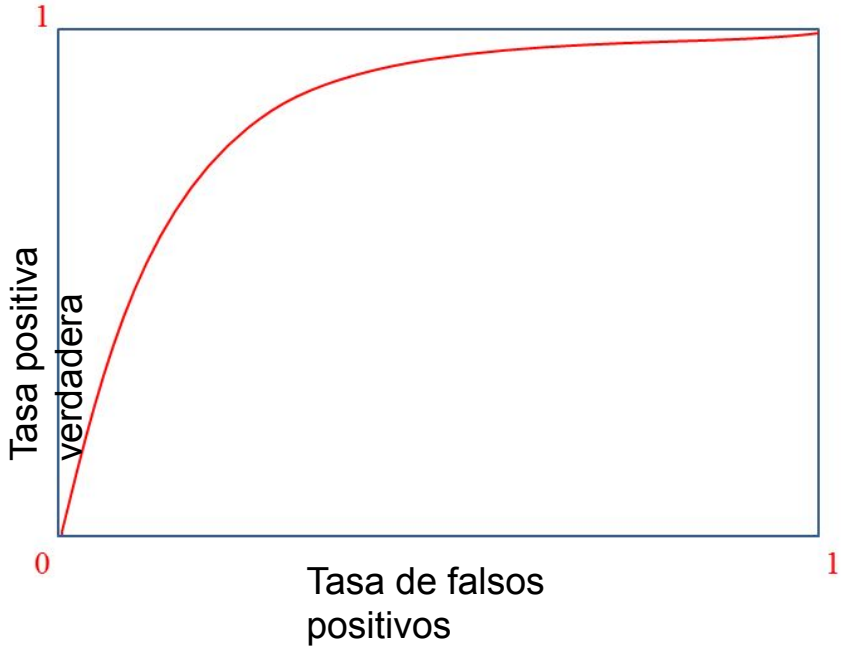
- Terminología

True Positive Rate = Sensitivity

True Negative Rate = Specificity

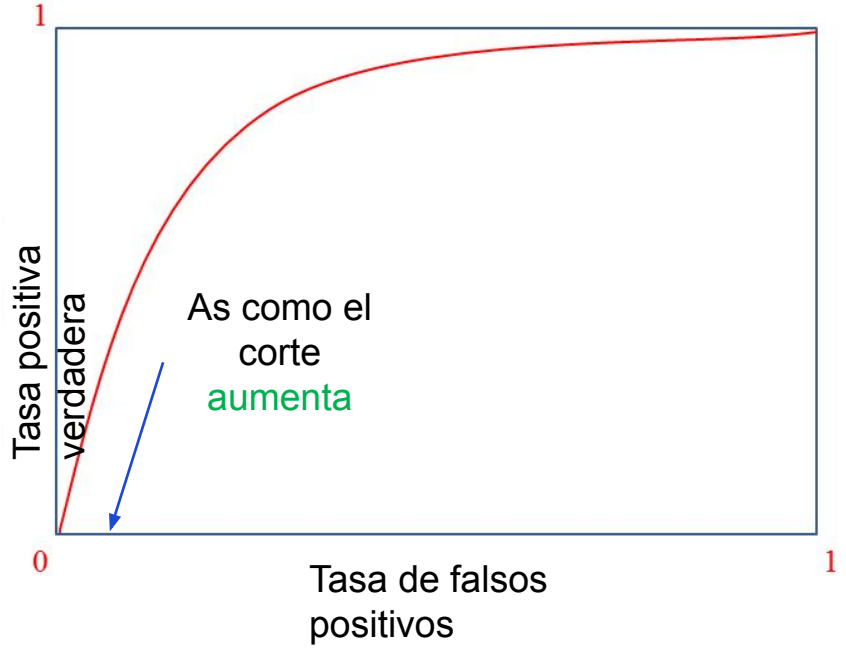
Positive Predicted Value = Precision

- curva ROC



- La curva ROC es una gráfica paramétrica con respecto a la probabilidad de corte.

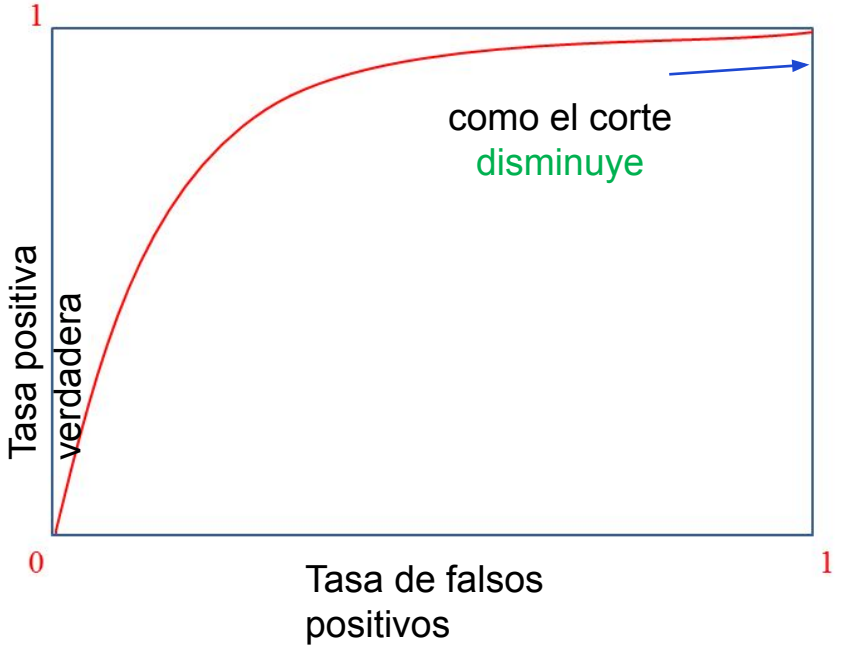
- curva ROC



As A medida que el límite aumenta (más cerca de 1)	
Métrica de rendimiento	Aumentar// Disminuir
Tasa de verdaderos positivos ((Sensibilidad)	⇓
Especificidad	⇑
Tasa de falsos positivos (1-Especificidad)	⇓
Precisión	⇑

- La curva ROC es una gráfica paramétrica con respecto a la probabilidad de corte.

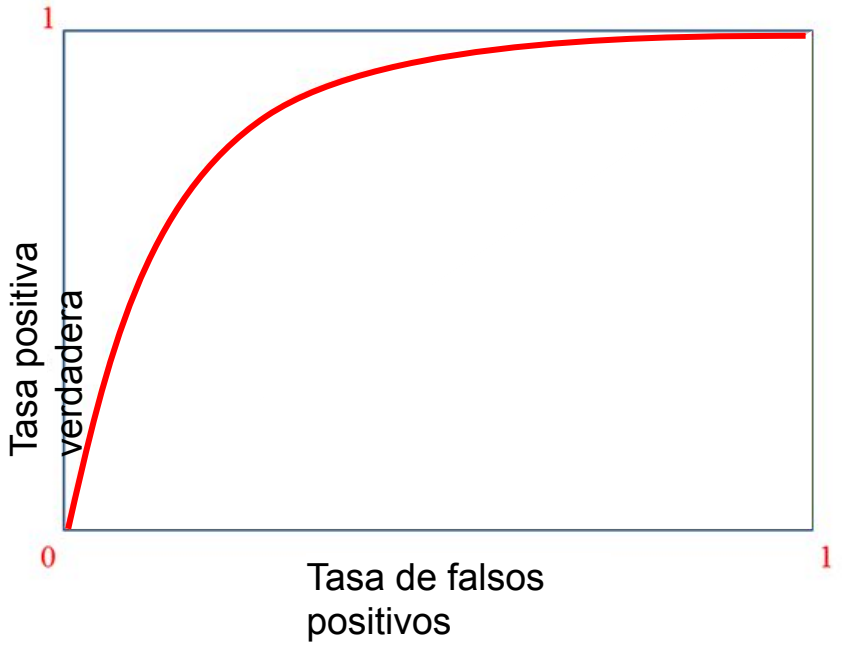
- curva ROC



A medida que el límite disminuye (más cerca de 0)	
Métrica de rendimiento	Aumentar// Disminuir
Tasa de verdaderos positivos ((Sensibilidad)	⇑
Especificidad	⇓
Tasa de falsos positivos (1-Especificidad)	⇑
Precisión	⇓

- La curva ROC es una gráfica paramétrica con respecto a la probabilidad de corte.

- curva ROC



- AUC significa Área bajo la curva.AUC
- Un AUC más cercano a 1 significa un buen rendimiento general.

- Interpretación utilizando el teorema de Bayes

Ex Hay 100 observaciones. En 6 casos, la respuesta real es 1.

En los 94 casos restantes, la respuesta real es 0. Se sabe que para un modelo de regresión logística dado, la sensibilidad = 0.92 y la especificidad = 0.90.Para una nueva observación (solo variables explicativas), este modelo predice 1 como respuesta.

¿Cuál es la probabilidad de que esta predicción sea correcta?

a) Tenemos $P(1) = 0.06$, $P(0) = 0.94$.

$P(Predicted\ 1 \mid Actual\ 1) = 0.92$ ⇦ "Sensitivity"

$P(Predicted\ 0 \mid Actual\ 0) = 0.90$ ⇦ "Specificity"

También podemos derivar $P(Predicted\ 1 \mid Actual\ 0) = 1 - P(Predicted\ 0 \mid Actual\ 0) = 0.1$

- Interpretación utilizando el teorema de Bayes

Ex Hay 100 observaciones. En 6 casos, la respuesta real es 1.

En los 94 casos restantes, la respuesta real es 0. Se sabe que para un modelo de regresión logística dado, la sensibilidad = 0.92 y la especificidad = 0.90.Para una nueva observación (solo variables explicativas), este modelo predice 1 como respuesta.

Para una nueva observación (solo variables explicativas), este modelo predice 1 como respuesta.

¿Cuál es la probabilidad de que esta predicción sea correcta?

b) La respuesta que buscamos viene dada por

$$P(Actual\ 1|Predicted\ 1) = \frac{P(Predicted\ 1|Actual\ 1)P(1)}{P(Predicted\ 1|Actual\ 1)P(1) + P(Predicted\ 1|Actual\ 0)P(0)}$$

$$= \frac{0.92 \times 0.06}{0.92 \times 0.06 + 0.1 \times 0.94} \cong 0.37$$

Ejercicio de codificación n.° 0307



Siga los pasos de práctica en el archivo 'ex_0307.ipynb'.

Unidad 4.

Árbol de decisión

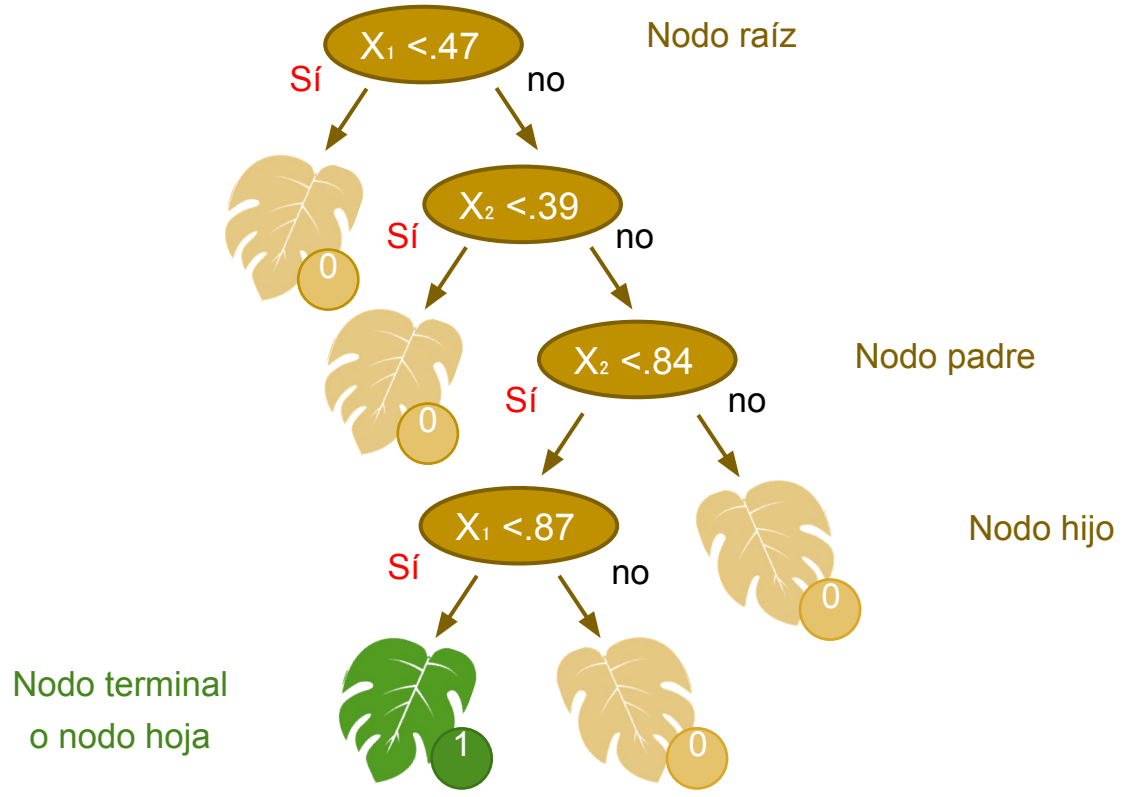
- 4.1. Algoritmo de árbol

Descripción general del árbol de decisión

- Definición
  - A Un modelo de clasificación que analiza datos recopilados del pasado y expresa los patrones encontrados (características de cada categoría) como una combinación de características.
- Objetivo
  - Clasificación de datos invisibles y predicción de valores categóricos.
  - Extracción de conocimiento generalizado en una estructura de árbol a partir de los datos.
- Clasificación en función de los tipos de variables objetivas
  - Variable categórica: árbol de clasificación
  - Variable continua: árbol de regresión

Composición

- Nodo, rama, profundidad



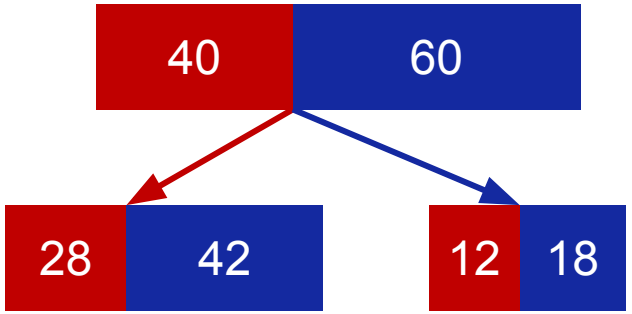
Cómo construir un modelo de árbol de decisión

Construcción de un árbol de decisión.

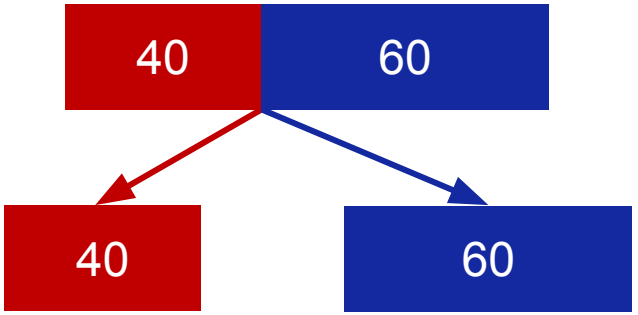
Construcción de un árbol de decisión	Elaborar un árbol de decisiones designando un criterio de división apropiado y reglas de parada de acuerdo con el propósito y la estructura de datos del análisis.
Derivación	Eliminar ramas que tienen un alto riesgo de tasa de error o reglas inapropiadas
Evaluación de validez	Evaluación del árbol de decisión mediante validación cruzada utilizando el gráfico de ganancia, el gráfico de riesgo o los datos de prueba. or
Interpretación y predicción	Interpretación del árbol de decisión y establecimiento de un modelo de predicción.

Criterio de división del árbol de decisión

- Proceso de análisis de un árbol de decisión.
  - División repetitiva: división repetitiva del espacio dimensional de variables independientes utilizando datos de entrenamiento:
  - Ramificación: ramificación con datos de evaluación.
- Criterio de división
  - Cree el árbol de clasificación para que la pureza del nodo hijo sea mayor que la del nodo padre.



Sin mejora

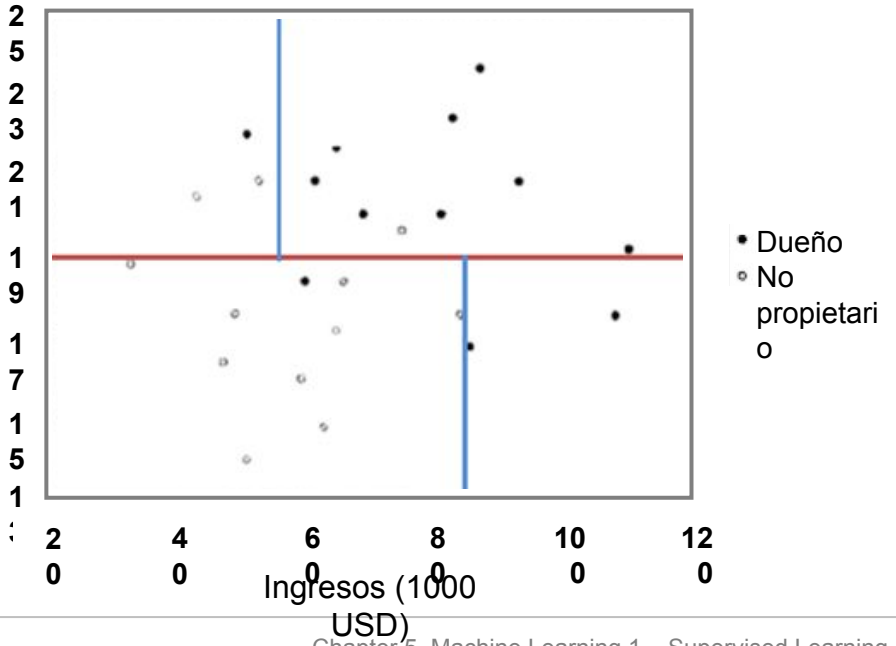


División perfecta

Proceso de división repetitivo

Objetivo

- Divide todo el espacio en rectángulos y haz que cada uno sea lo más puro u homogéneo posible. as or as
- Definición de "puro":‘’:
  - Dividir el área en espacios rectangulares puros u homogéneos tanto como sea posible.
  - Todas las variables del rectángulo final pertenecen al mismo grupo.



Proceso de división repetitivo

1) Selecciona x_i, una de las variables, y el valor de x_i (s_i como criterio de división) se designa para dividir el espacio de p-dimensiones en dos.

2) $x_i \rightarrow \{x_i <= s_i\} \cup \{x_i > s_i\}$

3) Selecciona una variable nuevamente y divídela de la misma manera.

4) Repite el proceso hasta que se alcance la pureza deseada.

Criterio dividido

Variables objetivas discretas

- Estadística de chi-cuadrado – valor p: crea nodos secundarios con una variable predictora con el valor p más bajo y la partición óptima
- Índice de Gini: selecciona nodos secundarios con una variable predictora que reduce el índice de Gini y la partición óptima.:
- Medida de entropía: crea nodos secundarios con una variable predictora con la medida de entropía más baja y la partición óptima.:

Criterio dividido

Variables objetivas continuas

- Estadística F en ANOVA: crea nodos secundarios con una variable predictora con el valor p más bajo y la partición óptima
- Reducción de varianza: crea nodos secundarios con la partición óptima que maximiza la reducción de varianza:

Selección de algoritmos y variables de clasificación.

	Variables objetivas discretas	Variables objetivas continuas
chaid (partición de múltiples espacios)	Estadística de chi-cuadrado	Estadística ANOVA F
CARRO (partición de espacio binario)	índice de Gini	Reducción de varianza
C4.5	Medida de entropía	

Medida de impureza

índice de Gini

- Selecciona los nodos hijos con una variable predictora que reduce el índice de Gini y realiza la partición óptima.
- Si el conjunto de datos T se divide en k categorías y las proporciones de rendimiento de las categorías son $p_1, \ldots, p_k$, se expresa mediante la siguiente ecuación:

$$Gini(T) = 1 - \sum_{l=1}^{k} p_l^2$$

alta impureza (diversidad), baja pureza



baja impureza (diversidad), alta pureza



Medida de entropía

- En termodinámica, la entropía mide el grado de desorden.
- Crea nodos hijos con una variable predictora con la medida de entropía más baja y la partición óptima.
- Si el conjunto de datos T se divide en k categorías y las proporciones de rendimiento de las categorías son $p_1, \ldots, p_k$, se expresa mediante la siguiente ecuación.

Ex Si 4 categorías constan de proporciones de 0,25, 0,25, 0,25, 0,25 (T0):

Ex Si 4 categorías consta de proporciones de 0,5, 0,25, 0,25, 0 (T1):

Criterios de parada

- Una regla para designar el nodo actual como nodo terminal sin más divisiones
  - Designa la profundidad del árbol de decisión.
  - Designa el número mínimo de registros en el nodo terminal.

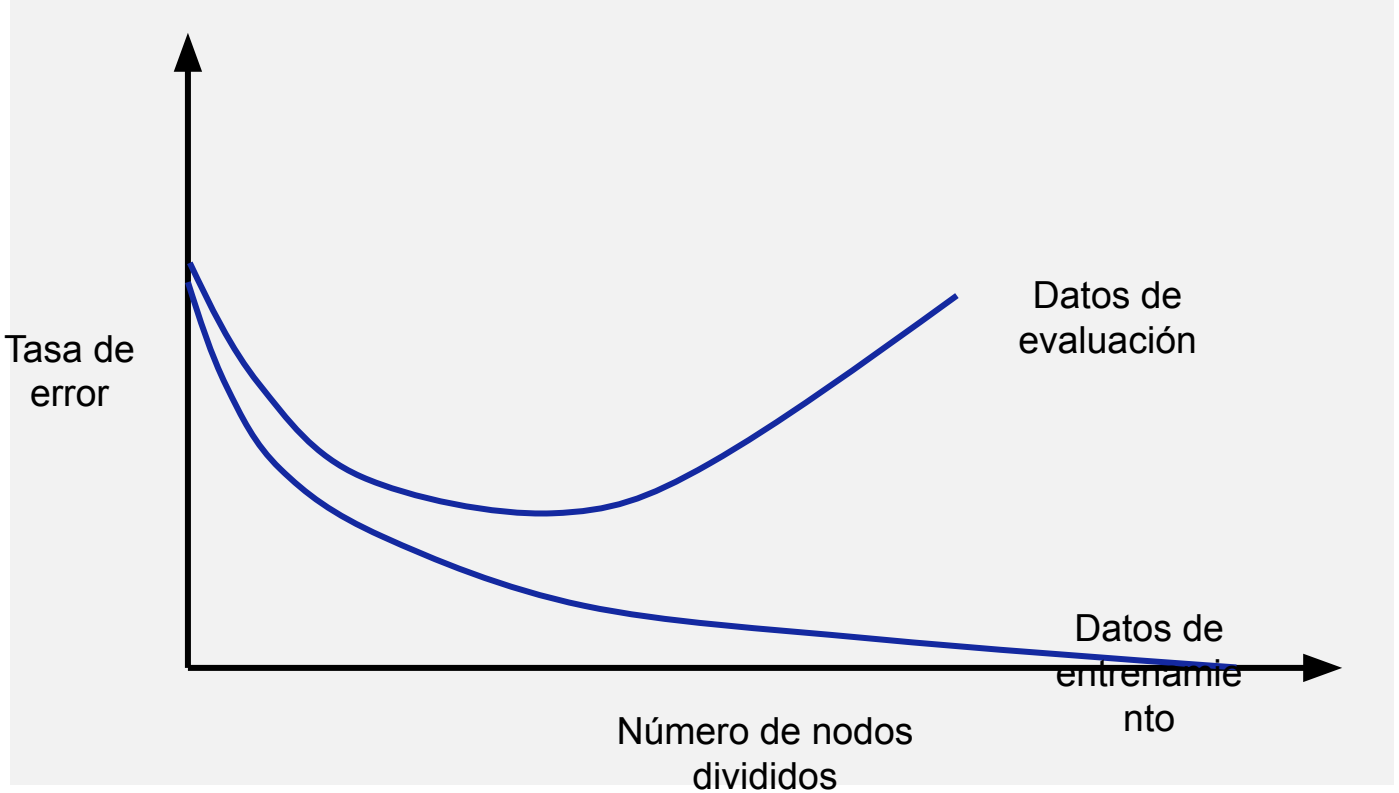
Criterios de ramificación

- Aplicación de datos de prueba.
  - Aplicación de los datos de prueba al modelo construido.
  - Revisar el valor predictivo del modelo construido a través de datos de prueba.
  - Eliminar las ramas que tienen un alto riesgo de tasa de error o regla de inferencia inapropiada

- Por un experto
  - Un experto revisa la validez de las reglas sugeridas en el modelo construido.
  - Eliminar reglas sin validez

Problema de sobreajuste

Gráfico del problema de sobreajuste



Ventajas

- Creación de reglas comprensibles (se pueden expresar con SQL)
- Útil en la predicción de clasificación.
- Capaz de trabajar con variables continuas y discretas.
- Muestra una variable más relativamente significativa

Contras

- No apto para predecir valores de variables continuas.
- No se puede realizar el análisis de series temporales
- No estable

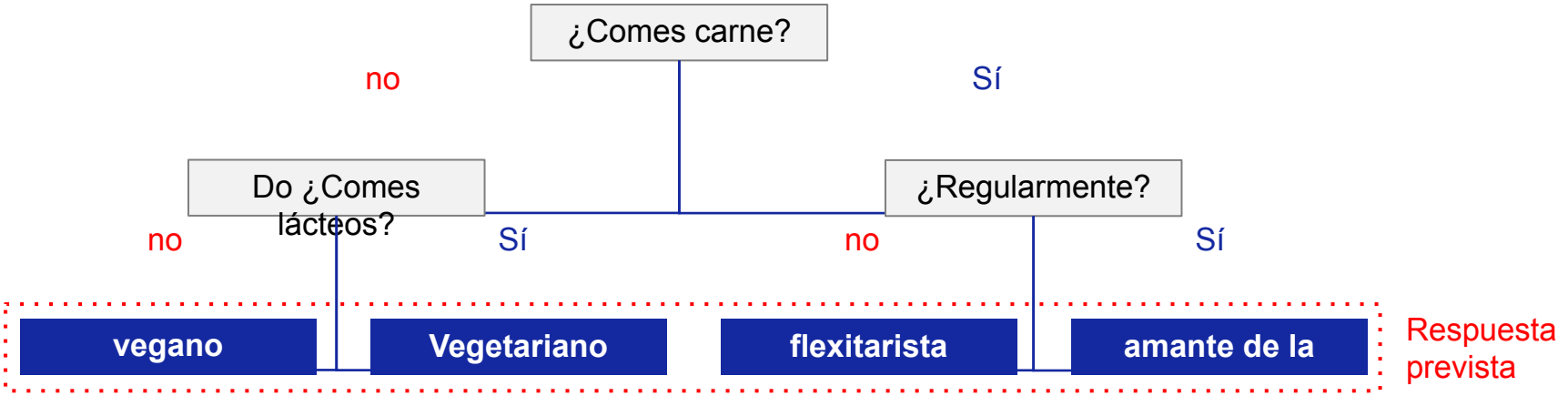
Algoritmo de árbol

- Ventajas
  - Intuitivo y fácil de entender.
  - Sin suposiciones sobre las variables.
  - No es necesario escalar ni normalizar los datos
  - No es tan sensible a los valores atípicos

- Contras
  - No tan poderoso en la forma más básica.
  - Propenso al sobreajuste. Por lo tanto, a menudo es necesario “poda”.

Árbol de clasificación

Ex



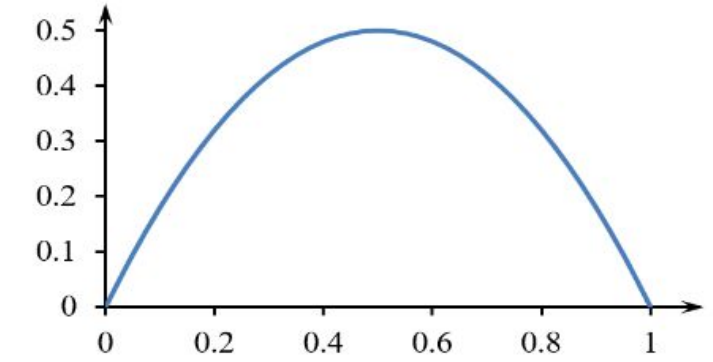
- El paso de entrenamiento crea una estructura de árbol invertida como se muestra arriba.
- Las condiciones se evalúan en los nodos y luego se ramifican.
- Cada nodo hoja corresponde a una región en el espacio configuracional.

Árbol de clasificación

- La estructura del árbol se entrena minimizando la impureza (o entropía) de Gini.

or

$\hat{p}_{mk}(1-\hat{p}_{mk})$



- G_m es la impureza de Gini en el nodo hoja m.
- $Entropy_m$ s la entropía en el nodo hoja m.

} Cuanto más pequeño, mejor.

- Here, $\hat{p}_{mk}$ es la proporción de la calse k en el nodo hoja m.
- Kes el número total de clases posibles.
- La clase con la mayor proporción es la predicción en ese nodo hoja.

Árbol de clasificación: procedimiento

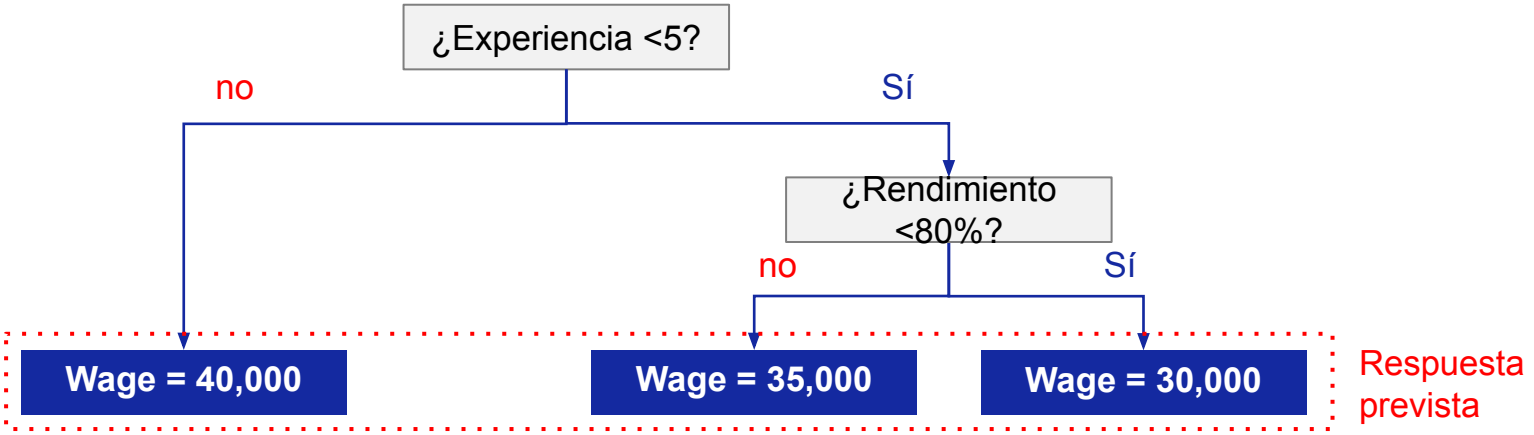
a) Haz un árbol básico.

b) Poda de ramas que no proporcionen mejor rendimiento. La poda se puede realizar durante el paso de validación cruzada.

c) Predecir con el árbol optimizado.

Árbol de regresión

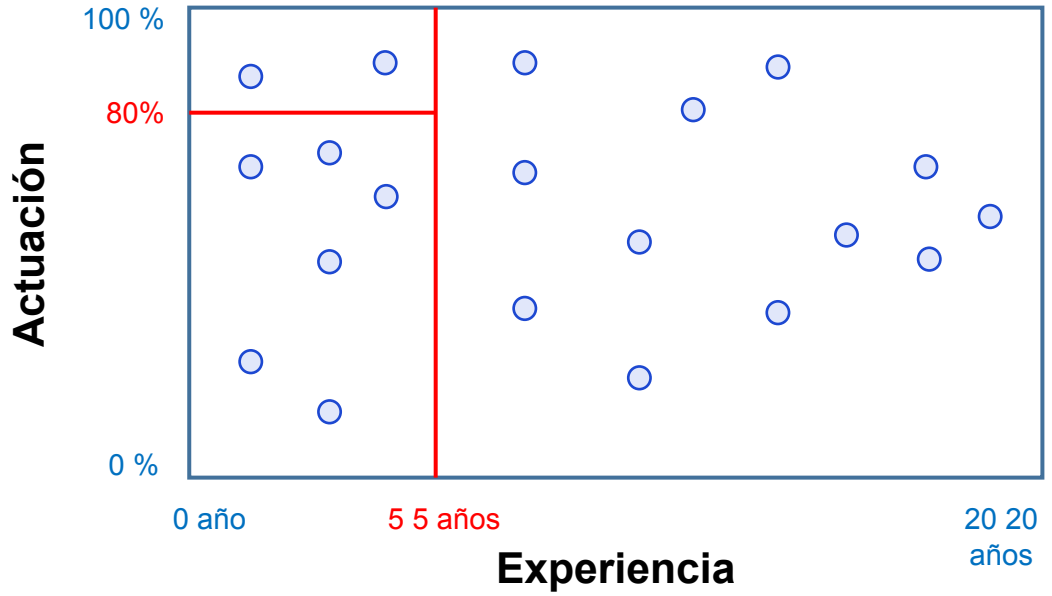
Ex



- Predice valores numéricos en lugar de categorías o clases.
- Las condiciones se evalúan en los nodos y luego se ramifican.
- Cada nodo hoja corresponde a una región en el espacio configuracional.

Árbol de regresión

Ex



- Cada nodo hoja corresponde a una región en el espacio configuracional.

Árbol de regresión

- El espacio de configuración se divide en regiones:$\{R_1, R_2, \ldots, R_J\}$.
- La estructura del árbol se entrena minimizando la RSS (suma de los cuadrados de los residuos):

  - J es el número total de nodos hoja.
  - El valor predicho en la j-ésima región $\hat{y}_{R_j}$ viene dado por el promedio en esa región.
  - Para las observaciones que pertenecen a la región R_j, se asigna la misma respuesta predicha $\hat{y}_{R_j}$, de forma análoga al árbol de clasificación

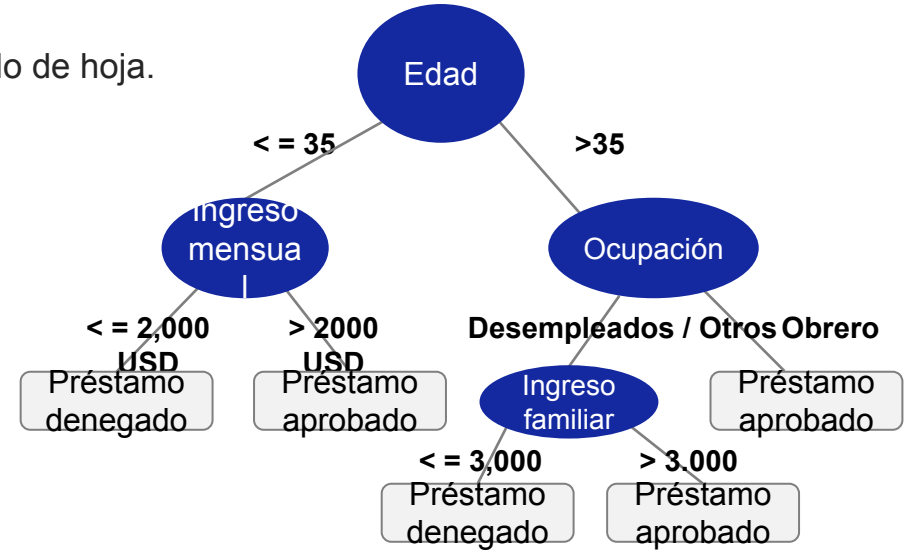
Hiperparámetros del clasificador/regresor del árbol de decisión de scikit-learn:

Hiperparámetro	Explicación
max_depth	La profundidad máxima de un árbol.
min_samples_leaf	El número mínimo de puntos de muestra necesarios para estar en un nodo hoja.
min_samples_split	El número mínimo de puntos de muestra necesarios para dividir un nodo interno.
max_features	La cantidad de características a considerar al buscar la mejor división
max_leaf_nodes	El número máximo de nodos de hojas en el árbol.

- Es necesario ajustarlo para obtener un rendimiento optimizado.
- Puede encontrar más información en:
- Clasificador: https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html Regresor: https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeRegressor.html

Árbol de decisión

- El árbol de decisión se refiere a una técnica de modelado como la forma de un árbol que se ramifica desde los nodos de raíz hasta los de hoja. Depende de valores de referencia de variables independientes (variables explicativas o de entrada) que afectan la clasificación o predicción de variables objetivas.
- En el árbol de decisión, cada nodo se divide en forma de si-entonces dependiendo de las características de las variables explicativas o los valores de referencia. Al seguir la estructura de árbol, es posible comprender fácilmente cómo se clasifica el valor del atributo de los datos en la categoría.
- La figura que se proporciona a continuación es una forma típica del árbol de decisión. Según el ejemplo, "edad" es la raíz. Se puede inferir que la “edad” es la variable más significativa a la hora de decidir la aprobación del préstamo.
- El nodo de forma cuadrada al final de cada rama es el nodo de hoja.



Criterios divididos del árbol de decisión

- El árbol de decisión le da una pregunta a cada nodo y separa los datos ramificándolos según la respuesta.
- Para evaluar qué tan bien están separados los datos, se requiere un criterio específico. En general, la impureza es el criterio de evaluación. La impureza aumenta a medida que se mezclan varias clasificaciones en el nodo; es el más bajo cuando hay una sola clasificación.
- Por lo tanto, si la impureza de cada nodo es baja después de la división del nodo, podemos decir que el árbol de decisión está bien clasificado.
- Las medidas de impureza incluyen la impureza de Gini y la entropía.

- El índice impureza es 0 cuando $p_i = 0$ or $p_i = 1$, y alcanza su valor máximo cuando $p_i = \frac{1}{2}$, formando así una parábola. En otras palabras, el índice de impureza es mínimo cuando existe una clasificación certera en el nodo o cuando el nodo está completamente libre de cualquier clasificación (pureza máxima). Por el contrario, la impureza llega a su punto máximo cuando se encuentran muchas clasificaciones mezcladas en el mismo nodo.

Ejercicio de codificación n.° 0308



Siga los pasos de práctica en el archivo 'ex_0308.ipynb'.

Unidad 5.

Algoritmo ingenuo de Bayes

- 5.1. Algoritmo ingenuo de Bayes

Algoritmo ingenuo de Bayes

- Acerca del algoritmo Naïve Bayes
  - Es una aplicación sencilla del teorema de Bayes.

- Ventajas
  - Intuitivo y sencillo
  - No es tan sensible al ruido y a los valores atípicos.
  - Rápido

- Contras
  - Se supone que las características son independientes, lo que puede no ser estrictamente cierto.
  - No se encuentra entre los algoritmos de mejor rendimiento

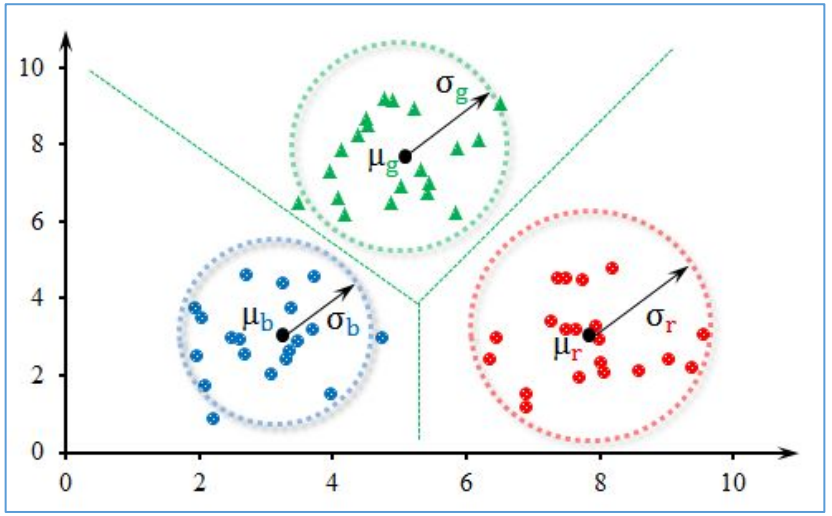
- Acerca del algoritmo Naïve Bayes
  - Teorema de Bayes:

  - Ahora tomamos A=$Class$ y B=$Data$, entonces:

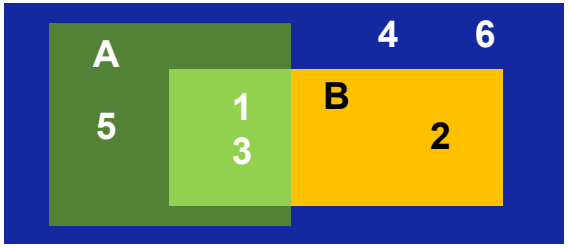
  - Estamos más interesados en comparar probabilidades relativas:

- Acerca del algoritmo Naïve Bayes
  - Podemos aproximar $P(Data \mid Class)$ mediante una distribución gaussiana (normal):$P(Data \mid Class$

donde los parámetros μ_j y σ_j se “aprenden” de los datos de entrenamiento.μ_j σ_j



- variable aleatoria
  - La variable cuyos valores se desconocen hasta el resultado.
  - Eventos independientes
    - Si la probabilidad de la ocurrencia simultánea de dos eventos es idéntica al producto de las probabilidades de que cada evento ocurra por separado, entonces los dos eventos son independientes entre sí.
    - $P(A \cap B) = P(A)P(B)$
  - Dados

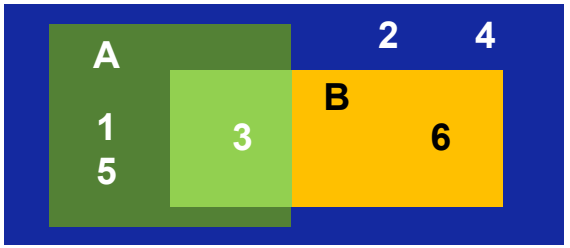


Probabilidad condicional

- Eventos de B (1,2,3) cuando ocurren los eventos A (1,3,5)
- 0.6666667

		Total	Evento A	Evento B	Probabilidad A	Probabilidad B
Evento A	Número impar	1	1	1	0.5	0.5
Evento B	Menos de 3	2		2		
		3	3	3		
		4				
		5	5			
		6				

- variable aleatoria
  - La variable cuyos valores se desconocen hasta el resultado.
  - Eventos independientes
    - No afectado
  - Dados

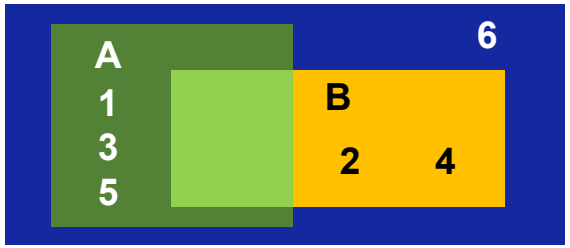


		Total	Evento A	Evento B	Probabilidad A	Probabilidad B
Evento A	Número impar	1	1		0.5	0.333
Evento B	Múltiplo de 3	2				
		3	3	3		
		4				
		5	5	6		
		6				

Probabilidad condicional

- Eventos de B (3, 6) cuando ocurren los eventos A (1,3,5)
- 0.333333333

- variable aleatoria
  - La variable cuyos valores se desconocen hasta el resultado.
  - Eventos exclusivos
    - La intersección es el conjunto nulo.
  - Dados



- Introducción a la estadística bayesiana

1) Diferenciar grupos de personas para "ir de compras" y "mirar escaparates" mediante la interpretación bayesiana.

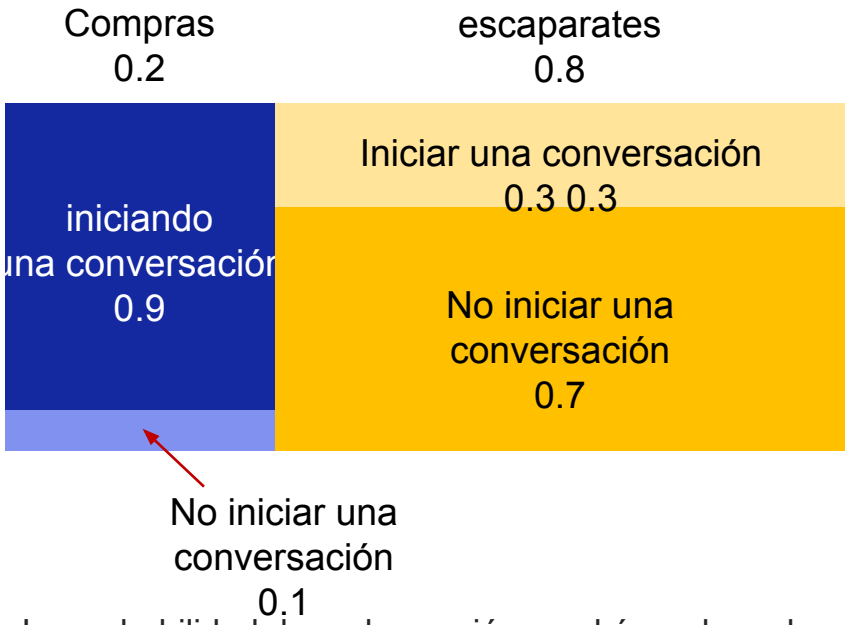
- Establecer probabilidad previa para diferentes tipos
- Probabilidad previa: Probabilidad antes de obtener cierta información.



- Introducción a la estadística bayesiana

2) Establecer la probabilidad condicional de "iniciar una conversación" para cada tipo.

	Probabilidad de iniciar una conversación.	Probabilidad de no iniciar una conversación	Total
Compras	0.9	0.1	1
escaparates	0.3	0.7	1

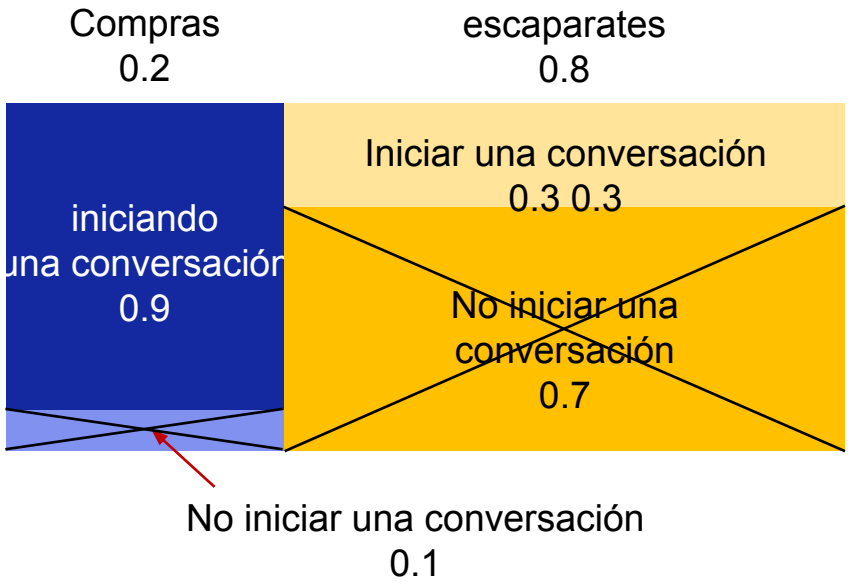


- La probabilidad de cada sección es el área de cada rectángulo. (0,18, 0,02, 0,24, 0,56)

- Introducción a la estadística bayesiana

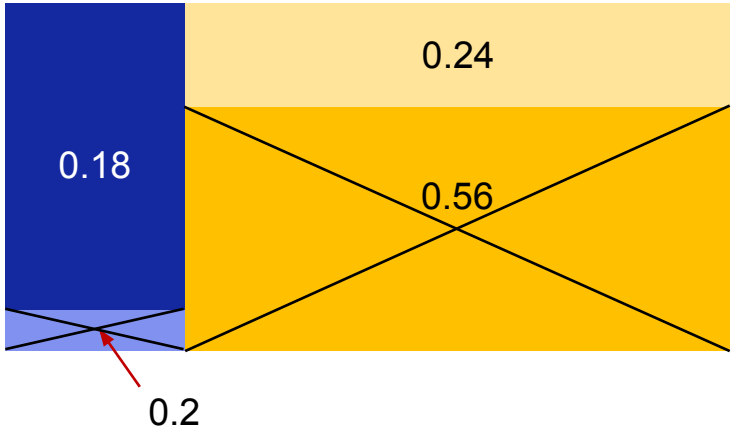
3) Eliminar el "evento imposible" de las observaciones.

- El cliente inició una conversación (información adicional).



- Introducción a la estadística bayesiana

4) Calcule la probabilidad bayesiana del grupo de compras.



	Compras	escaparates
Normalización	3/7	4/7
	43%	

Resultado
Iniciar una conversación

Causa
un grupo determinado

- El resultado es 0,18:0,24, que es 3:4.
- Así, se puede estimar que la probabilidad de que el cliente que inició una conversación esté en el grupo de compras es de 3/7 (43%). Se llama estadística bayesiana o probabilidad posterior.
- Elige si una persona que inició una conversación está en el grupo de compras o de escaparates según la probabilidad.

- Introducción a la estadística bayesiana
  - La probabilidad de que un cliente que inició una conversación realice una compra ahora es diferente.
  - Actualización bayesiana para tipo de cliente

Resumen final	La probabilidad previa de ser un grupo de compras.	0.2
	Observar para iniciar una conversación.	3/7 → 0.428571
	La probabilidad posterior de ser un grupo de compras.	

  - Seguramente el cliente no estaría en el grupo de compras, pero la probabilidad se duplica.
  - **La estimación bayesiana consiste en "realizar una actualización bayesiana de la probabilidad posterior basada en la observación del comportamiento de la probabilidad anterior".**is ‘’
  - Este método de estimación se denomina estadística bayesiana.
  - **Teorema de Bayes**
    - **Obtener la probabilidad posterior en función de la probabilidad anterior**

Ejercicio de codificación n.° 0309



Siga los pasos de práctica en el archivo 'ex_0309.ipynb'.

Unidad 6.

Algoritmo KNN

- 6.1. Algoritmo KNN

Algoritmo KNN

- Acerca de KNN (K-vecinos más cercanos)
  - Uno de los algoritmos más simples.
  - La predicción se basa en los k puntos vecinos más cercanos.
  - Existen variantes de clasificación y regresión.
    - Clasificación: predicción decidida por la clase mayoritaria de los vecinos más cercanos.
    - Regresión: predicción dada por la media de los vecinos más cercanos.

- Acerca de KNN (K-vecinos más cercanos)
  - KNN es una técnica de clasificación y predicción para un conjunto de datos con la categoría desconocida de la variable objetivo. Designa la categoría más similar del conjunto de datos circundante.
  - Para KNN, se requieren criterios específicos para medir la "analogía" entre un determinado conjunto de datos y el circundante y cuántos conjuntos de datos se utilizarán para la clasificación final de la categoría de variable objetivo.

1) Medición de analogía

- Hay muchas formas de medir la analogía entre datos. En general, el cálculo de analogía se realiza invirtiendo la distancia euclidiana al cuadrado entre dos puntos o utilizando el coeficiente de correlación de Pearson. Si los puntos son variables discretas, utilice el coeficiente de Jaccard.

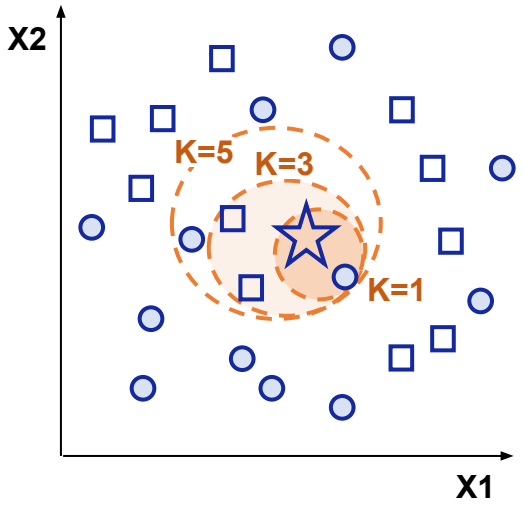
2) Criterios de clasificación de variables objetivas

- K en KNN se refiere al número de puntos de datos circundantes que se utilizan para clasificar variables objetivas del punto de datos específico después de medir la analogía entre el punto de datos específico y otros conjuntos de datos circundantes.

Ex Tomemos como ejemplo las recomendaciones de películas. Recomienda una película que es similar a la película favorita del cliente y que el cliente aún no ha visto. Cuando se considera solo una película que es más similar a la favorita del cliente, es "1 vecino más cercano". Cuando se consideran tres películas similares, sería "3 vecinos más cercanos".

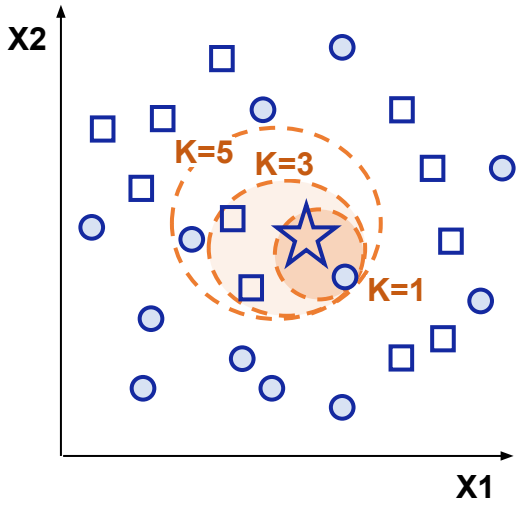
- Asimismo, el K-vecino más cercano es una técnica para determinar una nueva categoría según el principio de la regla de la mayoría. Encuentra k puntos de datos circundantes similares al punto de datos específico y considera la categoría de clasificación del punto de datos específico.

- Acerca de KNN (K-vecinos más cercanos)



- La figura anterior conceptualiza el cambio en las variables objetivas dependiendo de la configuración del valor K en el método del vecino K más cercano. El punto '☆' es el valor de los datos que debe clasificarse; los puntos en forma de cuadrado y círculo son otros puntos de datos presentes alrededor.
- Al establecer K=1, dado que el punto de datos más cercano a la estrella es un círculo, la variable objetiva de '☆' se clasificará como '○'. Por otro lado, si K=3, se considerarán los tres puntos más cercanos a '☆', que incluyen dos cuadrados y un círculo. En este caso, '☆' se clasificará como '□'. Asimismo, si K=5, dado que hay más círculos alrededor de '☆', se clasificará nuevamente como '○'.

- Acerca de KNN (K-vecinos más cercanos)

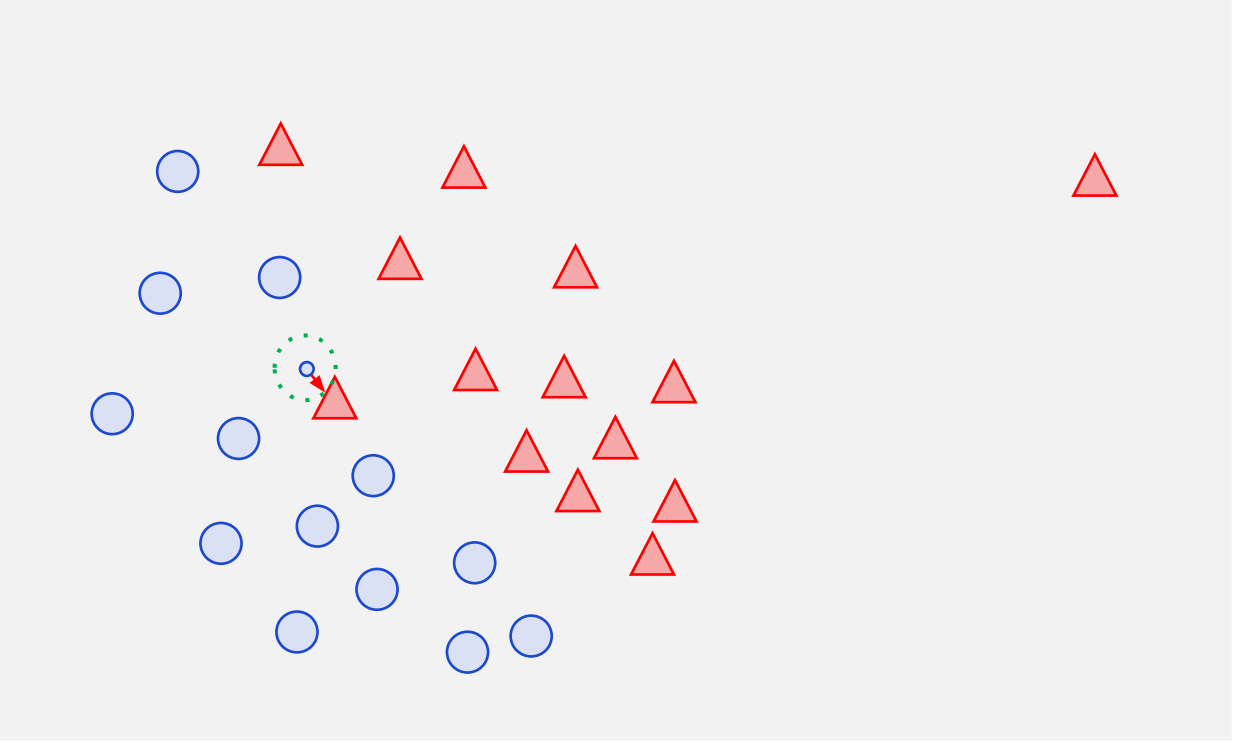


- Por lo tanto, diferentes valores de K cambian significativamente el resultado previsto por el vecino más cercano de la categoría de variable objetivo. Esta es la razón por la que es importante establecer un valor de K apropiado en el método del vecino más cercano.
- Sin embargo, no existen estándares teóricos o estadísticos claros para un valor de K apropiado. Generalmente se establecen diferentes valores de K para realizar pruebas repetidas, y el valor final de K se fija una vez que muestra el rendimiento de clasificación óptimo. Por lo general, se designa un valor de K inicial aleatorio entre K=3 y K=9, que luego se utiliza para probar el rendimiento de clasificación con los datos de entrenamiento y evaluación para seleccionar el valor de K óptimo.

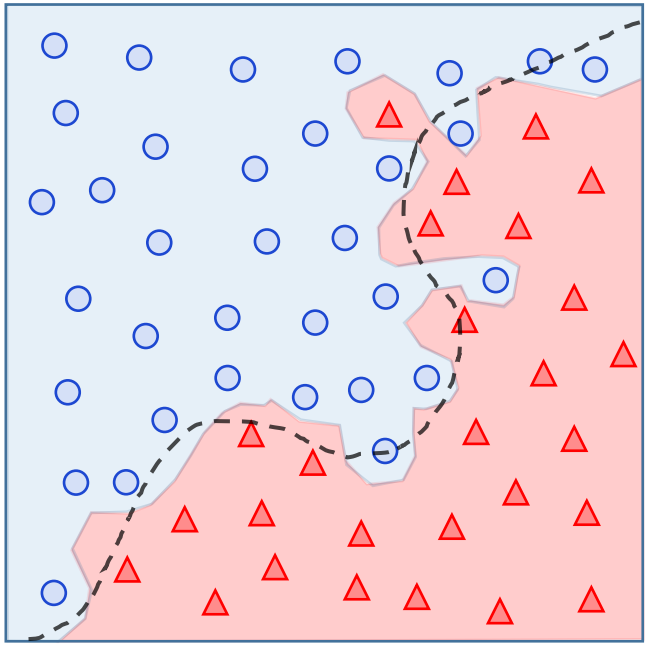
- Ventajas
  - Sencillo e intuitivo
  - No hay parámetros de modelo para calcular. Entonces, no hay ningún paso de entrenamiento.

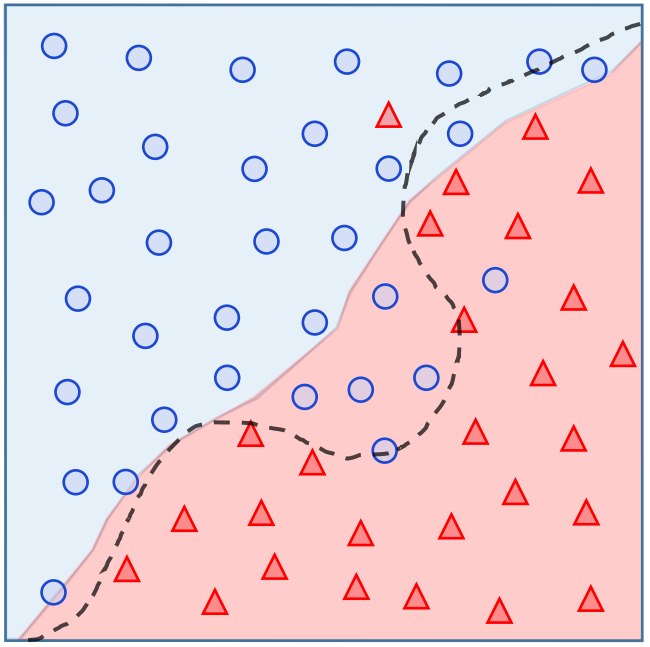
- Contras
  - Como no existe un “modelo”, se puede extraer poca información.
  - No hay parámetros de modelo que almacenen el patrón aprendido. El conjunto de datos de entrenamiento es necesario para la predicción.
  - La predicción no es eficiente ⇨ "Algoritmo vago".

- Algoritmo de clasificación KNN



- Algoritmo de clasificación KNN
  - Cuando k es demasiado pequeño, puede ocurrir un sobreajuste.





Ejercicio de codificación n.° 0310



Siga los pasos de práctica en el archivo 'ex_0310.ipynb'.

Unidad 7.

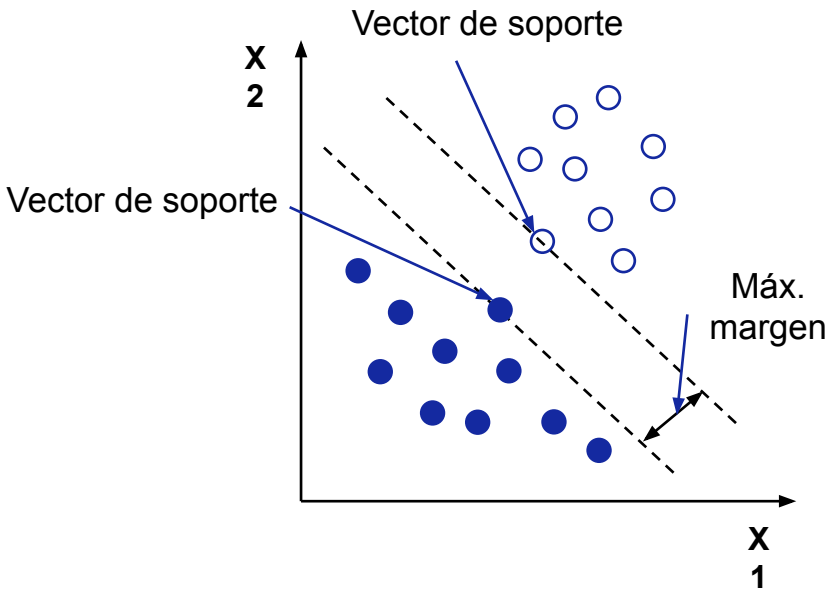
Algoritmo SVM

- 7.1. Algoritmo SVM

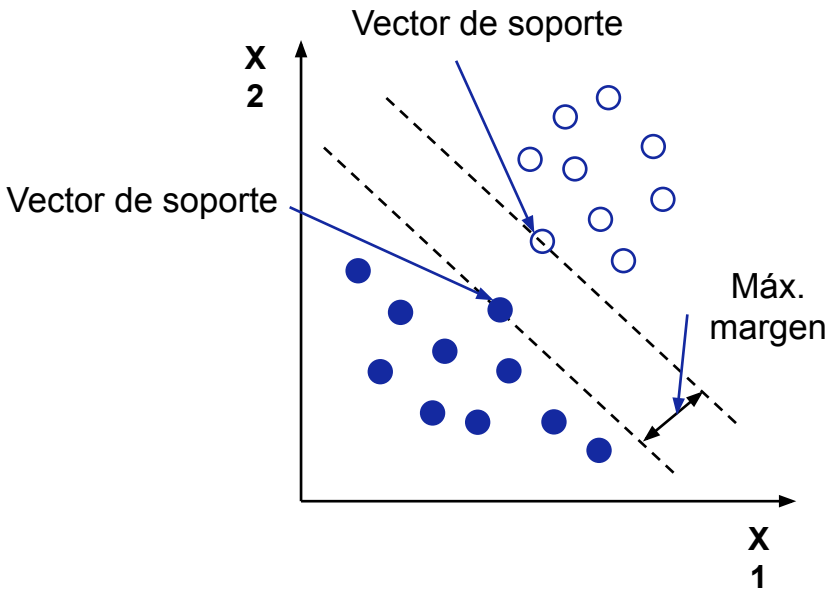
Algoritmo SVM

- Acerca de SVM (máquina de vectores de soporte)
  - Precisión de clasificación mejorada al maximizar el margen
  - Límite de clasificación no lineal efectivo mediante la transformación "kernel"

- Acerca de SVM (máquina de vectores de soporte)
  - El vector de soporte es un modelo que clasifica datos encontrando la línea (o hiperplano) donde la distancia (margen) entre datos en diferentes categorías se vuelve máxima.
  - Como se muestra en la figura siguiente, el modelo de máquina de vectores de soporte encuentra el hiperplano que divide los datos en dos categorías diferentes con el máximo margen para clasificar los datos.
  - Habría muchas líneas o planos que dividirían los datos en dos categorías diferentes. Sin embargo, los puntos que superan el límite de clasificación pueden ocurrir en datos de evaluación o en datos futuros desconocidos, ya que los puntos cerca del límite cambian ligeramente.



- Acerca de SVM (máquina de vectores de soporte)
  - Para minimizar tal posibilidad, se debe encontrar una línea (o hiperplano) que establezca el margen máximo entre diferentes categorías de datos. En otras palabras, el objetivo es encontrar un hiperplano que pueda inducir la máxima diferenciación para la mejor generalización posible para clasificar y predecir los datos futuros, no los datos de entrenamiento actuales.
  - El punto de la categoría más cercano a la línea límite se denomina vector de soporte y cada clasificación debe tener al menos un vector de soporte.

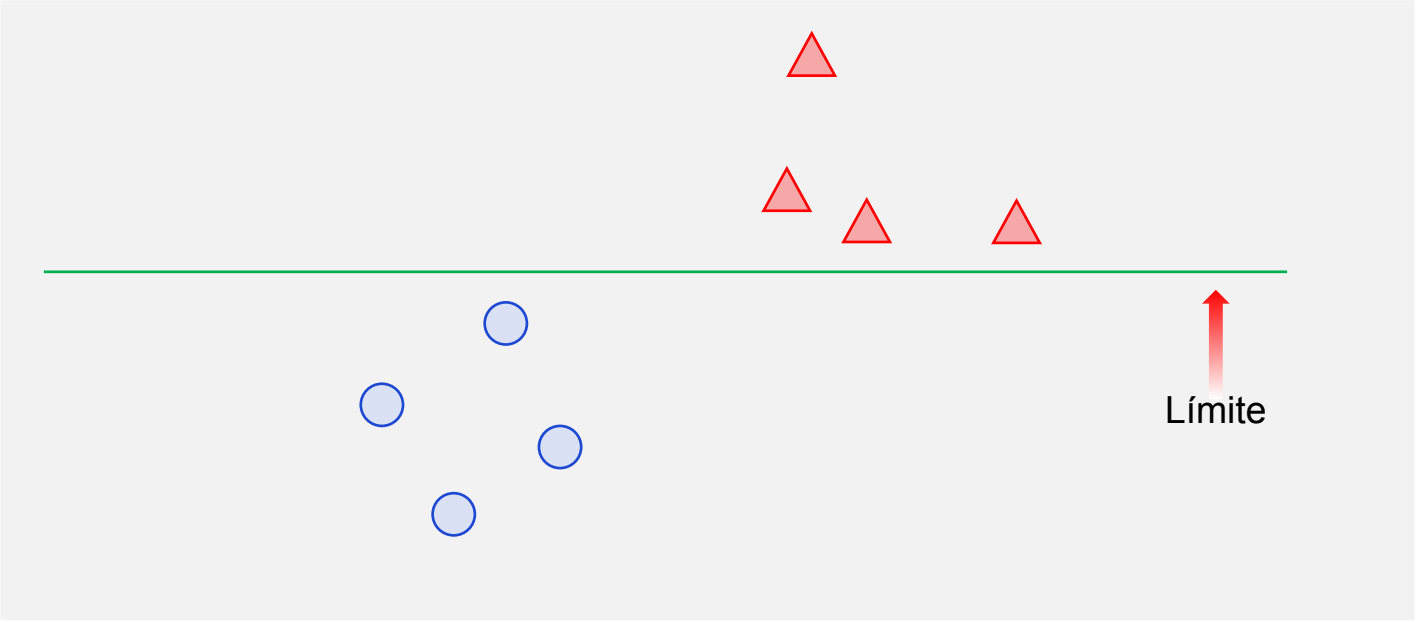


- Acerca de SVM (máquina de vectores de soporte)
  - Sin embargo, no siempre es posible clasificar todos los datos de forma lineal. A veces, la clasificación de datos debe realizarse en una curva o en un plano de clasificación no lineal más complejo.
  - Si es así, utilice el método truco del Kernel para asignar los datos dados a una dimensión superior y encontrar un hiperplano que pueda clasificar los datos en la dimensión convertida.
  - En lugar de convertir los datos a una dimensión superior, el método del truco Kernel utiliza la función Kernel que devuelve un valor similar al que se obtiene cuando se realizan cálculos internos entre vectores en la dimensión superior. Esto puede dar como resultado una clasificación no lineal sin mover los datos a una dimensión superior.
  - Las principales funciones de Kernel incluyen núcleos polinomiales, gaussianos y sigmoideos.

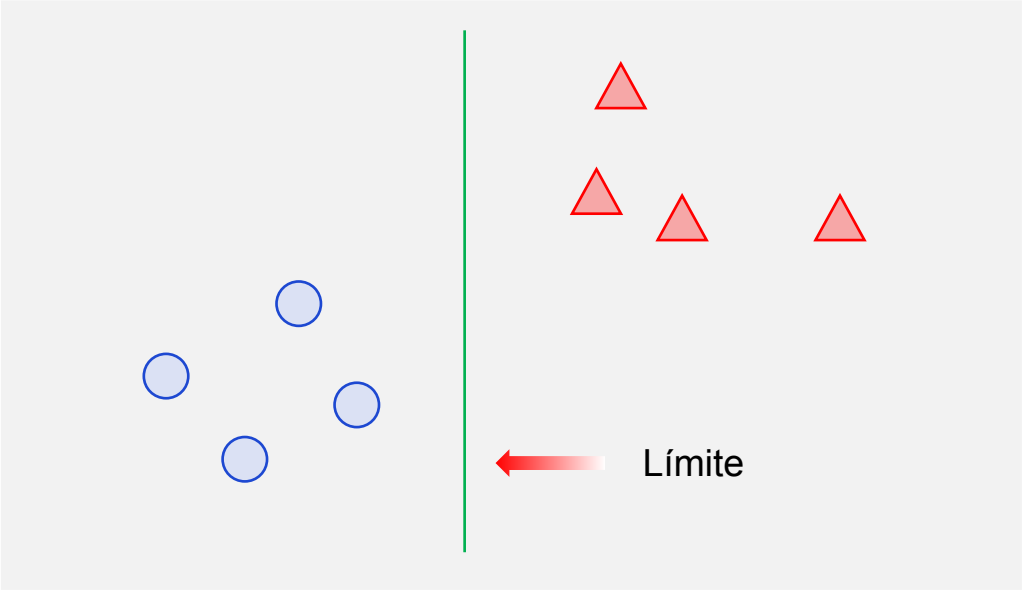
- Ventajas
  - No muy sensible a los valores atípicos.
  - El rendimiento es bueno.

- Contras
  - El entrenamiento es relativamente lento. Funciona mal para grandes cantidades de datos.
  - El kernel y el conjunto de hiperparámetros deben optimizarse cuidadosamente.
  - No se puede obtener mucha información.

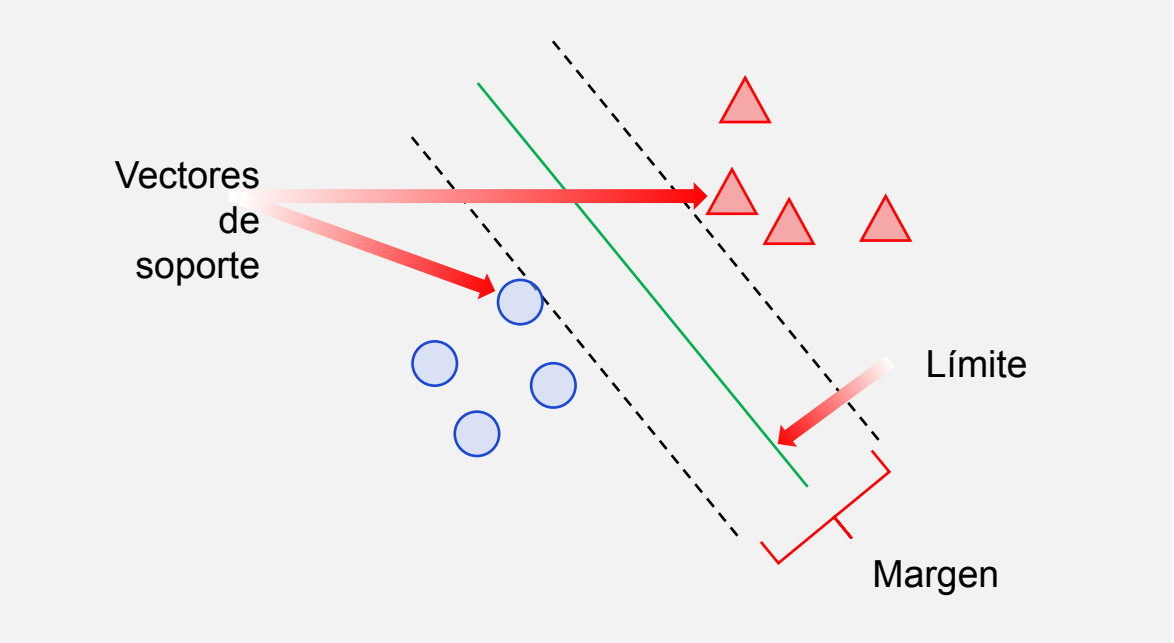
- Algoritmo de clasificación SVM



- Algoritmo de clasificación SVM



- Algoritmo de clasificación SVM



- Hiperplano
  - Para un espacio configuracional de k dimensión, el hiperplano tiene la dimensión $k-1$.

    Ex Para un espacio bidimensional, un hiperplano es una línea bisectriz que se puede parametrizar como:

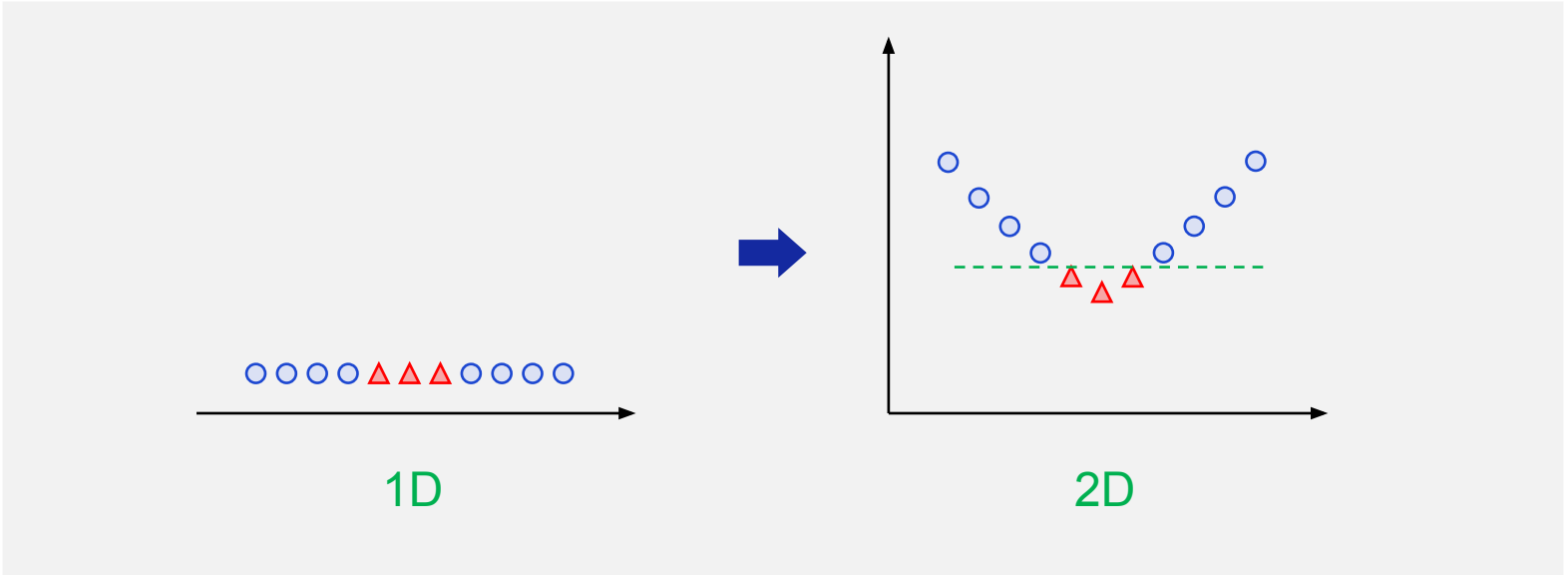
    El espacio bidimensional se subdivide en dos:

    ¡Una observación pertenecería a cualquiera de ellos ⇨ clasificación binaria!⇨ b

- Optimización de márgenes (para la y binaria)y)
  - Se maximiza el margen del límite (boundary margin).
  - Si no es posible establecer un margen despejado, se permite un error dentro de un límite.
  - El objetivo es maximizar M mediante la optimización de los parámetros $\beta_0, \beta_1, \ldots, \beta_k$ sujeto a las siguientes restricciones:
    - Constraint 1: $\sum_{j=1}^{k} {\beta_j}^2 = 1$
    - Constraint 2: $y_i(\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \beta_k x_{ik}) \geq M(1 - \varepsilon_i), \quad i = 1,2,\ldots,n$
      y_i o es -1 o +1.
    - Constraint 3: $\varepsilon_i \geq 0$ and $\sum_{i=1}^{k} \varepsilon_i \leq C$
      ⇦ C es un hiperparámetro relacionado con los errores de clasificación errónea.

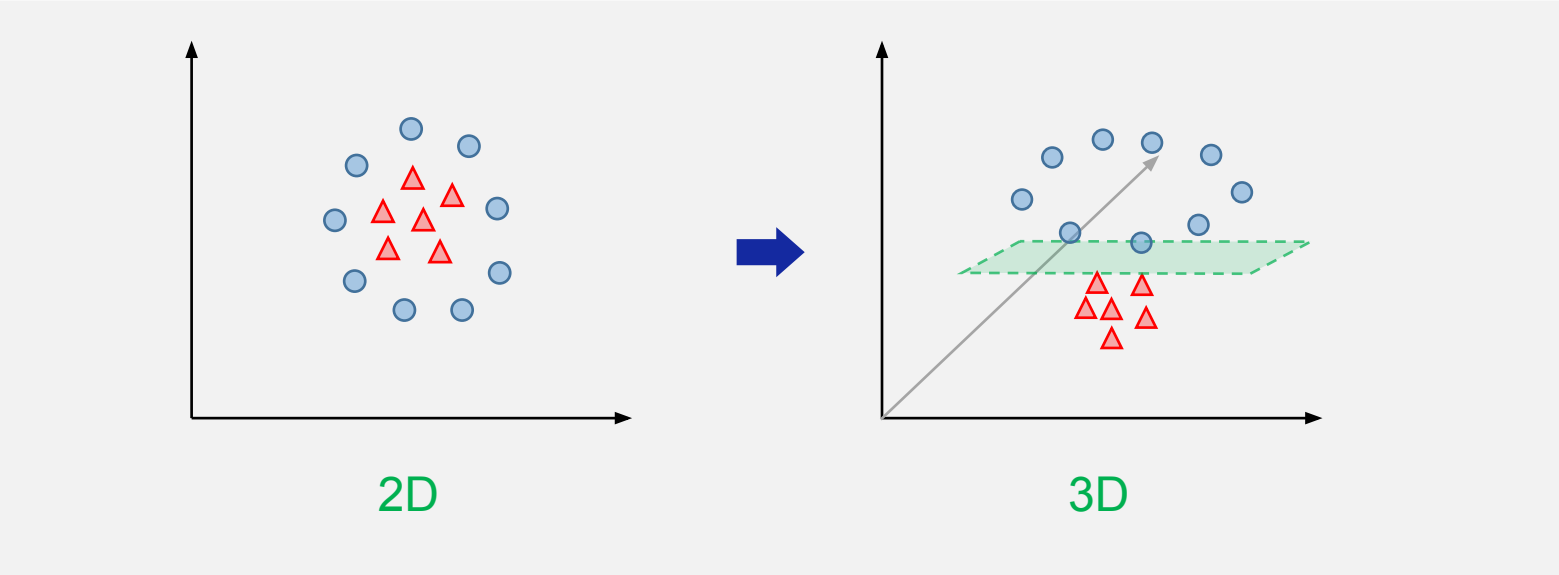
- Núcleo
  - Mapeo a una dimensión superior utilizando las funciones "kernel".
  - Las funciones del kernel introducen un límite de clasificación no lineal efectivo.

Ex Núcleo polinomial



- Núcleo
  - Mapeo a una dimensión superior utilizando las funciones "kernel".
  - Las funciones del kernel introducen un límite de clasificación no lineal efectivo.

Ex Núcleo polinomial



- Núcleo
  - Mapeo efectivo a una dimensión superior mediante el producto interno de dos vectores $\boldsymbol{x}_1$ y $\boldsymbol{x}_2$.
    - Lineal: $K(\mathrm{x}_1, \mathrm{x}_2) = \mathrm{x}_1^{\mathrm{t}}\mathrm{x}_2$
    - Polinomial: $K(\mathrm{x}_1, \mathrm{x}_2) = (\mathrm{x}_1^{\mathrm{t}}\mathrm{x}_2 + \mathrm{c})^{\mathrm{d}}$, donde $\mathrm{c} > 0$
    - Sigmoide: $K(\mathrm{x}_1, \mathrm{x}_2) = tanh(\mathrm{a}(\mathrm{x}_1^{\mathrm{t}}\mathrm{x}_2) + \mathrm{b})$, donde $\mathrm{a}, \mathrm{b} > 0$
    - Función de base radial (rbf): $K(\mathrm{x}_1, \mathrm{x}_2) = exp(-\gamma\|\mathrm{x}_1 - \mathrm{x}_2\|^2)$, donde $\gamma > 0$

Ejercicio de codificación n.° 0311



Siga los pasos de práctica en el archivo 'ex_0311.ipynb'.

Unidad 8.

Algoritmo de conjunto

- 8.1. El concepto de algoritmo de conjunto y votación
- 8.2. Ensacado y bosque aleatorio
- 8.3. boosting

Algoritmos de conjunto

- Acerca de los algoritmos de conjunto
  - Modelo predictivo fuerte basado en los alumnos más débiles.
  - **Tipo de votación::**
    - Una colección de estudiantes básicos que “votan”
    - Un conjunto de diferentes tipos de “learners” Ex Combine Tree, KNN, SVM, etc.
  - **Tipo de embolsado::**
    - Una colección de estudiantes débiles e independientes que “votan”
    - Un conjunto del mismo tipo de estudiantes débiles Ex Random Forest
  - **Tipo de impulso::**
    - Una serie de alumnos débiles que aprenden y predicen de forma adaptativa Ex AdaBoost, GBM, XGBoost, etc.
    - La serie crece agregando nuevos alumnos multiplicados por los "pesos de refuerzo".

- Acerca de los algoritmos de conjunto
  - Hacer una toma de decisiones más adecuada combinando adecuadamente múltiples opiniones obtenidas por diferentes expertos
  - Voto mayoritario: análisis que utiliza muchos modelos de clasificación diferentes con un conjunto de entrenamiento idéntico
  - Embolsado (conjunto de entrenamiento diferente)
    - arranque + agregación
    - Bootstrap + agregación
    - Ensacado (muestreo aleatorio con reemplazo (arranque aleatorio))

      Ex Random Forest

  - boosting
    - Retener el 50% de los datos clasificados incorrectamente o utilizar ponderación para la selección de muestras.

- bootstrap
  - Estimación de estadísticas desconocidas.
  - Método de estimación fácil y eficaz con distribución desconocida de la muestra de parámetros del modelo.
  - Proceso de recalcular las estadísticas y el modelo para cada muestra mediante muestreo adicional con reemplazo de la muestra actual.
    - No se requiere ninguna suposición de que los parámetros o las estadísticas de la muestra deban estar en una distribución normal.
  - muestra de arranque: agregación de datos observados (muestreo con reemplazo obtenido del valor del registro y la variable dependiente)

Contenidos a aprender en el futuro.

- Remuestreo: implica submuestreo de permutación sin reemplazo
- Agregación Bootstrap: obtener un resultado al agregar valores predichos obtenidos de diferentes muestras de bootstrap.

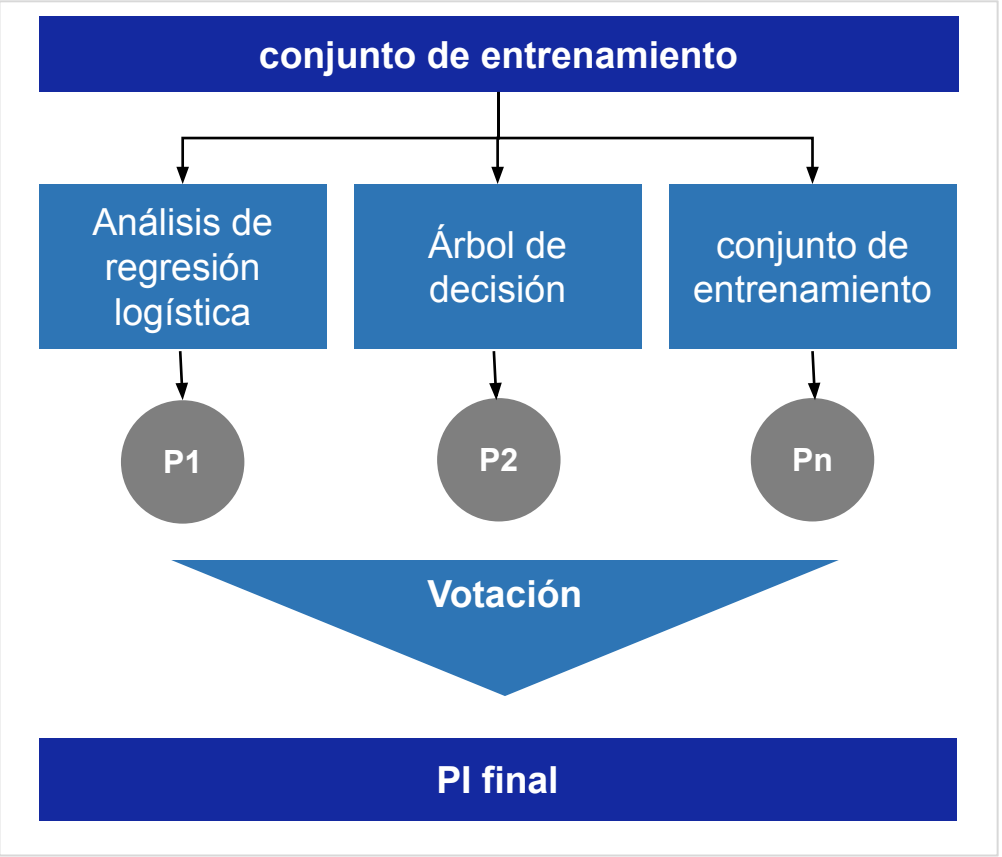
- ¿Qué es el aprendizaje en conjunto?
  - El término "conjunto" se puede definir de la siguiente manera:
    - In En mecánica estadística, un conjunto de un sistema se refiere a la colección de sistemas equivalentes.
  - En otras palabras, es una asamblea de grupos similares.
  - En lugar de esperar resultados de rendimiento de un solo modelo, el aprendizaje conjunto obtiene mejores resultados utilizando la inteligencia colectiva de diferentes modelos, como promediando muchos modelos individuales diferentes o tomando una decisión basada en el voto mayoritario.
  - Existen muchos métodos de conjunto diferentes que utilizan la inteligencia colectiva.
    - Voting - obtención de resultados mediante votación
    - Bagging - agregación Bootstrap (creación duplicada de varias muestras)
    - Boosting – Ponderación mediante la compensación de errores previos
    - Stacking - Un metamodelo basado en diferentes modelos
  - Puede haber otros métodos más diferentes, ya que el aprendizaje en conjunto aplica una determinada técnica/metodología. Sin embargo, los cuatro métodos enumerados anteriormente son las técnicas de conjunto más representativas en sklearn library.

Conjunto de votación

- conjunto de votación
  - Se puede aplicar a clasificación y regresión.
  - Los alumnos que forman un conjunto deben ser de diferentes tipos para una buena interpretación.
  - Dos métodos de votación para el clasificador:
    - Difícil: la etiqueta de clase prevista viene dada por la votación por regla mayoritaria.
    - Suave: la etiqueta de clase predicha viene dada por el argmax de la suma de las probabilidades predichas.

- Votación
  - Como la palabra misma, la votación toma una decisión a través de los votos. La votación es similar al embolsado ya que utiliza un método de votación, pero se diferencian mucho entre sí de la siguiente manera:
    - Votación: Combina diferentes modelos de algoritmos.
    - Bagging: Utiliza diferentes combinaciones de muestras dentro del mismo algoritmo.
  - La votación selecciona los resultados finales haciendo que la votación final se deduzca mediante diferentes algoritmos.
  - La votación se clasifica en votación dura y votación blanda.
    - Votación dura: Decide el valor final del resultado mediante votación.
    - Votación suave: extrae el valor final sumando todos los valores de probabilidad de obtener el resultado final y luego calculando cada probabilidad del resultado final.

- Votación



- Utilice un único conjunto de entrenamiento.
- Utilice diferentes modelos de clasificación.
- Valor predictivo
- Si los valores predictivos de cada modelo de análisis son diferentes de la votación, elija el resultado con más valores.
  - votación dura
  - votación suave
  - Predecir la clase más alta promediando la predicción del clasificador individual.

- Votación
  - Voto duro
    - Tomando como ejemplo la clasificación. Supongamos que los valores predictivos para la clasificación son 1,0,0,1,1 ya que 1 tiene tres votos y 0 tiene 2 votos. En ese caso, 1 se convierte en el valor predictivo final en el método de votación dura.
  - Voto suave
    - El método de votación suave calcula el valor promedio de cada probabilidad y luego determina cuál tiene la mayor probabilidad.
    - Supongamos que la probabilidad de obtener la clase 0 es (0.4, 0.9, 0.9, 0.4, 0.4), y la probabilidad de obtener la clase 1 es (0.6, 0.1, 0.1, 0.6, 0.6). La probabilidad final de obtener la clase 0 es (0.4 + 0.9 + 0.9 + 0.4 + 0.4) / 5 = 0.6; la probabilidad final de obtener la clase 1 es (0.6 + 0.1 + 0.1 + 0.6 + 0.6) / 5 = 0.4. Por lo tanto, el valor final seleccionado es diferente al resultado de la votación dura (hard voting) anterior
    - En general, utilizar el método de votación suave se considera más razonable que el método de votación dura en las competiciones porque el método de votación suave proporciona un resultado de rendimiento real mucho mejor.

- Conjunto de votación en scikit-learn
  - Para importar el conjunto de votación como clase:

```
from sklearn.ensemble import VotingClassifier # Para clasificación
sklearn.ensemble import VotingRegressor       # Para regresión
```

  - Para crear una instancia de un objeto que implemente un conjunto de votación:

Ex

```
myKNN = KNeighborsClassifier(n_neighbors = 3)
myLL = LogisticRegression()
myVotingEnsemble=VotingClassifier(estimators=[('lr',myLL),('knn',myKNN)],voting='hard')
```

- Conjunto de votación en scikit-learn
  - Podemos entrenar y predecir como cualquier otro estimador:

    Ex myVotingEnsemble.fit(X_train, Y_train)

    myVotingEnsemble.predict(X_test)

- Hiperparámetros del clasificador/regresor de votación de scikit-learn:

Hiperparámetro	Explicación
estimators	La lista de objetos básicos de aprendizaje.
voting	Ya sea 'suave' o 'duro' (solo para clasificador)

  - Puede encontrar más información en:
  - Clasificador: https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.VotingClassifier.html Regresor: https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.VotingRegressor.html

Unidad 8.

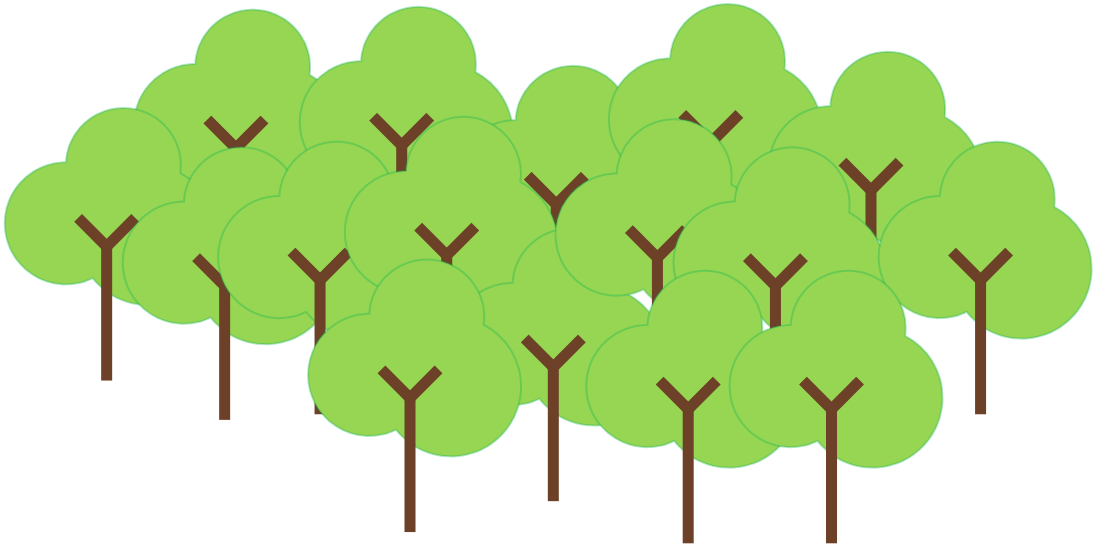
Algoritmo de conjunto

- 8.1. El concepto de algoritmo de conjunto y votación
- 8.2. Bagging & Random Forest
- 8.3. Boosting

Conjunto de embolsado: bosque aleatorio

- Acerca del bosque aleatorio
  - Un algoritmo de conjunto basado en árboles.
  - Se puede aplicar a clasificación y regresión.
- Ventajas
  - Poderoso
  - Pocas suposiciones
  - Poca o ninguna preocupación por el problema del sobreajuste
- Contras
  - La formación requiere mucho tiempo.

- Algoritmo de bosque aleatorio



- Los árboles elegidos al azar forman un bosque.
- La predicción se decide por mayoría de votos.

- Algoritmo de bosque aleatorio

1) Hacer árboles con variables y observaciones seleccionadas al azar.

2) Conserve sólo aquellos con la impureza (o entropía) de Gini más baja.

3) Repita desde el paso 1) un número determinado de veces.

4) Usando los árboles recolectados durante el paso de entrenamiento, podemos hacer predicciones por mayoría de votos.

- Hiperparámetros de RandomForestClassifier/Regresor de scikit-learn:

Hiperparámetro	Explicación
n_estimators	La cantidad de árboles en el bosque.
max_depth	La profundidad máxima de un árbol.
min_samples_leaf	El número mínimo de puntos de muestra necesarios para estar en un nodo hoja.
min_samples_split	El número mínimo de puntos de muestra necesarios para dividir un nodo interno.
max_features	La cantidad de características a considerar al buscar la mejor división

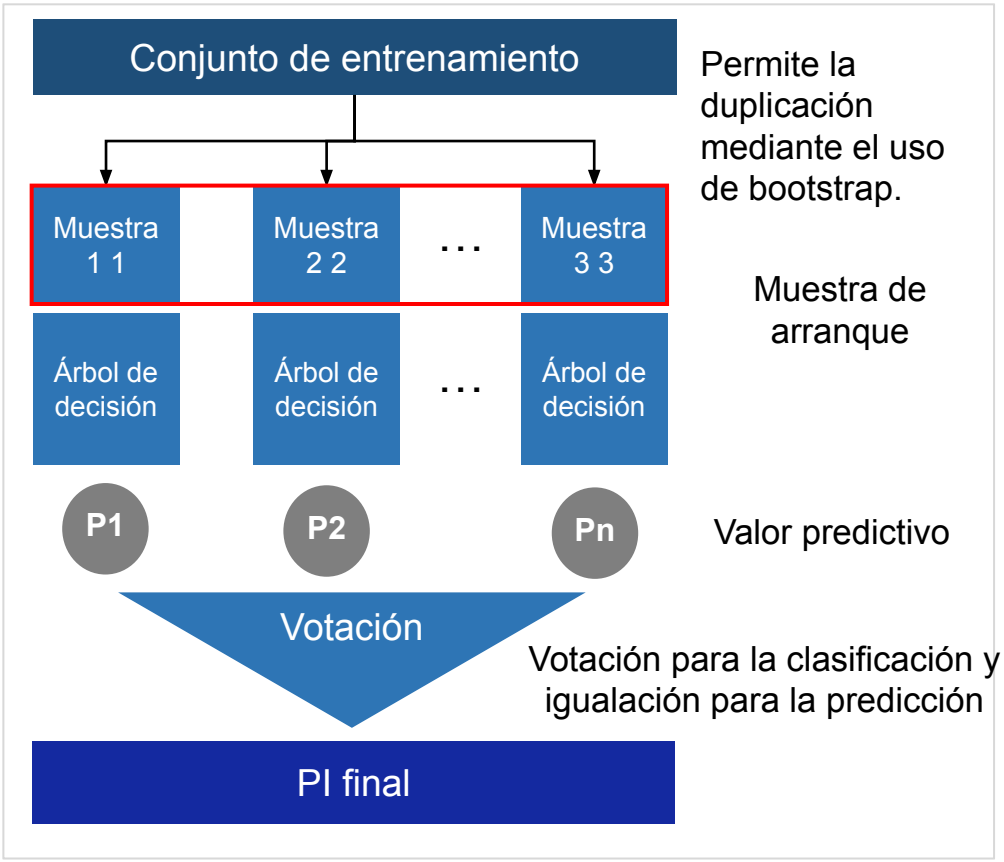
- Excepto "n_estimators", el resto son análogos a los de DecisionTreeClassifier/Regressor.
- Puede encontrar más información en:
- Clasificador: https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html
- Regresor: https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestRegressor.html

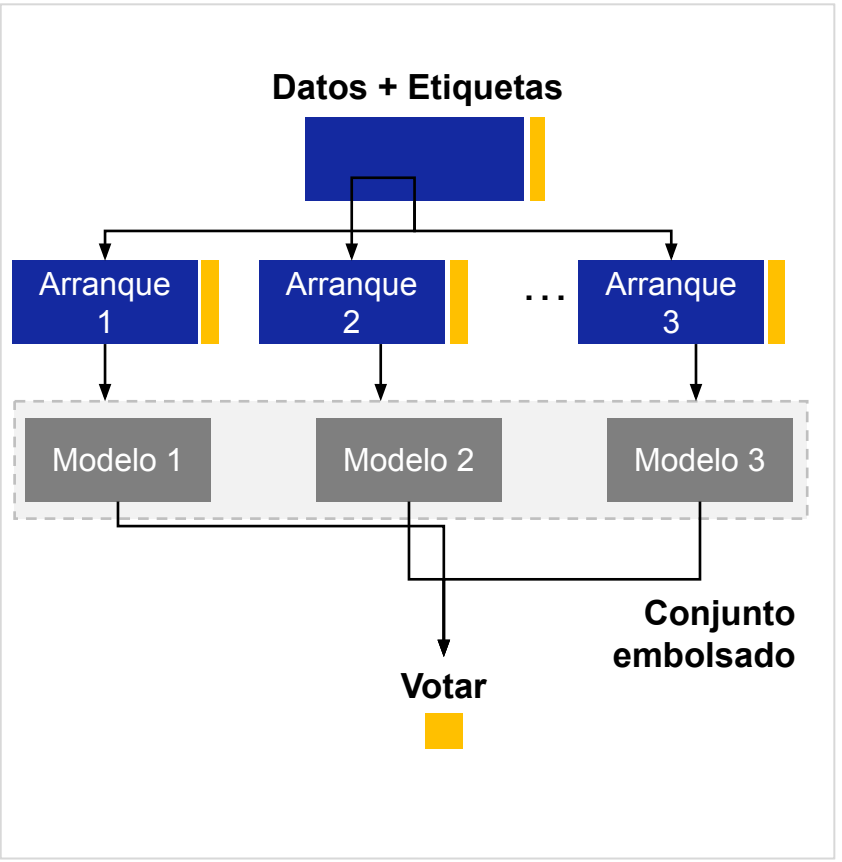
- Harpillera
  - Método de conjunto basado en embolsado

    Ex Algoritmo de bosque aleatorio
    - Fácil de usar ya que está bien construido en Sklearn library
    - Velocidad de rendimiento relativamente rápida
    - Rendimiento alto

  - El método de conjunto se ha utilizado ampliamente porque eleva el nivel de interpretación y es fácil de usar. El método de conjunto basado en embolsado se encuentra comúnmente en las soluciones de alto rango en Kaggle..

- Harpillera


Conjunto de entrenamiento
Muestra 1 1
Muestra 2 2
...
Muestra 3 3
Árbol de decisión
Árbol de decisión
...
Árbol de decisión
P1
P2
Pn
Votación
PI final
Permite la duplicación mediante el uso de bootstrap.
Muestra de arranque
Valor predictivo
Votación para la clasificación y igualación para la predicción


Datos + Etiquetas
Arranque 1
Arranque 2
...
Arranque 3
Modelo 1
Modelo 2
Modelo 3
Votar
Conjunto embolsado

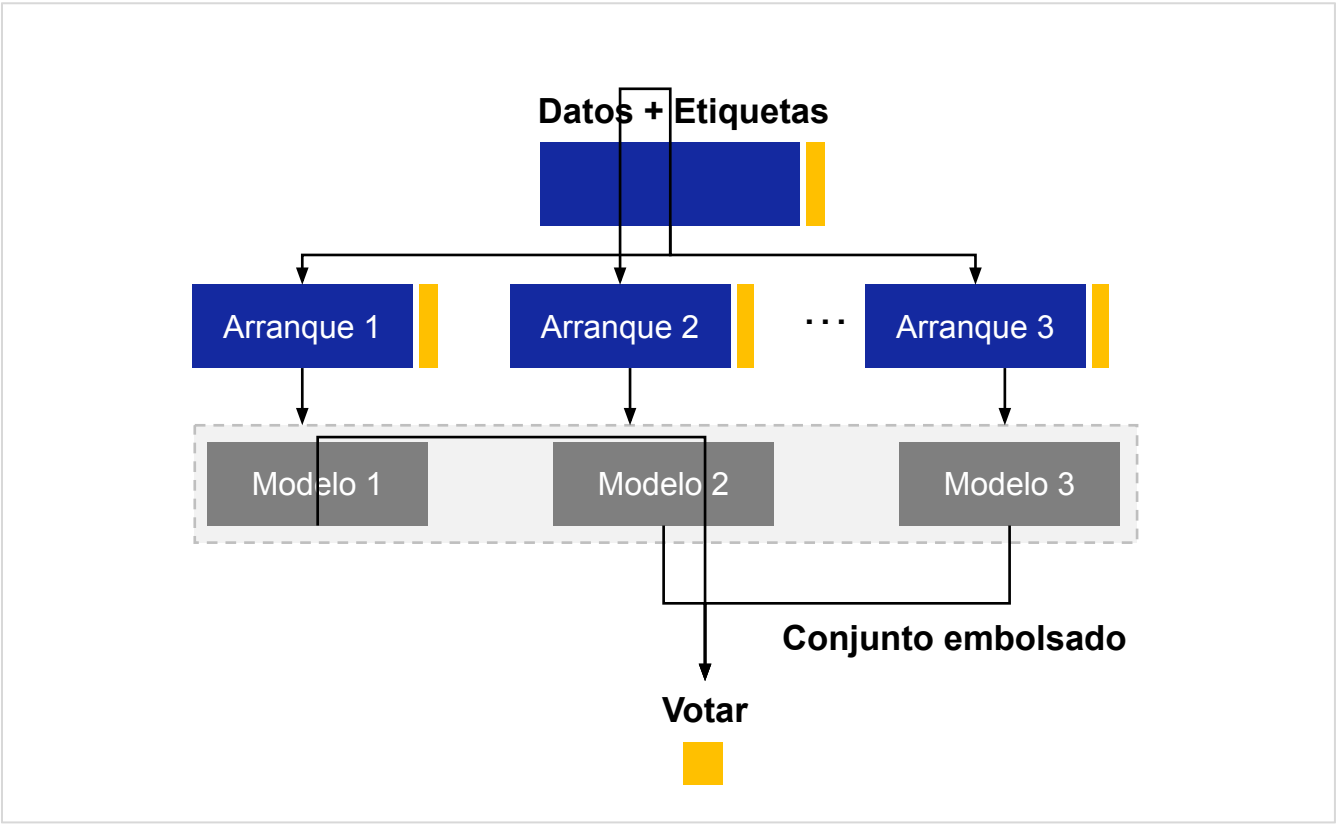
- Harpillera
  - Bagging es una abreviatura de Bootstrap Aggregating.
  - Bootstrap = Sample
  - Aggregating = Adding up
  - Bootstrap se refiere a un método que permite la superposición de diferentes conjuntos de datos para muestreo y división.

- Harpillera

Ex Random Forest es un algoritmo típico de método de ensacado.

- Crea múltiples árboles de decisión y realiza muestreos de diferentes conjuntos de datos al tiempo que permite conjuntos de datos superpuestos.
- Si el conjunto de datos consiste en [1, 2, 3, 4, 5]:
  - Grupo 1 = [1, 2, 3]
  - Grupo 2 = [1, 3, 4]
  - Grupo 3 = [2, 3, 5]
- Este es el método de arranque. En el problema de clasificación, la votación se realiza en cada árbol entrenado con un muestreo diferente para obtener el resultado final de la predicción.
- En los problemas de regresión se calcula la media de cada valor obtenido.

- Harpillera
  - “Bagging”: Bootstrap AGGregatING



- Diferencias entre embolsar(Bagging) y votar(Voting)
  - La mayor diferencia entre los métodos de embolsado y votación es si se utilizan varios algoritmos únicos o se aplican varios algoritmos al mismo conjunto de datos de muestra.
  - En general, el método de fanfarronear consiste en entrenar un único algoritmo con diferentes conjuntos de datos de muestreo y realizar una votación. Tiene una usabilidad relativamente mejor que el método de votación porque utiliza un único algoritmo. Por tanto, lo importante es el hiperparámetro del algoritmo único.
  - Tomando nuevamente el algoritmo Random Forest como ejemplo, es adecuado para obtener la puntuación de referencia, ya que requiere una configuración de hiperparámetros simple, como cuántos árboles usar (n_estimators), profundidad máxima (max_profundidad) y el número mínimo de muestras para dividir (min_samples_leaf).

- Ventajas del conjunto de ensacado
  - Cuando se utiliza el método de ensacado, se puede reducir la variación en comparación con hacer una predicción con un solo modelo. Hay tres errores principales de entrenamiento en un modelo: varianza, ruido y sesgo. (Por supuesto, los factores más importantes incluyen el sobreajuste o el desajuste y muchos problemas de preprocesamiento diferentes, pero se supone que ya se están configurando).
  - El método de conjunto reduce la variación, mejorando así el rendimiento del resultado final.

- sklearn.ensemble.BaggingClassifier/BaggingRegressor/
  - El paquete sklearn library proporciona la clase contenedora llamada BaggingClassifier/BaggingRegressor.
  - Al designar el algoritmo base para el parámetro base_estimator, BaggingClassifier/BaggingRegressor realiza el conjunto de ensacado.

Unidad 8.

Algoritmo de conjunto

- 8.1. El concepto de algoritmo de conjunto y votación
- 8.2. Ensacado y bosque aleatorio
- 8.3. boosting

Conjunto de impulso: AdaBoost

- Acerca de AdaBoost
  - Una secuencia de alumnos débiles como los árboles.
  - Un conjunto ponderado de alumnos débiles
    - Un conjunto de ponderaciones que aumentan la importancia de los estudiantes con mejor desempeño
    - Un conjunto de ponderaciones que aumentan la importancia de las observaciones mal clasificadas.
  - Pros y contras similares a los del Random Forest

- Algoritmo de clasificación AdaBoost
  - Supongamos n observaciones para el paso de entrenamiento:: $\boldsymbol{x}_i$ and y_i.
    También suponemos que $y_i \in \{-1,+1\}$. (y binaria)
  - Crearemos una serie de aprendices débiles $G_m(\boldsymbol{x})$ con $m = 1,\ldots,M$.
  - El clasificador de conjuntos se compone de una combinación lineal de estos alumnos débiles:

$$G_{ensemble}(\boldsymbol{x}) = sign\left(\sum_{m=1}^{M} \alpha_m G_m(\boldsymbol{x})\right)$$

donde α_m son los "pesos de boosting" que deben ser calculados

  - Se le da más peso a un alumno con mejor desempeño.

- Algoritmo de clasificación AdaBoost

1) Para el primer paso (m=1), se asigna el mismo peso a las observaciones: $w_i^{(1)}=\frac{1}{n}$

2) Para la secuencia de boost m=1,…, M:

a) Entrene al alumno Gm ($\boldsymbol{x}$) usando observaciones ponderadas por $wi(m).G_m$ ($\boldsymbol{x}w_i^{(m)}$.

b) Calcule la tasa de error: $$\varepsilon_m = \frac{\sum_{i=1}^{n} w_i^{(m)} I(y_i \neq G_m(\boldsymbol{x}_i))}{\sum_{i=1}^{n} w_i^{(m)}}$$

⇨ $I(y_i \neq G_m(\boldsymbol{x}_i))$ da 1 para una predicción incorrecta, de lo contrario 0.

⇨ $0 \leq \varepsilon_m \leq 1$

- Algoritmo de clasificación AdaBoost

3) Para la sequencia boost m=1,..., M:

c) Calcula el peso del boosting: α_m

$$\alpha_m = \frac{1}{2} log\left(\frac{1-\varepsilon_m}{\varepsilon_m}\right)$$

⇨ A medida que $\varepsilon_m \to 0$, α_m alpha_m es un número positivo grande. ¡Se le da más importancia al aprendiz!

⇨ A medida que $\varepsilon_m \cong 0.5$, $\alpha_m \cong 0$.

⇨ A medida que $\varepsilon_m \to 1$, α_m es un número negativo grande.

d) Para el siguiente paso, los pesos de la observación predicha erróneamente se reescalan en un factor e^{a}_{m}.

Esto se puede expresar de forma compacta como:

$$w_i^{(m+1)} = w_i^{(m)} \times e^{\alpha_m \cdot I\left(y_i \neq G_m(\boldsymbol{x}_i)\right)}, \text{ where } i = 1, \ldots, n$$

⇨ En el siguiente paso de la secuencia, las observaciones predichas erróneamente reciben mayor peso.

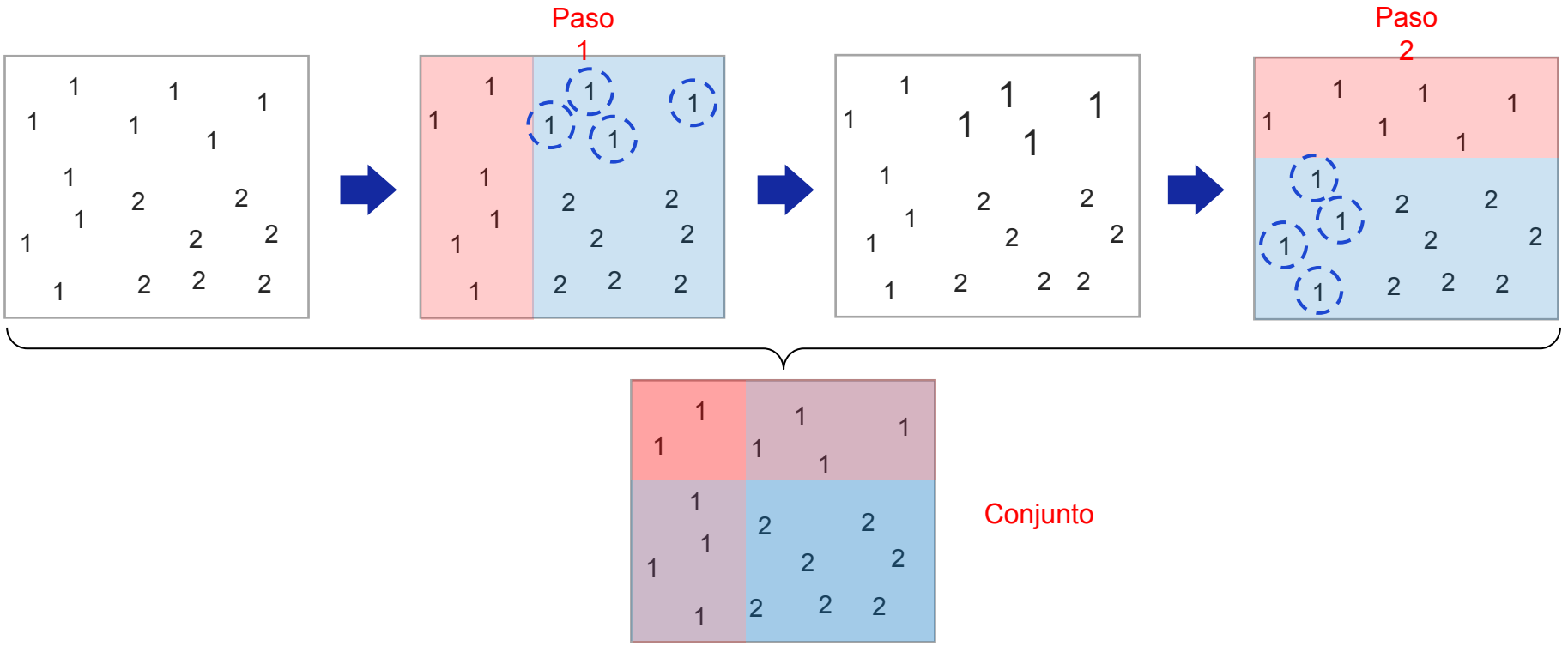
- Algoritmo de clasificación AdaBoost

4) El clasificador de conjuntos se compone de una combinación lineal de alumnos débiles:

$$G_{ensemble}(\boldsymbol{x}) = sign\left(\sum_{m=1}^{M} \alpha_m G_m(\boldsymbol{x})\right)$$

5) Para una nueva condición de prueba $\boldsymbol{x}'$, podemos predecir y' mediante $G_ensemble\ (\boldsymbol{x}').\boldsymbol{x}y'$ by $G_ensemble\ (\boldsymbol{x}')$.

- Algoritmo de clasificación AdaBoost



- Hiperparámetros de AdaBoostClassifier/Regresor de scikit-learn/

Hiperparámetro	Explicación
base_estimator	El estimador base con el que se construye el conjunto potenciado.
n_estimators	El número máximo de estimadores en los que finaliza el impulso.
learning_rate	La tasa a la que se reduce la contribución de cada alumno.
algorithm	Ya sea 'SAMME' o 'SAMME.R'

- “base_estimator” es por default None, lo que significa DecisionTreeClassifier(max_depth=1).
- Puede encontrar más información en: Clasificador: https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.AdaBoostClassifier.html Regresor: https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.AdaBoostRegressor.html

Conjunto de impulso: GBM y XGBoost

- Máquina de aumento de gradiente (GBM)
  - GBM también se compone de una secuencia de aprendices débiles (weak learners) similar a AdaBoost.
  - La discrepancia entre el valor predicho y_hat y el valor real y puede representarse mediante una función de "pérdida" (loss function).
  - En la secuencia de aprendices débiles, el aprendiz en el paso m,, $G_m(\boldsymbol{x})$ puede obtenerse moviendo al aprendiz del paso anterior $G_{m-1}(\boldsymbol{x})$ hacia la dirección que disminuye la pérdida definida anteriormente.
  - Estos movimientos de actualización están restringidos a un conjunto de aprendices base (base learners).

- scikit-learn Hiperparámetros de clasificador/regresor de aumento de gradiente

Hiperparámetro	Explicación
pérdida	La función de pérdida
n_estimadores	El número de pasos de impulso (alumnos débiles)
tasa_de_aprendizaje	La contribución de cada alumno débil
submuestra	La fracción de datos que utilizará el alumno débil individual.

- Es necesario ajustarlo para obtener un rendimiento optimizado.
- Puede encontrar más información en:
- Clasificador: https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.GradientBoostingClassifier.html
  Regresor: https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.GradientBoostingRegressor.html

- Aumento de gradiente extremo (XGBoost))
  - Mejora a GBM en la velocidad de ejecución.
  - Más resistente al sobreajuste que GBM.
  - No incluido en scikit-learn library. Requiere la instalación del “xgboost” library.

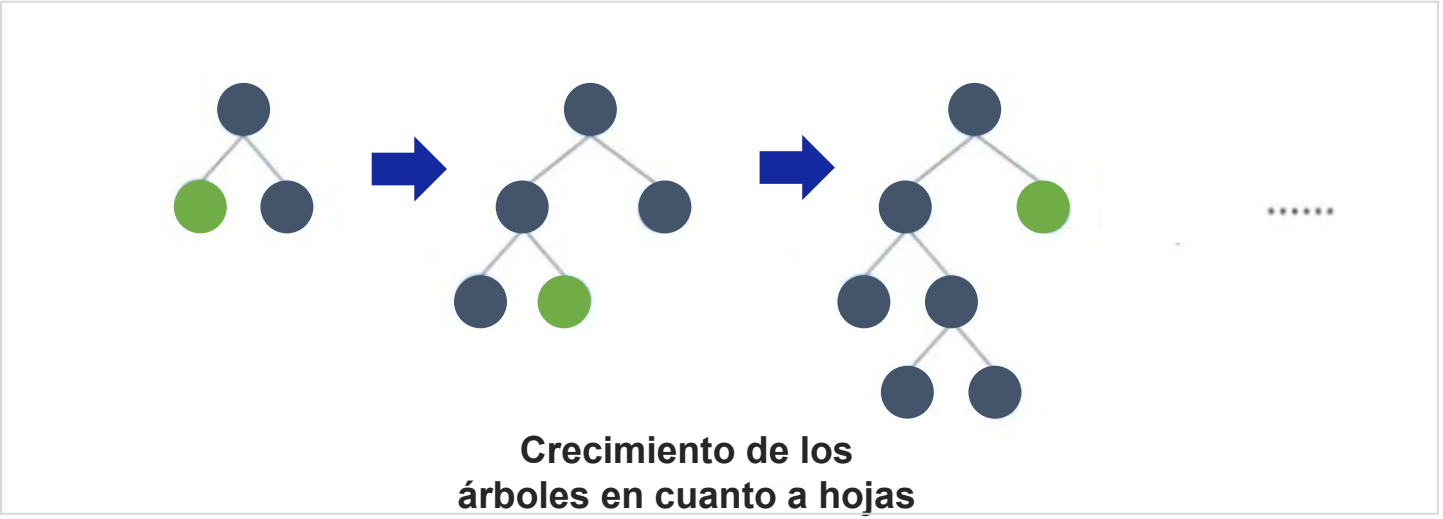
- Hiperparámetros del clasificador/regresor XGB

Hiperparámetro	Explicación
refuerzo	gbtree o gblineal or
n_estimadores	El número de pasos de impulso (alumnos débiles)
tasa_de_aprendizaje	La contribución de cada alumno débil
submuestra	La fracción de datos que utilizará el alumno débil individual.

- Es necesario ajustarlo para obtener un rendimiento optimizado.
- Puede encontrar más información en: https://xgboost.readthedocs.io/en/latest/python/index.html

- XGBoost
  - XGBoost es un library que implementa el algoritmo de aumento de gradiente que se utilizará en el sistema distribuido.
  - Admite problemas tanto de regresión como de clasificación. Este algoritmo de uso popular presenta un buen rendimiento y eficiencia de recursos.
  - El aumento de gradiente es un algoritmo representativo que utiliza el método de aumento. XGBoost es el library que utiliza este algoritmo para admitir el entrenamiento paralelo.
  - Se ha utilizado mucho recientemente debido a su alto rendimiento y tasa de aplicación de recursos informáticos. Se ha vuelto más popular ya que lo usaban con frecuencia los mejores clasificados de Kaggle.

- GBM ligero
  - Mientras que el árbol se expande verticalmente con Light GBM, otros algoritmos expanden el árbol horizontalmente. En otras palabras, mientras que Light GBM es en hojas, otros algoritmos lo son en niveles.
  - Para la expansión, se selecciona la hoja con máxima pérdida delta. Al expandir la misma hoja, el algoritmo de hoja puede reducir más pérdidas que el algoritmo de nivel.
  - El siguiente diagrama muestra cómo se implementa un algoritmo de impulso, que difiere del Light GBM.

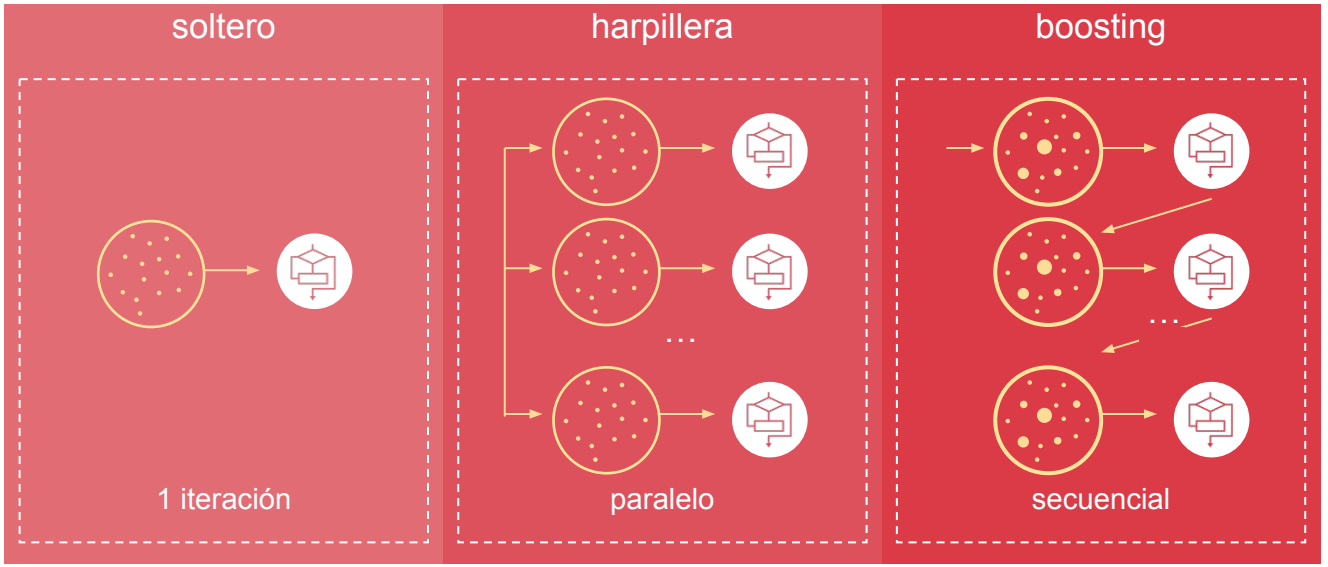


Crecimiento de los árboles en cuanto a hojas

Método de operación GBM ligero

- boosting
  - El algoritmo de refuerzo también es aprendizaje conjunto. Después de aprender secuencialmente máquinas de aprendizaje débiles, complementa los errores agregando peso a los datos predichos incorrectamente del aprendizaje anterior.
  - La diferencia con otros métodos de conjunto es que realiza un aprendizaje secuencial y complementa los errores agregando peso. Sin embargo, una de las desventajas es que es difícil procesar en paralelo debido a su propiedad secuencial, lo que lleva a un tiempo de aprendizaje más largo en comparación con otros conjuntos.

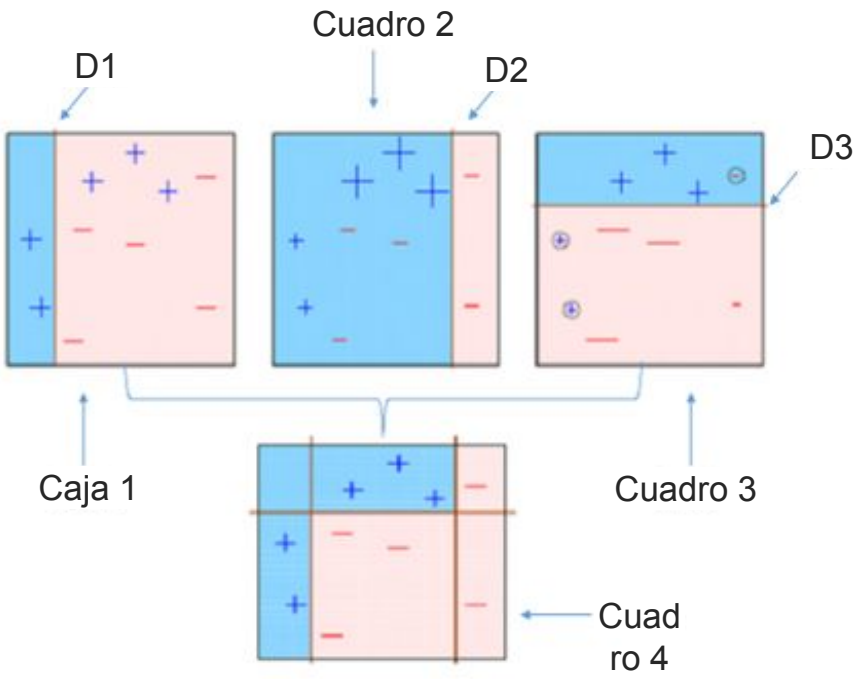
Estimador único, ensacado, impulso



- Aprender una serie de predictores complementando modelos anteriores.
  - AdaBoost
    - Entrene al primer distribuidor (por ejemplo, árbol de decisión) en el conjunto de entrenamiento y haga una predicción.
    - Aumente ligeramente el peso de la muestra de entrenamiento con un algoritmo clasificado de manera inexacta.
    - El segundo distribuidor utiliza el peso actualizado para entrenar en el conjunto de entrenamiento y vuelve a hacer una predicción.
    - El peso se actualiza nuevamente y se repite el mismo proceso.
  - Aumento de gradiente
    - De manera similar a AdaBoost, el aumento de gradiente agrega secuencialmente predictores al conjunto para corregir los errores anteriores.
    - Sin embargo, en lugar de modificar el peso de la muestra repetidamente como AdaBoost, entrena un nuevo predictor al error residual creado por el predictor anterior.
    - Para problemas de regresión: árbol de regresión potenciado por gradiente (GBRT)

- AdaBoost (impulso adaptativo)
  - AdaBoost (Adaptive Boosting) es un método de impulso adaptativo, uno de los algoritmos de impulso más representativos.
  - Como se explicó, AdaBoost aprende secuencialmente máquinas de aprendizaje débiles y complementa los errores aplicando ponderaciones a datos predichos de manera inexacta.

- Principio de AdaBoost



- El cuadro 1 es el resultado de una máquina de aprendizaje débil clasificada como sector D1. Sin embargo, la tasa de error es relativamente alta porque algunos conjuntos de datos se expresan en + distribuidos en el sector rojo.
- La línea D2 en el Cuadro 2 se mueve hacia la derecha para complementar la tasa de error del Cuadro 1. Aquí, los conjuntos de datos expresados en – se distribuyen en el sector azul para un mejor rendimiento, pero aún no es satisfactorio.
- La línea D3 en el Cuadro 3 está dibujada horizontalmente en la parte superior. Sin embargo, el conjunto de datos expresado en – está clasificado incorrectamente.
- Al entrenar los Cuadros 1, 2 y 3 anteriores, el Cuadro 4 encuentra la combinación más ideal. Muestra un rendimiento mucho mejor en comparación con las tres máquinas de aprendizaje individuales anteriores.

- Principio de AdaBoost
  - ¿Cómo se aplica el peso para la combinación?

    Ex Supongamos que se aplicará la siguiente ponderación al desempeño de los Cuadros 1 a 3.

    - Performance of Box 1: weight = 0.2
    - Performance of Box 2: weight = 0.5
    - Performance of Box 3: weight = 0.6

    Se puede expresar como la siguiente fórmula:

    0.2 * Box 1 + 0.5 + Box 2 + 0.6 * Box 3 = Box 4

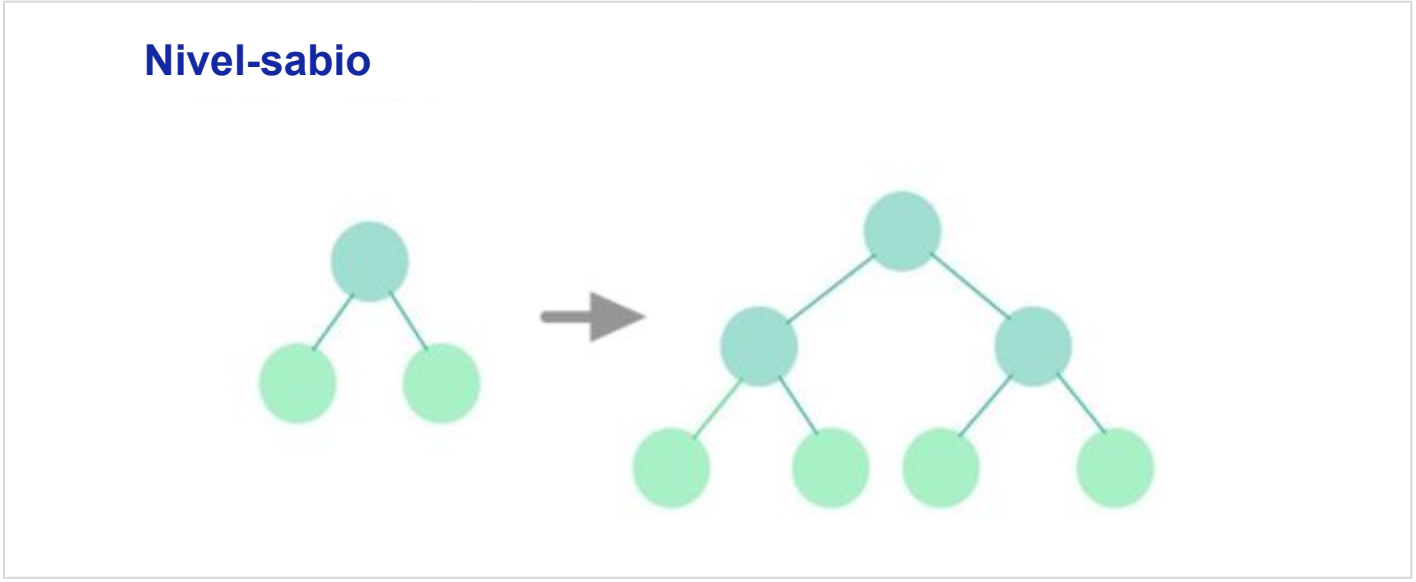
Descenso de gradiente

- La clave del método de refuerzo es complementar los errores del aprendizaje anterior.
- Los métodos AdaBoosting y descenso de gradiente son ligeramente diferentes en cómo complementar los errores. to
- El descenso de gradiente utiliza la diferenciación para minimizar la diferencia entre el valor predicho y los datos reales.
  - Peso
  - Input_data = feature data (input data)
  - Inclinación
  - Y_actual = actual data value
  - Y_predict = predicted value
  - Loss = error rate
- Y_predict = weight*input_data+bias
  - El valor predictivo se puede obtener a partir de la fórmula anterior. Calcular la diferencia con los datos reales dará como resultado la tasa de error total.
- Loss = Y_predict – Y_actual
  - (Existen muchas fórmulas funcionales diferentes para definir la tasa de error, incluida la raíz del error cuadrático medio y el error absoluto, pero la definición anterior se proporciona por conveniencia).
  - El propósito del descenso de gradiente es encontrar el peso que hace que la pérdida sea más cercana a 0.

- Máquina de aumento de gradiente (GBM)
  - El algoritmo de impulso en el descenso de gradiente se denomina máquina de impulso de gradiente, abreviado como GBM. Se proporciona en el paquete sklearn y es aplicable tanto a problemas de clasificación como de regresión.
    - Clasificador de impulso de gradiente
    - Regresor de impulso de gradiente
  - Es extremadamente fácil de aplicar en problemas reales.
  - XGBoost y LightGBM son dos paquetes de aprendizaje automático importantes que se utilizan principalmente en Kaggle.
  - XGBoost y LightGBM no se proporcionan en el paquete sklearn existente. Sin embargo, la clase contenedora sklearn permite una aplicación sencilla de ajuste y predicción similar a la clase ML del paquete sklearn existente.

- Máquina de aumento de gradiente (GBM)
  - Junto con XGBoost, LightGBM es el algoritmo de impulso más destacado. Aunque XGBoost tiene un rendimiento excelente, su tiempo de aprendizaje es demasiado largo.,
  - Ventajas del LightGBM
    - Corto tiempo de aprendizaje
    - Uso de memoria relativamente pequeño
    - Conversión automática y división óptima de características categóricas.

- Características del LightGBM
  - El algoritmo basado en árbol existente utiliza un método de nivel. Se divide pero mantiene un árbol equilibrado tanto como sea posible, por lo que la profundidad del árbol se vuelve mínima. Una desventaja del método de niveles es que lleva tiempo crear un árbol equilibrado.
  - Por otro lado, LightGBM utiliza un método de hoja. No equilibra el árbol, sino que divide continuamente el nodo hoja con la máxima pérdida de datos. La profundidad del árbol aumenta y se crea un árbol asimétrico. La repetición del nodo hoja con la máxima pérdida de datos minimiza la pérdida de error prevista que dividir un árbol equilibrado.



- Métodos de ajuste de hiperparámetros
  - El valor aumentado de Num_leaves aumenta la precisión. Sin embargo, también aumenta la profundidad del árbol y hace que el modelo sea complejo, aumentando así la probabilidad de sobreajuste.
  - Min_data_in_leaf es un parámetro importante para el sobreajuste. Depende de Num_leaves y del tamaño de los datos de entrenamiento, pero en general, evita un árbol más profundo cuando se establece como un valor más alto.
  - Max_ Depth limita el tamaño de profundidad y mejora el sobreajuste cuando se combina con los dos parámetros anteriores.

Parámetro	Valor predeterminado	Descripción
num_iterations	100	Designa el número de árboles para trabajos repetitivos. El sobreajuste se produce si el valor es demasiado alto.
learning_rate	0.1	Se actualiza cuando se realizan repetidamente pasos de impulso. El valor se designa entre 0~1.
max_depth	1	Idéntico a la profundidad máxima de los algoritmos basados en árboles. No se aplica ninguna restricción a la profundidad del árbol al ingresar un valor menor que 0.
min_data_in_leaf	20	Idéntico al min_samples_leaf de un árbol de decisión. Se utiliza como parámetro para controlar el sobreajuste.
num_leaves	3	Significa el máximo. El número de hojas de un árbol.
boosting	GBDT	
bagging_fraction	1.0	Designa la proporción para el muestreo de datos. Se utiliza para controlar el sobreajuste.
feature_fraction	1.0	La proporción de características aleatorias para el aprendizaje de árboles individuales. r
Lambda_l1	0.0	El valor para la regulación L1
Lambda_l2	0.0	El valor para la regulación L2